KH정보교육원

인터페이스 구현

CONTENTS

Part 1 인터페이스 설계서 확인하기

Chapter 1. 프레임워크 개요 및 Spring 프레임 워크 4

Chapter 2. MyBatis 소개 및 설치 8

Chapter 3. MyBatis 설치 및 환경설정 18

Part 2 인터페이스 기능 구현하기

Chapter 4. MyBatis Mapper 와 element 21

Chapter 5. MyBatis 페이징 처리 40

Chapter 6. ResultMap 52

Chapter 7. API 67

Part 3 인터페이스 구현 검증하기

Chapter 8. SqlSessionTemplate 193

Chapter 9. AOP Transaction 관리 194

Part 1. 인터페이스 설계서 확인하기

Chapter 1. 프레임워크 개요 및 Spring 프레임 워크

1 프레임워크란?

소프트웨어 프레임워크(software framework)는 복잡한 문제를 해결하거나 서술하는데 사용되는 기본 개념 구조이다. 간단히 뼈대, 골조, 프레임워크 라고도 한다. 소프트웨어 개발에 있어 하나의 뼈대 역할을 한다

1.1 라이브러리와의 차이점

라이브러리와 프레임워크를 헷갈리는 경우도 있는데 보통 프레임워크라고 하면 클래스와 라이브러리가 합쳐진 형태라고 볼 수 있다.
메소드 및 클래스화, 즉 모듈화를 는 이유 중 하나가 재사용성인데, 프레임워크는 이 재사용성을 큰 그룹 단위로 묶어주었다고 보면 된다. 재사용 가능한 수많은 클래스들과 라이브러리들을 융합한 채로 처음부터 제공해 주기 때문에, 여러 개의 표준을 만들지 않아도 돼서 개발자의 피곤함을 덜어준다.

1.2 종류

- Spring : JAVA의 웹 프레임워크
- Apache 시리즈 (Sling, Structs 2, Wicket 등)
- JWT
- Django: Python 의 웹 프레임워크
- Flask: Python 의 마이크로 웹 프레임워크
- Ruby on Rails: Ruby 의 웹 프레임워크
- Laravel, Codeigniter, CakePHP, Symfony, Zend-Skeleton: PHP 의 웹 프레임워크
- Bootstrap: 트위터의 jQuery 기반 웹 프레임워크
- .NET Framework
- Qt
- Node.js 의 Express.js 프레임워크
- 웹 프론트엔드의 SPA 용 프레임워크 (AngularJS, Angular, Vue.js 등)
- 안드로이드: 안드로이드에서 구동되는 안드로이드 앱용 프레임워크

2 Spring이란?

스프링 프레임워크는 엔터프라이즈급 애플리케이션을 만들기 위한 많은 기능을 제공하는 솔루션이다. 필요한 부분만 가져다 사용할 수 있도록 모듈화되어 있다. 자바 애플리케이션 개발을 위한 포괄적인 하부 구조를 제공하는 자바 플랫폼이며 애플리케이션에 집중할 수 있도록 하부 구조를 다룬다.

EJB의 단점을 보완한 프레임 워크이며 DI(의존성 주입)와 AOP(관점 지향 프로그래밍)가 지원되는 경량 컨테이너이며 프레임워크이다.

3 Spring의 주요 기능

3.1 Spring DI(Dependency Injection) – 의존성 주입

Spring Framework는 설정 파일이나 어노테이션을 통해 객체간 의존 관계를 설정하여 직접 의존하는 객체를 생성할 필요가 없다.

3.2 Spring AOP(Aspect Oriented Programming) – 관점 지향 프로그래밍

Database의 트랜잭션처리나 로깅처리와 같이 여러 모듈에서 공통으로 필요로 하는 기능의 경우 해당 기능을 분리하여 관리한다.

3.3 Spring JDBC – 영속성 프레임워크 지원

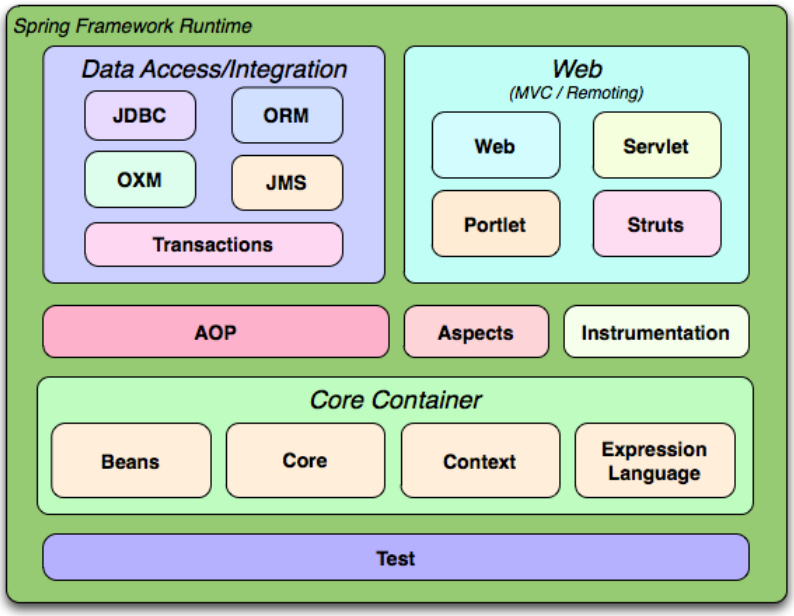
Mybatis나 Hibernate 등의 데이터베이스를 처리하는 라이브러리와 연결할 수 있는 인터페이스를 제공한다.

3.4 Spring MVC

Spring MVC 디자인 패턴을 통해 웹 어플리케이션의 Model, View, Controller 사이의 의존 관계를 DI 컨테이너에서 관리하고 통일된 유연한 웹 어플리케이션을 구축할 수 있다.

4 Spring의 구성 모듈

스프링 프레임워크는 약 20개의 모듈로 구성되어있으며, 모듈들은 코어 컨테이너, 데이터 접근/통합 웹, AOP(관점지향 프로그래밍), Instrumentation, 테스트로 그룹을 나눌 수 있다.



4.1 코어 컨테이너

Spring 프레임워크의 근간이 되는 IoC(또는 DI) 기능을 지원하는 영역을 담당하고 있다. BeanFactory를 기반으로 Bean 클래스들을 제어할 수 있는 기능을 지원한다. BeanFactory을 사용하면 프로그래밍 적으로 싱글톤을 구현할 필요가 없고 실제 프로그램 로직에서 의존성에 대한 설정과 명세를 분리할 수 있다.

코어(Core), 빈즈(Beans), 컨텍스트(Context), 표현언어 (Expression Language) 모듈로 이루어졌다.

4.2 데이터 접근/통합 계층

JDBC, ORM, OXM, JMC, 트랜잭션 모듈로 이루어져 있다. JDBC모듈은 JDBC 추상화 계층을 제공한다. JDBC 코딩과 데이터베이스 벤더에 따라 다른 오류코드를 파싱할 필요가 없다.

ORM 모듈은 JPA, JDO , Hibernate, iBatis를 포함하는 인기 있는 객체-관계 매핑 API에 대한 통합계층을 제공한다. OXM 모듈은 JAXB, Castor, XMLBeans, JiBX, XStream에 대한 객체/XML매핑 구현을 지원하는 계층이다.

자바 메시징 서비스(JMS) 모듈은 메시지를 생산하고 소비하는 기능을 포함한다.

트랜잭션 모듈은 특별한 인터페이스와 모든 POJO (plain old Java objects)의 클래스에 대한 트랜잭션 관리를 지원한다. 트랜잭션 관리는 프로그래밍적으로 하거나 선언적으로 할 수 있다.

4.3 웹 계층

Spring 프레임워크에서 독립적으로 Web UI Layer에 Modd-View-Controller를 지원하기 위한 기능이다. 지금까지 Struts, Webwork가 담당했던 기능들을 Spring Web MVC를 이용하여 대체하는 것이 가능하다. 또한 Velocity, Excel, PDF와 같은 다양한 UI 기술들을 사용하기 위한 API를 제공하고 있다.

웹 계층은 웹, 웹-서블릿, 웹-스트러츠, 웹-포틀릿 모듈로 이루어졌다. 웹-서블릿 모듈은 웹 어플리케이션을 위한 스프링의 모델-뷰-컨트롤러 (MVC) 구현을 포함한다. 웹-스트러츠(Web-Struts) 모듈은 스프링 어플리케이션에서 전통적인 스트러츠 웹티어를 통합을 지원하는 클래스를 포함한다. 이 지원은 스프링 3.0에서는 폐기되었다. 웹-포틀릿(Web-Portlet) 모듈은 포틀릿 환경에서 사용되는 MVC 구현과 웹-서블릿 모듈 기능의 미러 기능을 제공한다.

4.4 AOP 계층

Spring 프레임워크에 Aspect Oriented Programming을 지원하는 기능이다. 이 기능은 AOP Alliance 기반 하에서 개발되었다.

Chapter 2. MyBatis 소개 및 설치

1 MyBatis란?

MyBatis는 관계형 데이터베이스의 레코드와 자바 도메인 객체 사이의 매핑을 자동화 해주는 프레임워크로 데이터베이스의 SQL 문을 거의 그대로 사용할 수 있게 단순화 해주는 영속성 프레임워크이다. 따라서 활용하기 어려운 집계나 조인도 데이터베이스의 SQL 작성을 통해 간단하게 구현할 수 있다.

MyBatis 프레임워크는 iBatis 프레임워크를 동일한 개발자들이 재설계하여 새롭게 만든 것이다. 내부적으로는 iBatis라는 패키지 이름이 계속 사용되고 있으며 버전도 이어 받고 있다.

iBatis는 JDK 1.4이상에서 사용 가능하며 MyBatis는 JDK 1.5 이상을 요구한다. MyBatis 3.2 이상 버전은 JDK 1.6 이상을 요구하고 있다.

4.5 iBatis로부터의 변경점

4.5.1 Package 내부 구조 변경

이름은 변경되었지만 내부적으로는 iBatis와 같은 구조를 사용한다

- iBatis : com.ibatis.*
- MyBatis : org.apache.ibatis.*

4.5.2 SqlMap.xml 내부 구조의 변경

iBatis의 parameterMap이 Deprecated되었으며 기존 부분을 parameterType에 정의하게 되었다

```
ex)
<insert id="insertData" parameterType="DataVo">
  insert into test (user_no, user_name, data, reg_date)
  values ( #{ userNo }, #{ username }, #{ data }, #{ regDate } )
</insert>
```

4.5.3 용어의 변경

- SqlMapConfig -> Configuration
- sqlMap -> mapper

4.5.4 네임스페이스 방식의 변경

sqlMap 별로 약식 명칭을 사용할 수 없게 되었으며 경로를 모두 명시해 주어야 한다. 또한 namespace가 필수 항목이 되었다

- iBatis : <sqlMap namespace="Data">
- MyBatis : <mapper namespace="com.sample.mapper.DataMapper">

5 MyBatis 설치

MyBatis를 사용하기 위해 홈페이지에서 최신버전을 다운 받는다.
현재 최신 버전은 3.4.5 버전이다

http://www.mybatis.org/mybatis-3/ko/

사이트를 접속해 시작하기 탭에서 설치 > mybatis-x.x.x.jar링크를 클릭하고 하단의 Downloads 항목의 mybatis-3.4.5.zip을 클릭해 다운 받을 수 있다

다운받은 압축 파일을 해제하면 mybatis-3.4.5.jar 파일을 확인할 수 있다. .jar 파일을 WEB-INF/lib 폴더에 넣어 사용한다. Oracle JDBC를 사용한다면 ojdbcX.jar파일 또한 WEB-INF/lib 폴더에 넣어서 사용한다.

1-3. Spring 프레임워크 설치

1 Spring 설치

Spring 관련 라이브러리

http://repo.spring.io/release

위 사이트로 접속해 org - springframework - spring 경로에서 spring-framework-5.0.2.RELEASE-dist.zip 를 다운 받아 압축 해제한다

1.1 Spring 라이브러리의 폴더 구성

- libs - 스프링 프레임워크의 각 모듈별 jar파일과 소스 jar파일을 포함
- docs - API문서 및 레퍼런스 문서 : 도움말
- schema - 각 모듈별 설정 관련 XML 스키마

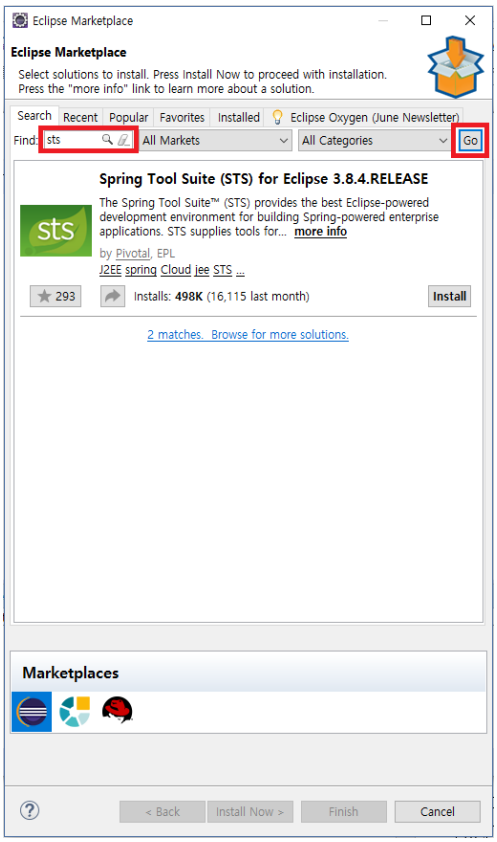
1.1.1 Spring의 모듈 구성

모듈명 (jar파일)	설명
core	DI 기능을 비롯한 프레임워크의 기반을 제공
beans	BeanFactory 인터페이스를 통해 구현
expression	객체에 접근하고 객체를 조작하기 위한 표현 언어를 제공 (JSP 2.1 규약에 명시된 통합 EL을 확장)
context	spring-core와 spring-beans 모듈을 확장해서 국제화, 이벤트 처리, 리소스 로딩, 서블릿 컨테이너를 위한 컨텍스트 생성 등의 기능을 주가로 제공 (ApplicationContext 인터페이스를 통해 구현)
context.support	cache, 메일 , 스케줄링 , UI의 VeloCity 지원기능을 제공
aop	AOP Alliance에 호환되는 AOP 구현을 제공
aspects	AspectJ와의 통합을 지원
web	파일업로드, loCale 처리 등 웹을 위한 통합 기능을 제공 원격 지원 기능 중 웹 관련 기능을 제공
webmvc	스프링 MVC를 제공한다. JSP, VeloCity에 대한 뷰 연동을 지원
web.struts	스프링과 스트럿츠 연동 기능을 제공
web.portlet	포틀릿 환경에서 사용되는 MVC를 구현
transaction : tx	AOP를 이용한 선언적 트랜잭션 관리 및 코드를 이용한 트랜잭션관리 기능을 제공
jdbc	JDBC프로그래밍을 위한 추상 레이어를 제공 간결한 코드로 JDBC프로그래밍을 할 수 있도록 지원
orm	하이버네이트, JPA, iBatis, JDO등 ORM API를 위한 통합 레이어를 제공 스프링이 제공하는 트랜잭션 관리와의 연동을 지원
oxm	객체와 XML 사이의 매핑을 처리하기 위한 추상 레이어를 제공 JAXB, Castor, XMLBeans, JiBX, XStream과의 연동을 지원
jms	JMS의 메시지를 생성하고 수신하는 기능을 제공
test	JUnit이나 TestNG를 이용한 스프링 컴포넌트의 테스트를 지원
instrument	instrumentation 지원 클래스를 제공
intrument.tomcat	톰캣 서버를 위한 instrumentation 지원 클래스를 제공

이클립스에 STS(Spring Tool Suite) 플러그인 설치하기

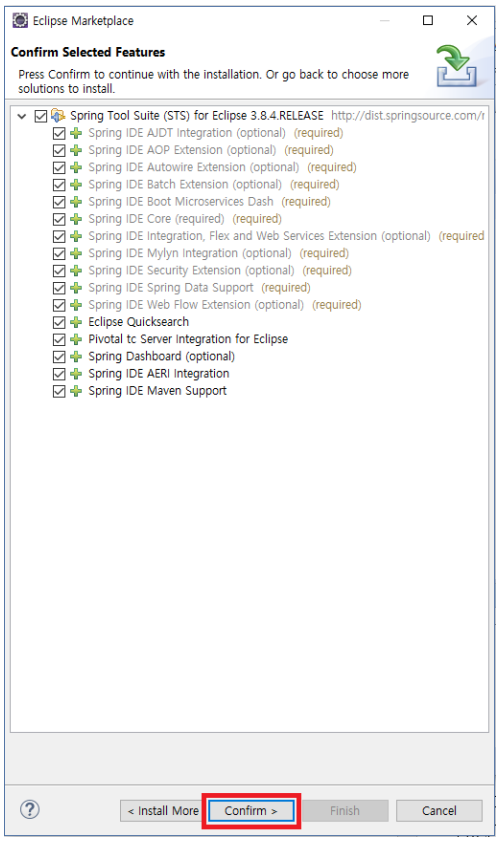
① 이클립스를 실행하고 메뉴에서 [Help] - [Eclipse Marketplace]를 클릭

② Find : sts 입력하고 [Go] 클릭

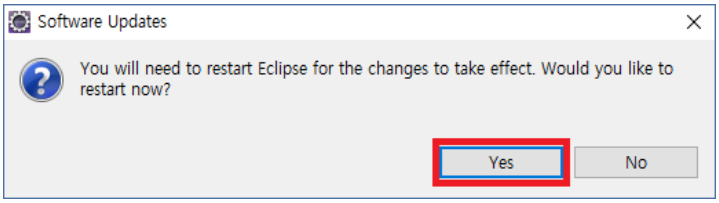


③ STS(Spring Tool Suite) 이 검색되면 [Install] 클릭

④ Features가 모두 선택된 상태에서 [Confirm]



⑤ 설치가 완료되면 Eclipse 재시작



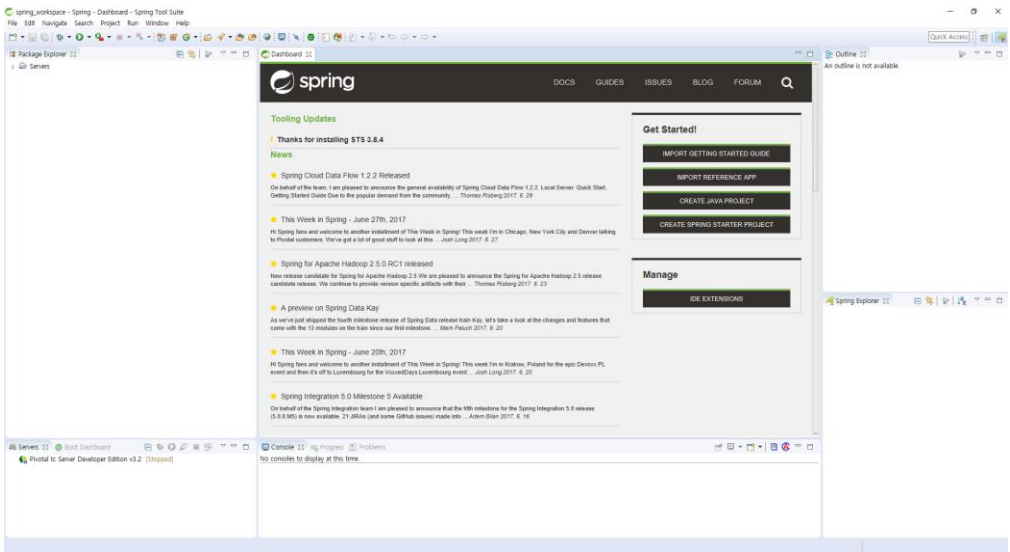
STS(Spring Tool Suite) 툴 설치

① STS 다운로드 및 설치

https://spring.io/tools/sts/all 사이트 접속 후, 원하는 버전 다운

② 다운받은 압축파일을 해제하면 설치 완료

③ sts-3.9.1.RELEASE 폴더의 STS.exe를 실행하면 된다



1-4. 라이브러리 자동관리도구 maven 설치

1 Maven 설치

스프링 프로젝트를 사용하기 위해서는 다양한 라이브러리 파일들을 설치하고 관리해야 하는데, Maven은 이러한 라이브러리들을 통합해서 관리해 주는 플러그인 실행 프레임워크이다.

즉, 기존에 lib 폴더 내에 사용하고자 하는 라이브러리를 등록해서 사용했다면, Maven은 이를 자동으로 관리해 주기 때문에 라이브러리를 이용하는데 더욱 편리함을 느낄 수 있다.

Maven의 5가지 특징	
Plugin	Maven에서 제공하는 모든 기능은 Plugin기반으로 동작한다
LifeCycle	Maven은 일련의 단계를 통한 빌드 순서를 가진다
Dependency	<dependencies>/<dependency> 엘리먼트를 사용하여 의존성을 관리
Profile	서로 다른 환경에 따라 달라지는 설정을 관리하는 기능
POM (Project Object Model)	하나의 프로젝트당 pom.xml을 하나씩 가짐으로써 프로젝트 자체와 의존성에 대한 설정을 포함한다

A. Maven 다운로드

https://maven.apache.org/download.cgi

다음 사이트에 접속해 Files 항목 하단의 Binary zip archive에서 Maven 최신 파일을 다운받아 압축을 해제한다.

B. Maven 라이브러리 폴더 구성

- bin - Maven 실행파일과 Debug 파일 위치
- boot - Maven 실행과 관련된 class 들이 정의되어 있음
- conf - Maven 환경 설정과 관련된 파일 위치
- lib - Maven의 각 모듈별 jar파일과 소스 jar파일을 포함

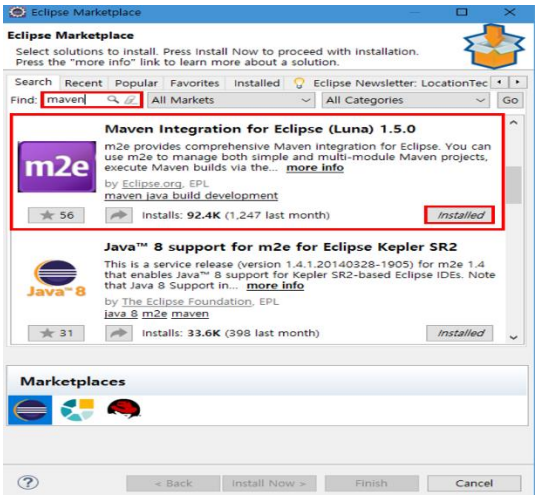
C. 사용자 Repository 설정

압축 해제한 폴더에서 conf/settings.xml 파일을 연 뒤, localRepository 가 주석 처리 되어있는 항목의 <localRepository> 엘리먼트를 주석 밖으로 꺼내어 Repository 위치를 원하는 경로로 설정한다.

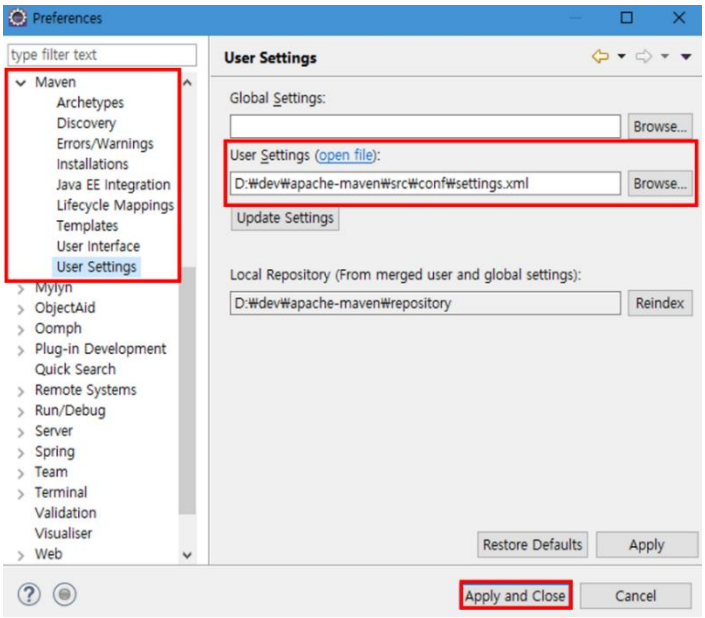
Ex) <localRepository>D:₩dev₩local_repository</localRepository>

D. 이클립스 IDE 연동

이클립스에서 [Help] - [Eclipse Marketplace] 에 접속하여, maven으로 검색, Maven Plugin이 설치 되어 있는지 확인하고, 설치가 되어 있지 않다면 설치한다.



설치가 완료되었다면 이클립스를 한 번 재시작 한 뒤, [Windows] - [Preferences] 에서 Maven - User Settings 항목을 클릭하여 User Settings 의 경로를 이전에 설정한 conf/settings.xml 경로로 설정하고 Apply and Close를 클릭한다.



마지막으로 이클립스에서 사용하는 JDK와 Maven에서 필요로 하는 JDK가 매칭 시켜야 하기 때문에, 이클립스 설치 폴더 내의 이클립스 환경 설정 파일인 eclipse.ini 를 열고 하기 코드를 vmargs 옵션 위쪽에 추가해야 한다. Ex) --vm C:₩dev₩jdk1.7.0_60₩bin₩javaw.exe

--vm JDK 안의 javaw.exe 위치 경로

그렇지 않으면 JDK가 일치하지 않아 하기의 에러를 발생 시킨다.

Maven Integration for Eclipse JDK Warning The Maven Integration requires that Eclipse be running in a JDK, because a number of Maven core plugins are using jars from the JDK. Please make sure the -vm option in eclipse.ini is pointing to a JDK and verify that Installed JREs are also using JDK installs.

Chapter 3. MyBatis 설치 및 환경설정

1 MyBatis 환경설정

1.1 MyBatis 설정 파일 (mybatis-config.xml) 생성하기

MyBatis의 설정파일은 getConnection( ) 메소드의 데이터베이스 연결정보를 대체하는 기능을 제공한다. 설정 파일에 대한 양식은 Mybatis 폴더에 동봉된 mybatis-3.4.5.pdf, 3page를 참조한다.

- Oracle Express Edition에서 SCOTT/TIGER 계정으로 연결을 설정한다 가정하자.

src/mybatis/mybatis-config.xml

```
01 <?xml version="1.0" encoding="UTF-8" ?>
02 <!DOCTYPE configuration PUBLIC "-//mybatis.org//DTD Config 3.0//EN"
   "http://mybatis.org/dtd/mybatis-3-config.dtd">
03 <configuration>
04   <environments default="development">
05     <environment id="development">
06 <!-- 트랜잭션 관리자를 설정, JDBC를 대체하는 의미에서 type을 JDBC로 지정한
07 다. -->
08       <transactionManager type="JDBC" />
09
10       <!-- 데이터베이스 세부 설정 -->
11       <dataSource type="POOLED">
12         <property name="driver" value="oracle.jdbc.driver.OracleDriver" />
13         <property name="url" value="jdbc:oracle:thin:@localhost:1521:xe" />
14         <property name="username" value="scott" />
15         <property name="password" value="tiger" />
16       </dataSource>
17     </environment>
18   </environments>
19   <mappers>
```

```
20    <!-- SQL을 선언해 둔 XML이나 인터페이스 형태의 Mapper 위치를 지정한다 --
21  >
22      <mapper resource="member/model/mapper/member-mapper.xml" />
    </mappers>
  </configuration>
```

A. MyBatis 객체 생성하기 - SqlSessionFactory

만들어진 설정파일(mybatis-config.xml)을 로드해서 MyBatis 객체를 생성한다

객체 생성 이후 MyBatis를 이용하여 데이터베이스 처리를 하려면 getSqlSessionFactory( ) 메소드를 이용하여 객체를 생성하고 API를 호출하여 사용하면 된다.

```
01  private SqlSessionFactory getSqlSessionFactory() {
02      String resource = "mybatis/mybatis-config.xml";
03      SqlSessionFactory factory = null;
04      try {
05          InputStream inputStream = Resources.getResourceAsStream(resource);
06          SqlSessionFactoryBuilder builder = new SqlSessionFactoryBuilder();
07          factory = builder.build(inputStream);
08      } catch (IOException e) {
09      }
10      return factory;
11  }
```

B. SqlSession 객체 생성하기

SqlSessionFactory를 통해 SqlSession 객체를 생성할 때 여러 메소드가 오버로드 되어 있으나 기본적으로 2개의 메소드를 사용한다.

```
SqlSession openSession()
SqlSession openSession(boolean autoCommit)
```

Default 속성인 openSession()을 하게 되면, 자동 커밋 설정을 true로 적용하기 때문에, false 속성을 주어 직접 관리하는 것을 권장한다.

Part 2. 인터페이스 기능 구현하기

Chapter 4. MyBatis Mapper 와 element

1 MyBatis 요소

SqlSession 객체로 CRUD 구문을 실행하기 위한 메소드는 다음과 같다.

```
<T> T selectOne(String statement, Object parameter)
<E> List<E> selectList(String statement, Object parameter)
<K,V> Map<K,V> selectMap(String statement, Object parameter, String mapKey)
int insert(String statement, Object parameter)
int update(String statement, Object parameter)
int delete(String statement, Object parameter)
```

이 메소드들은 SQL 매핑 XML 파일에 정의된 SELECT, INSERT, UPDATE 그리고 DELETE 구문을 실행하기 위해 사용된다. 메소드 이름 자체가 그 역할을 설명하도록 명명되었으며, Select 구문의 경우 객체나 객체 리스트를, insert / update / delete의 경우 수행한 적용한 행의 개수를 리턴한다.

selectOne과 selectList의 차이점은 selectOne메소드는 오직 하나의 객체만을 리턴해야 한다는 것이다. 한개 이상을 리턴하거나 null 이 리턴된다면 예외가 발생할 것이다. 얼마나 많은 객체가 리턴될지 모른다면 selectList를 사용해야 하며, 객체의 존재여부를 체크하고 싶다면 개수를 리턴하는 방법을 사용하는 것이 효율적이다.

1.1 Mapper XML 파일 작성 – Select –

select구문은 마이바티스에서 가장 흔히 사용할 엘리먼트이다. 데이터베이스에서 데이터를 가져온다. 조회는 다음 예제처럼 단순한 경우에는 단순하게 설정된다.

```
<select id="selectMember" parameterType="string" resultType="hashmap">
  SELECT * FROM MEMBER WHERE ID = #{id}
</select>
```

속성	설명
resultType	반환되는 타입의 전체 클래스명이나 별명
resultMap	외부 resultMap 참조이름, 마이바티스의 가장 강력한 기능이다
useCache	구문의 결과에 캐시 사용 여부 지정, 디폴트는 true
fetchSize	지정된 결과의 수만큼 결과를 반환하는 값
resultSetType	FORWARD_ONLY, SCROLL_SENSITIVE, SCROLL_INSENSITIVE 중 선택

1.2 Mapper XML 파일 작성 – Insert, Update, Delete –

Insert, update, delete의 경우 사용 문법은 다음과 같다.
각각에 대한 파라미터로 원하는 객체를 지정하여 설정할 수도 있다.

```
<insert id="insertMember" parameterType="member.model.vo.Member">
  insert into MEMBER (id,username,password,email)
  values (#{id},#{username},#{password},#{email})
</insert>

<update id="updateMember" parameterType="member.model.vo.Member">
  update MEMBER set username = #{username},
      password = #{password},
      email = #{email}
  where id = #{id}
</update>

<delete id="deleteMember" parameterType="string">
  delete from MEMBER where id = #{id}
</delete>
```

속성	설명
parameterType	구문에 전달될 파라미터의 패키지 경로를 포함한 전체 클래스명이나 별칭
useGeneratedKeys	(insert, update) 데이터베이스에서 내부적으로 생성한 키 (MySQL, SQL Server과같이 자동 증가 필드가 있는 경우만 해당), 디폴트는 false
keyProperty	getGeneratedKeys 메서드나 insert 문의 selectKey 하위 요소에 의해 반환된 키를 지정할 프로퍼티 지정
keyColumn	(insert, update) 생성키를 가진 테이블의 칼럼명을 셋팅. 키 칼럼이 테이블의 첫번째 칼럼이 아닌 데이터베이스(PostgreSQL 처럼)에서만 필요

2 동적 SQL 작성하기

마이바티스의 가장 강력한 기능 중 하나는 동적 SQL을 처리하는 방법이다. 마이바티스에서는 if, choose (when, otherwise), trim (where, set), foreach를 동적 SQL 엘리먼트로 제공하는데, JSTL 과 사용 문법이 유사하다.

2.1 if

동적 SQL 에서 가장 공통적으로 사용되는 것으로 where의 일부로 포함될 수 있다.

```
<select id="findActiveBlogWithTitleLike" resultType="Blog">
  SELECT * FROM BLOG WHERE state = 'ACTIVE'
  <if test="title != null">
    AND title like #{title}
  </if>
</select>
```

2.2 choose, when, otherwise

자바의 switch, ORACLE의 CASE구문과 유사하며, 원하는 조건에 해당하는 경우를 조회할 수 있다.

```
<select id="findActiveBlogLike"
     resultType="Blog">
  SELECT * FROM BLOG WHERE state = 'ACTIVE'
  <choose>
    <when test="title != null">
      AND title like #{title}
    </when>
    <when test="author != null and author.name != null">
      AND author_name like #{author.name}
    </when>
    <otherwise>
      AND featured = 1
    </otherwise>
  </choose>
</select>
```

2.3 trim, where

본 예제를 통해 원하는 검색 결과를 얻는 상황을 생각해보자.

```
<select id="findActiveBlogLike"
     resultType="Blog">
  SELECT * FROM BLOG
  WHERE
  <if test="state != null">
    state = #{state}
  </if>
  <if test="title != null">
    AND title like #{title}
  </if>
  <if test="author != null and author.name != null">
    AND author_name like #{author.name}
  </if>
</select>
```

만약 받은 파라미터가 어떤 조건에도 해당하지 않는다면, 다음과 같은 SQL이 생성될 것이다.

```
SELECT * FROM BLOG
WHERE
```

그럼 해당 SQL은 미완성되어 호출 시 에러가 발생 할 것이다.

만약 두번째 조건만 해당하는 경우에도 하기와 같은 SQL로 생성되어 실패할 것이다.

```
SELECT * FROM BLOG
WHERE
AND title like 'TEST'
```

이 같은 상황을 해결하기 위해 마이바티스에서는 다음과 같이 <where> 엘리먼트를 제공한다.

```
<select id="findActiveBlogLike"
     resultType="Blog">
  SELECT * FROM BLOG
  <where>
```

```
        <if test="state != null">
            state = #{state}
        </if>
        <if test="title != null">
            AND title like #{title}
        </if>
        <if test="author != null and author.name != null">
            AND author_name like #{author.name}
        </if>
    </where>
</select>
```

<where> 엘리먼트는 해당 SQL을 호출할 경우 WHERE 조건을 추가하며, 만약 AND 나 OR로 시작하는 조건으로 시작한다면 해당하는 접두사를 지워버린다. 만약 <where> 엘리먼트로 해결되지 않는다면, trim을 사용하여 구현할 수도 있다.

```
<trim prefix="WHERE" prefixOverrides="AND |OR ">
    ...
</trim>
```

2.4 foreach

동적 SQL 에서 반드시 필요한 것은 collection에 대한 반복처리일 것이다. 이러한 처리에 대해서 마이바티스는 Foreach를 통해 List나 Set, Map과 같은 Collection 파라미터를 받아 처리할 수 있다.

```
<select id="selectPostIn" resultType="domain.blog.Post">
    SELECT *
    FROM POST P
    WHERE ID IN
   <foreach item="item" index="index" collection="list" open="(" separator="," close=")">
            #{item}
   </foreach>
</select>
```

엘리먼트 내부에서 각각 item, index 변수를 통해 단일 객체 접근이 가능하며, Map을 사용할
경우, index는 key객체가 되고 항목은 value객체를 의미하게 된다. 또한 열고 닫는 문자열로 명시할 수 있고 반복간에 둘 수 있는 구분자도 추가할 수 있다.

2.5 Mapper XML 사용 예제

실제 이를 사용하는 문법을 간략화 하여 설명해보면, 먼저 Member라는 사용자 정보를 조회할수 있는 'memberInfo' 쿼리가 작성된 mapper.xml 라는 파일이 있다고 하자. 그럼 이전에 선언한 getSqlSessionFactory()를 통해 SqlSession을 생성하여 다음과 같이 구문을 작성하여 사용한다.

src/member/model/mapper/member-mapper.xml

```
01 <?xml version="1.0" encoding="utf-8"?>
02 <!DOCTYPE mapper PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
03 "http://mybatis.org/dtd/mybatis-3-mapper.dtd">
04 <mapper namespace="Member"> <!—namespace를 통해 내부 쿼리에 접근이 가
05 능 -->
06
07  <!--  정의한 ID로 해당 쿼리의 정보를 가져온다. -->
08   <select                    id="memberInfo"                    parameterType="string"
09 resultType="member.model.vo.Member">
10      SELECT * FROM MEMBER WHERE ID = #{userid}
    </select>
  </mapper>
```

member.model.dao.MemberDao.java

```
01 public Member selectMember(String userid) {
02      Member member = null;
03      SqlSession session = null;
04      try {
05         session = getSqlSessionFactory().openSession(false);
06
07         // 매퍼 파일 안에 작성된 쿼리문 실행시키고 결과 받는다.
08          member = session.selectOne("Member.memberInfo", userid);
09
10      } catch (Exception e) {
11         e.printStackTrace();
12      }finally{
13         session.close();
14      }
15      return member;
16   }
```

3 MyBatis 3.x를 이용한 CRUD 예제

mybatis-3.4.5.jar를 WEB-INF/lib 폴더에 넣는다
ojdbc6.jar 파일을 WEB-INF/lib 폴더에 넣는다

3.1 테스트를 위한 테이블 스키마

```
DROP TABLE MEMBER CASCADE CONSTRAINTS;

CREATE TABLE MEMBER(
    ID    VARCHAR2(15 BYTE),
    PASSWD    VARCHAR2(15 BYTE) NOT NULL,
    NAME    VARCHAR2(20 BYTE) NOT NULL,
    EMAIL    VARCHAR2(30 BYTE),
    GENDER    CHAR(1 BYTE) CHECK (GENDER IN ('M', 'F')),
    AGE    NUMBER,
    PHONE    CHAR(13 BYTE),
    ADDRESS    VARCHAR2(50 BYTE),
    ENROLL_DATE        DATE DEFAULT SYSDATE,
    CONSTRAINT PK_MEMBER PRIMARY KEY (ID)
);
```

3.2 MyBatis 설정파일 작성

src/mybatis/mybatis-config.xml

```
01  <?xml version="1.0" encoding="utf-8"?>
02  <!DOCTYPE configuration PUBLIC "-//mybatis.org//DTD Config 3.0//EN"
      "http://mybatis.org/dtd/mybatis-3-config.dtd">
03  <configuration>
04    <settings>
05      <setting name="jdbcTypeForNull" value="NULL"/> <!-- Null 값 처리 -->
06    </settings>
07
08    <typeAliases> <!-- Class 별칭을 선언하여 모든 경로를 적어주지 않아도 된다. --
09  >
```

```
10      <typeAlias type="member.model.vo.Member" alias="Member" />
11    </typeAliases>
12    <environments default="development">
13      <!-- DB 연동 설정 부, 'environment id'로 구분한다. -->
14      <environment id="development">
15        <transactionManager type="JDBC" />
16        <!—Mybatis는 transactionManager로 트랜잭션을 관리하므로
17              Autocommit 에 대한 설정을 할 수 있다. -->
18        <dataSource type="POOLED">
20          <property name="driver" value="oracle.jdbc.driver.OracleDriver" />
21          <property name="url" value="jdbc:oracle:thin:@127.0.0.1:1521:xe" />
22          <property name="username" value="student" />
23          <property name="password" value="student" />
24        </dataSource>
25      </environment>
26
27    <mappers>   <!—사용하고자 하는 mapper를 등록한다. -->
28      <mapper resource="member/model/mapper/member-mapper.xml" />
29    </mappers>
    </configuration>
```

3.3 매퍼 파일 작성

src/member/mapper/member-mapper.xml

```
01  <?xml version="1.0" encoding="utf-8"?>
02  <!DOCTYPE mapper PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
    "http://mybatis.org/dtd/mybatis-3-mapper.dtd">
03  <mapper namespace="Member">
04  <!-- Member클래스를 Alias로 지정하였다면, namespace에서 별칭으로 접근이 가
05  능하다-->
06
07    <resultMap id="resultMember" type="Member">
08    <!-- property 는 필드명, column 은 DB 컬럼 명을 기술한다 -->
09      <id property="id" column="ID" />
10      <result property="passwd" column="PASSWD" />
11      <result property="name" column="NAME" />
12      <result property="email" column="EMAIL" />
13      <result property="gender" column="GENDER" />
14      <result property="age" column="AGE" />
15      <result property="phone" column="PHONE" />
16      <result property="address" column="ADDRESS" />
17      <result property="enrollDate" column="ENROLL_DATE" />
18    </resultMap>
19
20    <select id="memberInfo" parameterType="string" resultType="_int"
21  resultMap="resultMember">
22      SELECT * FROM MEMBER WHERE ID = #{userid}
23    <!-- #{field}는 java.sql.PreparedStatement의 ?의 역할이다 -->
24    </select>
25
26    <select id="loginCheck" parameterType="Member"
27      resultType="Member" resultMap="resultMember">
28      SELECT * FROM MEMBER WHERE ID = #{id} AND PASSWD = #{passwd}
29    </select>
30
```

```
31  <insert id="insertMember" parameterType="Member" statementType="PREPARED">
32     INSERT INTO MEMBER VALUES(#{id}, #{passwd}, #{name}, #{email}, #{gender},
33  #{age}, #{phone}, #{address}, DEFAULT)
34    </insert>
35
36  <update id="updateMember" parameterType="Member"
37  statementType="PREPARED">
38    UPDATE MEMBER SET PASSWD = #{passwd}, EMAIL = #{email}, AGE = #{age},
39       PHONE = #{phone}, ADDRESS = #{address} WHERE ID = #{id}
40   </update>
41
42  <delete id="deleteMember" parameterType="string" statementType="PREPARED">
      DELETE FROM MEMBER WHERE ID = #{userid}
     </delete>
    </mapper>
```

3.4 Member VO 작성

member.model.vo.Member.java

```
01  package member.model.vo;
02
03  import java.sql.Date;
04
05  public class Member implements java.io.Serializable{
06          private static final long serialVersionUID = 1111L;
07          private String id;
08          private String passwd;
09          private String name;
10          private String email;
11          private String gender;
12          private int age;
13          private String phone;
14          private String address;
15          private Date enrollDate;
16
17          public Member(){}
18
19          public Member(String id, String passwd, String name, String email,
20                  String gender, int age, String phone, String address) {
21              super();
22              this.id = id;
23              this.passwd = passwd;
24              this.name = name;
25              this.email = email;
26              this.gender = gender;
27              this.age = age;
28              this.phone = phone;
29              this.address = address;
30          }
31
```

```
32          public Member(String id, String passwd, String name, String email,
33                  String gender, int age, String phone, String address, Date enroll_date) {
34              super();
35              this.id = id;
36              this.passwd = passwd;
37              this.name = name;
38              this.email = email;
39              this.gender = gender;
40              this.age = age;
41              this.phone = phone;
42              this.address = address;
43          }
44  /* ... Field 에 대한 Getter와 Setter 선언 ... */
45
46          @Override
47          public String toString(){
48              return this.id + ", " + this.passwd + ", " +
49              this.name + ", " + this.gender + ", "
50              + this.age + ", " + this.email + ", "
51              + this.phone + ", " + this.address
52              + ", " + this.enrollDate;
53          }
54  }
```

3.5 MemberDao 작성

member.model.dao.MemberDao.java

```
01  package member.model.dao;
02
03  import java.io.IOException;
04  import java.io.InputStream;
05
06  import org.apache.ibatis.io.Resources;
07  import org.apache.ibatis.session.SqlSession;
08  import org.apache.ibatis.session.SqlSessionFactory;
09  import org.apache.ibatis.session.SqlSessionFactoryBuilder;
10
11  import member.model.vo.Member;
12
13  public class MemberDao {
14      public MemberDao(){}
15        // Session 객체를 얻어 올 getSqlSessionFactory 메소드 구현
16      private SqlSessionFactory getSqlSessionFactory() {
17          String resource = "mybatis/mybatis-config.xml";
18          SqlSessionFactory factory = null;
19          try {
20              // InputStream으로 Mybatis의 설정 정보를 읽어 온다
21              InputStream inputStream = Resources.getResourceAsStream(resource);
22              // 읽어 온 Mybatis의 설정 정보를 바탕으로 SqlSessionFactoryBuilder를
23  생성한다
24              SqlSessionFactoryBuilder builder = new SqlSessionFactoryBuilder();
25              factory = builder.build(inputStream);
26          } catch (IOException e) {
27              e.printStackTrace();
28          }
29          return factory;
30      }
31
```

```
32      public Member checkIdDup(String userid) {
33          Member member = null;
34          SqlSession session = null;
35          try {
36              // JDBC의 getSession()을 통한 Connection 객체 생성 처리와
37              // 동일한 의미를 가진 코드이다
38              session = getSqlSessionFactory().openSession(false);
39
40              // mapper 파일 안에 작성된 memberInfo 쿼리문을 실행하고 결과를 받
41  는다
42              member = session.selectOne("Member.memberInfo", userid);
43
44          } catch (Exception e) {
45              e.printStackTrace();
46          }finally{
47              session.close();
48          }
49          return member;
50      }
51
52      public Member selectMember(Member m) {
53          Member member = null;
54          SqlSession session = null;
55          try {
56              session = getSqlSessionFactory().openSession(false);
57
58              member = session.selectOne("Member.loginCheck", m);
59              System.out.println(session);
60          } catch (Exception e) {
61              e.printStackTrace();
62          } finally{
63              session.close();
64          }
```

```
65          return member;
66      }
67
68      public int insertMember(Member m) {
69          int result = 0;
70          SqlSession session = null;
71          try {
72              session = getSqlSessionFactory().openSession(false);
73              result = session.insert("Member.insertMember", m);
74             // 세션 생성 시 AutoCommit을 false로 설정하였으므로
75             // commit / rollback을 직접 관리한다.
76              if(result > 0){
77                  session.commit();
78              } else {
79                  session.rollback();
80              }
81          } catch (Exception e) {
82              e.printStackTrace();
83          }finally{
84              session.close();
85          }
86          return result;
87      }
88
89      public int updateMember(Member m) {
90          int result = 0;
91          SqlSession session = null;
92          try {
93              session = getSqlSessionFactory().openSession(false);
94
95              result = session.update("Member.updateMember", m);
96              if(result > 0){
97                  session.commit();
```

```
98              } else {
99                  session.rollback();
100             }
101         } catch (Exception e) {
102             e.printStackTrace();
103         }finally{
104             session.close();
105         }
106         return result;
107     }
108
109     public int deleteMember(String userid) {
110         int result = 0;
111         SqlSession session = null;
112         try {
113             session = getSqlSessionFactory().openSession(false);
114
115             result = session.delete("Member.deleteMember", userid);
116             if(result > 0){
117                 session.commit();
118             } else {
119                 session.rollback();
120             }
121         } catch (Exception e) {
123             e.printStackTrace();
124         }finally{
125             session.close();
126         }
127         return result;
    }
}
```

3.6 MemberService 작성

member.model.service.MemberService.java

```
01  package member.model.service;
02
03  import member.model.dao.MemberDao;
04  import member.model.vo.Member;
05
06  public class MemberService {
07      public MemberService(){}
08      // 기존 JDBC에서 쓰이던 Connection 연결을 Dao에서 관리하므로,
09      // 별도 Connection 정보를 담당하지 않는다.
10      public Member selectMember(Member m){
11          return new MemberDao().selectMember(m);
12      }
13
14      public int insertMember(Member m) {
15          return new MemberDao().insertMember(m);
16      }
17
18      public int updateMember(Member m) {
19          return new MemberDao().updateMember(m);
20      }
21
22      public int deleteMember(String userid) {
23          return new MemberDao().deleteMember(userid);
24      }
25
26      public Member checkIdDup(String userid){
27          return new MemberDao().checkIdDup(userid);
28      }
29  }
```

Chapter 5. MyBatis 페이징 처리

1 List 페이징 처리

이전에 게시글에 대한 조회 구문을 ROWNUM을 사용하여 구현 하였다면, 마이바티스에서는 Rowbounds 클래스를 통해 페이징 처리 구현에 대한 편리성을 제공한다.

Rowbounds 클래스는 특정 개수의 레코드를 건너뛰게 한다.

```
int offset = 100;    // 뛰어 넘어야 할 최소값
int limit = 25;      // 읽는 레코드의 개수
RowBounds rowBounds = new RowBounds(offset, limit);
```

그럼, ROWNUM을 사용한 경우와 Rowbounds 를 사용한 경우 코드가 어떻게 달라지는지 확인해보자.

- 테스트를 위한 테이블 스키마

```
DROP TABLE BOARD CASCADE CONSTRAINTS;

CREATE TABLE BOARD (
    BOARD_NUM NUMBER,              -- 게시글 번호
    BOARD_TITLE VARCHAR2(50),      -- 게시글 제목
    BOARD_WRITER VARCHAR2(15),     -- 게시글 작성자
    BOARD_CONTENT VARCHAR2(2000), -- 게시글 내용
    BOARD_ORIGINAL_FILENAME VARCHAR2(100),  -- 업로드한 원래 파일명
    BOARD_RENAME_FILENAME VARCHAR2(100),  -- 바뀐 파일명
    BOARD_DATE  DATE  DEFAULT SYSDATE,-- 게시글 올린 날짜
    BOARD_LEVEL  NUMBER  DEFAULT 0,-- 글레벨 : 원글 0, 원글의 답글 1, 답글의 답글 2
    BOARD_REF  NUMBER,   -- 원글일때는 자기번호, 답글일 때 참조하는 원글 번호
    BOARD_REPLY_REF NUMBER,
        -- 원글일때는 0, 레벨이 1이면 자기번호, 레벨이 2이면 참조하는 답글번호
    BOARD_REPLY_SEQ  NUMBER  DEFAULT 0, -- 답글의 순번
    BOARD_READCOUNT  NUMBER  DEFAULT 0, -- 조회수
    CONSTRAINT PK_BOARD PRIMARY KEY (BOARD_NUM),
    CONSTRAINT FK_BOARD_WRITER FOREIGN KEY (BOARD_WRITER)
        REFERENCES MEMBER (ID) ON DELETE SET NULL,
    CONSTRAINT FK_BOARD_REF FOREIGN KEY (BOARD_REF)
        REFERENCES BOARD (BOARD_NUM) ON DELETE CASCADE,
    CONSTRAINT FK_BOARD_REPLY_REF FOREIGN KEY (BOARD_REPLY_REF)
        REFERENCES BOARD (BOARD_NUM) ON DELETE CASCADE
  );
```

- 테스트를 위한 mybatis-config.xml

src/mybatis/mybatis-config.xml

```
01 <?xml version="1.0" encoding="utf-8"?>
02 <!DOCTYPE configuration PUBLIC
03 "-//mybatis.org//DTD Config 3.0//EN"
04 "http://mybatis.org/dtd/mybatis-3-config.dtd">
05 <configuration>
06   <settings>
07     <setting name="jdbcTypeForNull" value="NULL"/>
08   </settings>
09
10   <typeAliases> <!-- 자료형(Class) 별칭 선언 -->
11     <typeAlias type="board.model.vo.Board" alias="Board" />
12   </typeAliases>
13   <environments default="development">
14     <environment id="development">
15       <transactionManager type="JDBC" />
16       <dataSource type="POOLED">
17         <property name="driver" value="oracle.jdbc.driver.OracleDriver" />
18         <property name="url" value="jdbc:oracle:thin:@127.0.0.1:1521:xe" />
19         <property name="username" value="student" />
20         <property name="password" value="student" />
21       </dataSource>
22     </environment>
23   </environments>
24
25   <mappers>
26     <mapper resource="board/model/mapper/board-mapper.xml" />
27   </mappers>
28 </configuration>
```

- BoardListServlet 구현

board.controller.BoardListServlet.java

```
01  package board.controller;
02
03  import java.io.IOException;
04  import java.util.ArrayList;
05
06  import javax.servlet.RequestDispatcher;
07  import javax.servlet.ServletException;
08  import javax.servlet.annotation.WebServlet;
09  import javax.servlet.http.HttpServlet;
10  import javax.servlet.http.HttpServletRequest;
11  import javax.servlet.http.HttpServletResponse;
12
13  import board.model.service.BoardService;
14  import board.model.vo.Board;
15  import board.model.vo.BoardPage;
16
17  // 게시글 페이지별 조회 처리용 컨트롤러
18  @WebServlet("/boardlist")
19  public class BoardListServlet extends HttpServlet {
20     private static final long serialVersionUID = 1L;
21
22        public BoardListServlet() {}
23
24     protected void doGet(HttpServletRequest request, HttpServletResponse response)
25  throws ServletException, IOException {
26        response.setContentType("text/html; charset=utf-8");
27
28        // 페이지 값 처리용
29        int currentPage = 1;
30        // 한 페이지당 출력할 목록 갯수
31        int limit = 10;
```

```
32
33      // 전달된 페이지값 추출
34      if(request.getParameter("page") != null)
35          currentPage = Integer.parseInt(request.getParameter("page"));
36
37      BoardService bservice = new BoardService();
38
39      // 전체 게시글 개수와 해당 페이지별 목록을 리턴받음
40      int listCount = bservice.getListCount();
41       ArrayList<Board> list = bservice.selectList(currentPage, limit);
42
43      //  총 페이지 수 계산 : 목록이 최소 1개일 때 1page로 처리하기 위해 0.9를
44  더한다.
45      int maxPage = (int)((double)listCount / limit + 0.9);
46
47      // 현재 페이지에 보여줄 시작 페이지 번호 (1, 11, 21, .....)
48      // 예를 들어 현재 페이지가 13page 라면, 시작 페이지는 11page가 되어야 한
49  다
50      int startPage = (((int)((double)currentPage / limit + 0.9)) - 1) * limit + 1;
51
52      // 만약, 목록으로 보여질 페이지 수가 10개 이면, 끝 페이지는 20page가 되어
53  야 한다
54      int endPage = startPage + limit - 1;
55      if(maxPage < endPage)
56          endPage = maxPage;
57
58      RequestDispatcher view = null;
59      if(list != null && list.size() > 0){
60          view = request.getRequestDispatcher("views/board/boardListView.jsp");
61          request.setAttribute("list", list);
62          request.setAttribute("currentPage", currentPage);
63          request.setAttribute("maxPage", maxPage);
64          request.setAttribute("startPage", startPage);
```

```
65          request.setAttribute("endPage", endPage);
66          request.setAttribute("listCount", listCount);
67
68          view.forward(request, response);
69       }else{
70          view = request.getRequestDispatcher("views/board/boardError.jsp");
71          request.setAttribute("message", "게시글 페이지별 조회 실패");
72          view.forward(request, response);
         }
      }
   }
```

- BoardService 구현

board.model.service.BoardService.java

```
01  package board.model.service;
02
03  import java.util.*;
04
05  import board.model.vo.Board;
06  import board.model.dao.BoardDao;
07
08  public class BoardService {
09     public BoardService(){}
10
11     public int getListCount() {
12        return new BoardDao().getListCount();
13     }
14
15     public ArrayList<Board> selectList(int currentPage, int limit) {
16        return new BoardDao().selectList(currentPage, limit);
17     }
18  }
```

- BoardDao 구현

board.model.dao.BoardDao.java

```
01  package board.model.dao;
02
03  import java.util.*;
04
05  import org.apache.ibatis.io.Resources;
06  import org.apache.ibatis.session.RowBounds;
07  import org.apache.ibatis.session.SqlSession;
08  import org.apache.ibatis.session.SqlSessionFactoryBuilder;
09
10  import board.model.vo.Board;
11
12  public class BoardDao {
13    public BoardDao() {}
14    private SqlSessionFactory getSqlSessionFactory() {
15        String resource = "mybatis/mybatis-config.xml";
16        SqlSessionFactory factory = null;
17        try {
18            InputStream inputStream = Resources.getResourceAsStream(resource);
19            SqlSessionFactoryBuilder builder = new SqlSessionFactoryBuilder();
20            factory = builder.build(inputStream);
21        } catch (IOException e) {
22            e.printStackTrace();
23        }
24        return factory;
25     }
26
27    public int getListCount() {
28    // 총 게시글 개수를 리턴하는 메소드
29      int listCount = 0;
30      SqlSession session = null;
31      try {
```

```
32          session = getSqlSessionFactory().openSession(false);
33          listCount = session.selectOne("Board.listCount");
34        } catch (Exception e) {
35          e.printStackTrace();
36        }finally{
37          session.close();
38        }
39        return listCount;
40      }
41
42      public ArrayList<Board> selectList(int currentPage, int limit) {
43        // 페이지 단위로 게시글 목록 조회
44        ArrayList<Board> list = null;
45
46      /* // # 1. ROWNUM을 사용하는 경우
47        // 직접 시작 행과 끝나는 행을 계산하여 Map의 형태로 값을 전달한다.
48         int startRow = (currentPage - 1) * limit + 1; // 시작 행
49         int endRow = startRow + limit - 1; // 끝나는 행
50         HashMap<String, Integer> boardPage = new HashMap<String, Integer>( );
51         boardPage.put("start", startRow);
52         boardPage.put("end", endRow);
53         try {
54           session = getSqlSessionFactory().openSession(false);
55           // 생성한 HaspMap을 selectList() 메소드의 인자로 전달한다
56            list = new ArrayList<Board>(session.selectList("Board.selectAll", boardPage));
57         } catch (Exception e) {
58           e.printStackTrace();
59         }finally{
60           session.close();
61         }  */
62
63        // # 2. Rowbounds 를 사용하는 경우
64        // 대상의 시작 행과 제한 개수를 인자로 Rowbounds 객체를 생성하여 전달
```

```
65  한다.
66        SqlSession session = null;
67        int startRow = (currentPage-1) * limit;
68
69       try {
70          session = getSqlSessionFactory().openSession(false);
71          RowBounds row = new RowBounds(startRow, limit);
72          // Parameter Object가 없으므로, 가운데 인자를 null로 하여 전달한다.
73          list = new ArrayList<Board>(session.selectList("Board.selectAll", null, row));
74       } catch (Exception e) {
75          e.printStackTrace();
76       }finally{
77          session.close();
78       }
79       return list;
     }
```

- Board Mapper 구현

src/board/model/mapper/board-mapper.xml

```
01 <?xml version="1.0" encoding="utf-8"?>
02 <!DOCTYPE mapper PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
03 "http://mybatis.org/dtd/mybatis-3-mapper.dtd">
04 <mapper namespace="Board">
05     <resultMap id="resultBoard" type="Board">
06     <!-- prop 는 필드명, column 은 DB 컬럼 명 -->
07         <id property="boardNum" column="BOARD_NUM" />
08         <result property="boardTitle" column="BOARD_TITLE" />
09         <result property="boardWriter" column="BOARD_WRITER" />
10         <result property="boardContent" column="BOARD_CONTENT" />
11         <result property="boardOriginalFileName"
12 column="BOARD_ORIGINAL_FILENAME" />
13         <result property="boardRenameFileName"
14 column="BOARD_RENAME_FILENAME" />
15         <result property="boardDate" column="BOARD_DATE" />
16         <result property="boardLevel" column="BOARD_LEVEL" />
17         <result property="boardRef" column="BOARD_REF" />
18         <result property="boardReplyRef" column="BOARD_REPLY_REF" />
19         <result property="boardReplySeq" column="BOARD_REPLY_SEQ" />
20         <result property="boardReadCount" column="BOARD_READCOUNT" />
21     </resultMap>
22
23     <!-- 1. ROWNUM을 사용하는 경우 -->
24     <!-- <select id="selectAll" parameterType="BoardPage" resultType="arraylist"
25             resultMap="resultBoard">
26         쿼리 내부에 부등호를 사용할 경우, 태그의 구분자('<', '>')로 인식하기 때문
27 에
28          <![CDATA[ "Query" ]]> 코드로 쿼리를 감싸  부등호로 인식시켜야 한다.
29         <![CDATA[
30         SELECT * FROM (SELECT ROWNUM RNUM, BOARD_NUM, BOARD_TITLE,
31 BOARD_WRITER,
```

```
32          BOARD_CONTENT,                          BOARD_ORIGINAL_FILENAME,
33  BOARD_RENAME_FILENAME,
34          BOARD_DATE,     BOARD_LEVEL,     BOARD_REF,     BOARD_REPLY_REF,
35  BOARD_REPLY_SEQ,
36          BOARD_READCOUNT
37          FROM (SELECT * FROM BOARD ORDER BY BOARD_REF DESC,
38  BOARD_REPLY_REF DESC,
39          BOARD_LEVEL, BOARD_REPLY_SEQ))
40        WHERE RNUM >= #{start} AND RNUM <= #{end}
41        ]]>
42    </select> -->
43
44    <!-- 2. Rowbounds 을 사용하는 경우 -->
45    <select id="selectAll" resultType="arraylist" resultMap="resultBoard">
46        <![CDATA[
47        SELECT * FROM (SELECT ROWNUM RNUM, BOARD_NUM, BOARD_TITLE,
48  BOARD_WRITER,
49          BOARD_CONTENT,                          BOARD_ORIGINAL_FILENAME,
50  BOARD_RENAME_FILENAME,
51          BOARD_DATE,     BOARD_LEVEL,     BOARD_REF,     BOARD_REPLY_REF,
52  BOARD_REPLY_SEQ,
          BOARD_READCOUNT
          FROM (SELECT * FROM BOARD ORDER BY BOARD_REF DESC,
    BOARD_REPLY_REF DESC,
        BOARD_LEVEL, BOARD_REPLY_SEQ))
        ]]>
    </select>

    <select id="listCount" resultType="_int">
        SELECT COUNT(*) FROM BOARD
    </select>
</mapper>
```

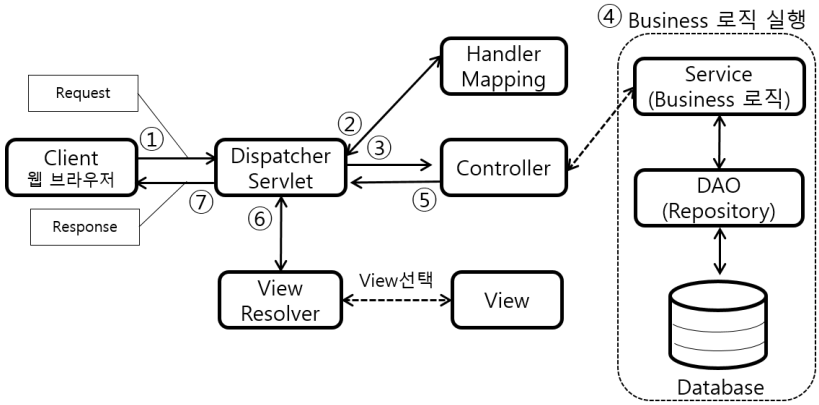
Chapter 6. ResultMap

1 스프링 MVC

스프링은 비즈니스 로직을 표현하는 도메인 모델(Model)과 프레젠테이션을 표현하는 View를 분리하고 사이에 컨트롤러를 배치하도록 설계한 디자인 패턴인 MVC 패턴을 지원한다

즉, 스프링 MVC 란 스프링이 제공하는 웹 어플리케이션 구축 전용 MVC 프레임워크를 말한다. 스프링 MVC를 이용해 Model, View, Controller 사이의 의존 관계를 DI 컨테이너에서 관리하고 통일되고 유연한 웹 어플리케이션을 구축할 수 있다.

2 스프링 MVC의 클라이언트 요청 처리 과정



① 웹 브라우저의 요청(Request)이 DispatcherServlet 인스턴스로 송신된다.

② RequestURL 과 Request 컨트롤러의 맵핑을 관리하는 HanadlerMapping 인스턴스와 DispatcherServlet 인스턴스를 참조하여 HandlerMapping 인스턴스로부터 반환된 Controller 로 파라미터를 전달한다.

③ 파라미터가 전달된 Controller 는 필요한 비즈니스 로직을 호출하여 요청 정보를 전달한다.

④ Service <-> DAO <-> Database 을 통해 일련의 작업을 처리 후 Controller 에 결과를 리턴한다

⑤ Controller 에서 처리 결과(모델)와 이동할 View 정보를 ModeAndView 객체에 담아 DispatcherServlet 에 반환한다.

(모델과 View 는 스프링 MVC 가 제공하는 ModeAndView 인스턴스로 취급된다)

⑥ DispatcherServlet 인스턴스는 View 의 실체를 ViewResolver 인스턴스에 확인한다. (Controller 인스턴스로부터 반환된 View 는 논리 정보이기 때문이다)

⑦ DispatcherServlet 은 ViewResolver 인스턴스에 의해 처리된 View 인스턴스의 모델을 렌더링하여 처리 결과를 브라우저에 표시한다.

3 스프링 MVC의 구성요소

구성 요소	설명
DispatcherServlet	클라이언트의 요청을 전달받음 요청에 맞는 컨트롤러가 리턴한 결과값을 View에 전달하여 알맞은 응답을 생성
HandlerMapping	클라이언트의 요청 URL을 어떤 컨트롤러가 처리할지 결정
Controller	클라이언트의 요청을 처리한 뒤, 결과를 DispatcherServlet에게 리턴
ModelAndView	컨트롤러가 처리한 결과 정보 및 뷰 선택에 필요한 정보를 담음
ViewResolver	컨트롤러의 처리 결과를 생성할 뷰를 결정
View	컨트롤러의 처리 결과 화면을 생성, JSP나 Velocity 템플릿 파일 등을 뷰로 사용

4 스프링 MVC을 이용한 개발

① 클라이언트의 요청을 받을 DispatcherServlet을 web.xml 파일에 설정

② 클라이언트의 요청을 처리할 컨트롤러 작성

③ ViewResolver 설정 (컨트롤러가 전달한 값을 이용해서 응답 화면을 생성할 뷰를 결정)

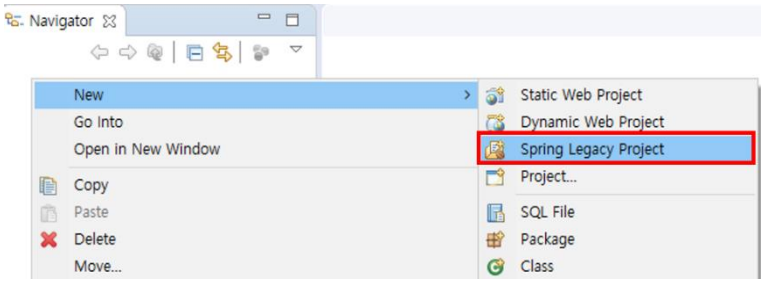
④ JSP나 Velocity 등을 이용하여 뷰 영역의 코드를 작성

5 스프링 MVC를 이용하여 Hello World 출력하기

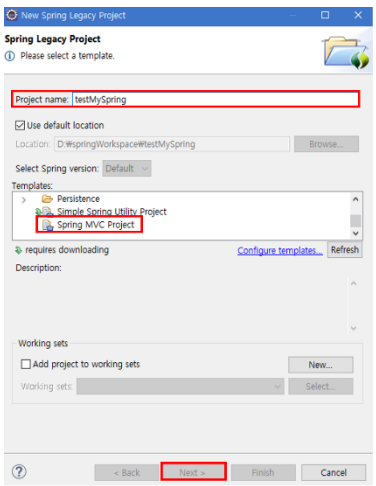
스프링 MVC를 이용하여 Hello World를 출력하는 웹 어플리케이션을 작성해 본다

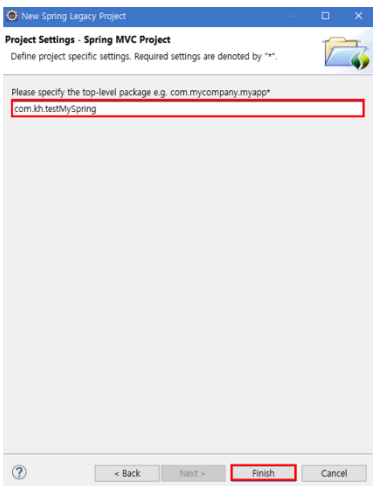
5.1 Spring Legacy Project 를 생성

ProjectExplorer에서 우클릭하여 [New] - [Spring Legacy Project] 를 선택한다.



프로젝트 생성 창에서 프로젝트 명과 Templates 의 'Spring MVC Project' 를 선택하고 Next 를 눌러 패키지 선택창으로 넘어간다. 패키지 작성은 최소 3 단계로 나누어 XXX.OOO.프로젝트명 형식으로 생성한다. 이렇게 패키지를 나누는 까닭은 프로젝트의 그룹을 패키지 이름을 통해 알기 쉽게 하기 위함이다. 만약 회사에서 여러 프로젝트를 진행하는 경우, 해당 프로젝트의 패키지 구성을 보게 되면 어떤 회사의 어떤 부서에서 작성한 프로그램인지 알기 쉬울 것이다.





5.2 pom.xml 및 Project Facets 설정 변경

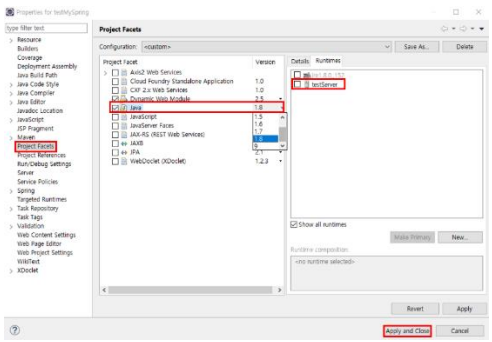
Templates의 기본형으로 프로젝트가 생성된 경우, 메이븐에서 사용하는 pom.xml과 프로젝트의 설정인 facet 정보를 수정해야 한다. 각각 다음과 같이 수정하자.

- pom.xml 에서 JDK 버전과 spring의 버전을 우리가 사용하는 버전과 동일하게 변경한다.

```
<project xmlns=http://maven.apache.org/POM/4.0.0
 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/maven-v4_0_0.xsd">
<!-- ... 중략 ... -->
        <properties>
                <java-version>1.8</java-version>
                <org.springframework-
version>5.0.2.RELEASE</org.springframework-version>
                <org.aspectj-version>1.6.10</org.aspectj-version>
                <org.slf4j-version>1.6.6</org.slf4j-version>
        </properties>
    <!--  ... 중략 ...  -->
</project>
```

변경이 완료되었다면, Project Explorer에서 프로젝트를 선택 후 우클릭, [Properties] - [Project facets]에서 자바 버전을 사용하는 버전으로 변경하고, 우측 Runtime에서 구동할 서버 환경을 선택한 뒤,

Apply and Close 를 선택하여, 적용하도록 한다.



5.3 Dispatcher Servlet 설정

web.xml 파일에서 DispatcherServlet을 설정하고, init-param 속성으로 DispatcherServletdp 관련된 설정을 등록한다.

```
<servlet>
    <servlet-name>appServlet</servlet-name>
    <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
    <init-param>
        <param-name>contextConfigLocation</param-name>
        <param-value>/WEB-INF/spring/appServlet/servlet-context.xml</param-
value>
    </init-param>
    <load-on-startup>1</load-on-startup>
</servlet>
```

5.4 공통 어플리케이션 컨텍스트 설정

web.xml 파일에 어플리케이션 컨텍스트를 설정한다

```
<!-- *.do로 들어오는 클라이언트의 요청을 DispatcherServlet이 처리하도록 설정한
다. -->
  <servlet-mapping>
      <servlet-name>appServlet</servlet-name>
      <url-pattern>*.do</url-pattern>
  </servlet-mapping>
```

DispatcherServlet은 WEB-INF/ 폴더 내에 있는 [서블릿이름]-servlet.xml 파일을 스프링 설정파일로 사용한다

- 설정이 완료된 web.xml 파일

WEB-INF/web.xml

```
01 <?xml version="1.0" encoding="UTF-8"?>
02 <web-app version="2.5" xmlns="http://java.sun.com/xml/ns/javaee"
03     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
04     xsi:schemaLocation="http://java.sun.com/xml/ns/javaee
05 http://java.sun.com/xml/ns/javaee/web-app_2_5.xsd">
06    <servlet>
07        <servlet-name>appServlet</servlet-name>
08        <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-
09 class>
10        <init-param>
11            <param-name>contextConfigLocation</param-name>
12            <param-value>/WEB-INF/spring/appServlet/servlet-context.xml</param-
13 value>
14        </init-param>
15        <load-on-startup>1</load-on-startup>
16    </servlet>
17
18    <servlet-mapping>
19        <servlet-name>appServlet</servlet-name>
20        <url-pattern>*.do</url-pattern>
    </servlet-mapping>
</web-app>
```

5.5 스프링 설정 파일 추가

5.5.1 어노테이션 자동 인식 설정

DispatcherServlet이 스프링 컨테이너에서 어노테이션으로 표기한 객체들을 검색할 수 있도록 스프링 설정파일(WEB-INF/spring/appServlet/servlet-context.xml)에 <annotation-driven /> 을 등록한다

```
<annotation-driven />
```

5.5.2 ViewResolver 등록

DispatcherServlet이 스프링 컨테이너에서 컨트롤러를 객체를 검색할 수 있도록 스프링 설정파일에 컨트롤러 빈을 등록한다

```
<beans:bean
class="org.springframework.web.servlet.view.InternalResourceViewResolver">
    <beans:property name="prefix" value="/WEB-INF/views/" />
    <beans:property name="suffix" value=".jsp" />
<!—ViewResolver 가 "/view/뷰이름.jsp" 형식의 뷰 JSP를 사용하도록 설정한다. →
</beans:bean>
```

다음과 같이 등록해두면, 사용자가 어떠한 서비스를 요청하고 결과 URL을 받을 때, 컨트롤레에서 넘겨주는 ViewName의 앞부분엔 /WEB-INF/views/ 를, 뒷부분엔 .- 58 -sp확장자를 붙여 응답화면을 전송한다.

- 완성된 servlet-context.xml 설정

WEB-INF/spring/appServlet/servlet-context.xml

```
01 <?xml version="1.0" encoding="UTF-8"?>
02 <beans:beans xmlns="http://www.springframework.org/schema/mvc"
03   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
04   xmlns:beans="http://www.springframework.org/schema/beans"
05   xmlns:context="http://www.springframework.org/schema/context"
06   xsi:schemaLocation="http://www.springframework.org/schema/mvc
07 http://www.springframework.org/schema/mvc/spring-mvc.xsd
08       http://www.springframework.org/schema/beans
09 http://www.springframework.org/schema/beans/spring-beans.xsd
10       http://www.springframework.org/schema/context
11 http://www.springframework.org/schema/context/spring-context.xsd">
12 <!—Spring MVC @Controller 를 통한 의존성 주입 →
13   <annotation-driven />
14
15   <!—ViewResolver 설정 등록 →
16   <beans:bean
17 class="org.springframework.web.servlet.view.InternalResourceViewResolver">
18       <beans:property name="prefix" value="/WEB-INF/views/" />
19       <beans:property name="suffix" value=".jsp" />
20   </beans:bean>
   </beans:beans>
```

5.6 컨트롤러 작성

@Controller 어노테이션을 적용한 컨트롤러 클래스를 구현한다.

Com.kh.testMySpring.HomeController.java

```
01 package com.kh.testMySpring;
02
03 import java.util.Calendar;
04
05 import org.springframework.stereotype.Controller;
06 import org.springframework.web.bind.annotation.RequestMapping;
07 import org.springframework.web.bind.annotation.RequestMethod;
08 import org.springframework.web.servlet.ModelAndView;
09
10 @Controller
11 public class HomeController {
12 // @RequestMapping 어노테이션을 통해 클라이언트의 요청을 처리할 메소드를
13 매핑한다
14   @RequestMapping(value = "hello.do", method = RequestMethod.GET)
15   public ModelAndView hello(ModelAndView mv) {
16       mv.setViewName("hello");
17       mv.addObject("message", getGreeting());
18
19       return mv;
20   }
21   private String getGreeting() {
22       int hour = Calendar.getInstance( ).get(Calendar.HOUR_OF_DAY);
23       StringBuilder sb = new StringBuilder();
24       sb.append("안녕하세요. ");
25        if (hour >= 6 && hour <= 10) {
26          sb.append("좋은 아침입니다.");
27        } else if (hour >= 12 && hour <= 15) {
28          sb.append("점심 식사는 하셨나요?");
29
```

```
30          } else if (hour >= 18 && hour <= 21) {
31             sb.append("좋은 밤 되세요");
32          }
33          return sb.toString();
        }
    }
```

"/hello.do"로 처리할 경로를 지정하면서 http://localhost:port/testMySpring/hello.do 라는 URL로 접근한 요청을 HelloController 클래스의 hello( ) 메소드가 처리하게 된다

5.7 뷰 작성

WEB-INF/views/hello.jsp

```
01 <%@ page language="java" contentType="text/html; charset=UTF-8"
02 pageEncoding="UTF-8"%>
03 <!DOCTYPE html>
04 <html>
05 <head>
06 <meta charset="UTF-8">
07 <title>Test Spring MVC</title>
08 </head>
09 <body>
10 인사말 : <strong>${message}</strong>
11 </body>
   </html>
```

HomeController 의 addObject( )메소드를 통해 전달한 message 값을 뷰 JSP에서 사용할 수 있게 된다.

6 DispatcherServlet 설정과 ApplicationContext의 관계

DispatcherServlet은 클라이언트의 요청을 중앙에서 처리하는 스프링 MVC의 핵심 구성 요소이다

web.xml 파일에 한 개 이상의 DispatcherServlet을 설정할 수 있으며 각 DispatcherServlet은 한 개의 WebApplicationContext를 갖게 된다. 또한 각 DispatcherSerlvet이 공유하는 빈을 설정할 수 있다.

6.1 DispatcherServlet 설정

DispatcherServlet은 기본적으로 웹 어플리케이션의 /WEB-INF/[서블릿이름]-servlet.xml 파일로부터 스프링 설정 정보를 읽어온다.

```
<servlet>
    <servlet-name>dispatcher</servlet-name>
    <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-
class>
    <init-param>
        <param-name>contextConfigLocation</param-name>
        <param-value>
            /WEB-INF/beans.xml
            /WEB-INF/main.xml
        </param-value>
    </init-param>
</servlet>
```

한 개 이상의 설정파일을 사용해야 하는 경우나 기본설정이 아닌 다른 이름을 사용하고 싶다면 contextConfigLocation 초기화 파라미터를 설정하면 된다. contextConfigLocation 초기화 파라미터는 설정 파일 목록을 값으로 갖는데 이때 각 설정파일은 콤마(","), 공백 문자(" "), 탭("₩t"), 줄바꿈("₩n"), 세미콜론(";")을 이용하여 구분한다.

6.2 웹 어플리케이션을 위한 ApplicationContext 설정

DispatcherServlet은 그 자체가 서블릿이기 때문에 한 개 이상의 DispatcherServlet을 설정하는 것이 가능

```
<servlet>
  <servlet-name>dispatcher</servlet-name>
  <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
  <init-param>
    <param-name>contextConfigLocation</param-name>
    <param-value>
      /WEB-INF/beans.xml
      /WEB-INF/main.xml
    </param-value>
  </init-param>
</servlet>
```

두 DispatcherServlet은 각각 별도의 WebApplicationContext를 생성하게 된다
단, front.xml에서는 rest.xml에 설정된 빈 객체를 사용할 수 없다는 점을 주의하자.

만약 공통의 빈의 필요로 하는 경우 ContextLoaderListener를 사용하여 공통으로 사용될 빈을 설정할 수 있다.

```
<context-param>
  <param-name>contextConfigLocation</param-name>
  <param-value>
    /WEB-INF/service.xml
    /WEB-INF/persistence.xml
  </param-value>
</context-param>
<listener>
  <listener-class>
    org.springframework.web.context.ContextLoaderListener
  </listener-class>
</listener>
<servlet>
  <servlet-name> front </servlet-name>
  <servlet-class>
    org.springframework.web.servlet.DispatcherServlet
  </servlet-class>
</servlet>
<servlet>
  < servlet — name>re st
</servlet-name>
  <servlet-class>
    org.springframework.web.servlet.DispatcherServlet
  </servlet-class>
</servlet>
```

ContextLoaderListener를 ServletListener로 등록하고 contextConfigLocation 컨텍스트 파라미터를 이용하여 공통으로 사용될 빈 정보를 담고 있는 설정 파일 목록을 설정할 수 있다.

```
  <context-param>
    <param-name>contextConfigLocation</param-name>
    <param-value>
      /WEB-INF/service.xml
      /WEB-INF/persistence.xml
    </param-value>
  </context-param>
  <listener>
    <listener-class>org.springframework.web.context.ContextLoaderListener</listener-
class>
  </listener>
  <servlet>
    <servlet-name>front</servlet-name>
    <servlet-class>
      org.springframework.web.servlet.DispatcherServlet
    </servlet-class>
  </servlet>
  <servlet>
    < servlet — name>rest</servlet-name>
    <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
  </servlet>
```

ContextLoaderListener가 생성하는 WebApplicationContext는 웹 어플리케이션에서 루트 컨텍스트가 되며 DispatcherServlet이 생성하는 WebApplicationContext는 루트 컨텍스트를 부모로 사용하는 자식 컨텍스트가 된다.

ContextLoaderListener는 contextConfigLocation 컨텍스트 파라미터를 명시하지 않으면 /WEB-INF/applicationContext.xml을 설정파일로 사용한다. 만약 클래스 패스에 위치한 파일로부터 설정 정보를 읽어오고 싶다면 "classpath:" 접두어를 사용해서 명시한다

7 캐릭터 인코딩 처리를 위한 필터 설정

요청 파라미터의 캐릭터 인코딩이 ISO-8859-1(UTF-8)이 아닌 경우 setCharacterEncoding( ) 메소드를 사용해서 인코딩을 해주어야 한다

```
response.setCharacterEncoding("UTF-8");
```

web.xml파일에 CharacterEncodingFilter 클래스를 설정함으로써, 모든 컨트롤러를 한 번에 인코딩을 적용시킬 수 있다

```
<filter>
  <filter-name>encodingFilter</filter-name>
  <filter-class>
    org.springframework.web.filter.CharacterEncodingFilter
  </filter-class>
  <init-param>
    <param-name>encoding</param-name>
    <param-value>
      UTF-8
    </param-value>
  </init-param>
</filter>
<filter-mapping>
  <filter-name>encodingFilter</filter-name>
  <url-pattern>/* </url-pattern>
</filter-mapping>
```

Chapter 7. API

1 컨트롤러 구현

스프링 3.0 부터는 @Controller 어노테이션을 이용해서 컨트롤러를 클래스를 구현하도록 권장하고 있다.

1.1 @Controller 어노테이션과 @RequestMapping 어노테이션

컨트롤러 클래스를 구현하려면 ©Controller 어노테이션과 @RequestMapping 어노테이션을 이용

- 컨트롤러 클래스에 @Controller 어노테이션을 적용
- 클라이언트의 요청을 처리할 메소드에 @RequestMapping 어노테이션을 적용
- 설정 파일에 컨트롤러 클래스를 빈으로 등록

```
@Controller
public class HelloController {
  @RequestMapping( "/hello.do")
  public String hello( ){
    return "HelloSpring";
  }
}
```

@RequestMapping 어노테이션은 해당 메서드에서 처리할 URI를 값으로 갖는다.

실제로 처리할 URI는 web.xml에서 설정한 DispatcherServlet에 설정한 매핑과 스프링 설정 파일의 전체 경로 이용 여부 설정에 따라 달라진다. Hello( ) 메소드의 리턴 타입이 String인데 이 경우 메서드의 리턴 값으로 컨트롤러의 처리 결과를 보여줄 뷰 이름으로 사용하게 된다. 컨트롤러 클래스를 구현하면 DispatcherServlet이 사용하는 스프링 설정 파일에 해당 컨트롤러 클래스를 등록해 준다

```
<bean id="helloController" class="com.lsj.spring.controller.HelloController" />
```

1.2 컨트롤러 메서드의 HTTP 전송방식 한정

하나의 요청 URR에 대해 GET 요청과 POST 요청을 한 개의 컨트롤러에서 처리해야 할 경우, @RequestMapping 어노테이션의 method 속성을 이용해서 처리할 HTTP 메서드를 제한할 수 있다.

```
@Controller
@RequestMapping("/board/newBoard.do")
public class NewBoardController {
  @RequestMapping( method = RequestMethod.GET)
  public String form( ) {
    return "board/newBoardForm";
  }

  @RequestMapping(value="/board/newBoard.do", method = RequestMethod.POST)
  public String submit () {
    return "board/newBoardSubmitted";
  }
}
```

클라이언트 요청을 처리하는 두 메소드가 동일한 URI를 처리할 경우 @RequestMapping 어노테이션을 클래스에 적용해서 해당 클래스가 처리할 기본 URI 를 지정할 수 있다

@RequestMapping 어노테이션에 method 속성을 설정하지 않을 경우 모든 HTTP 메소드를 처리하게 된다

2 뷰 지정

컨트롤러 처리 메소드는 처리 결과를 보여줄 뷰 이름이나 View 객체를 리턴하고 DispatcherServlet은 뷰 이름이나 View 객체를 이용해서 뷰를 생성한다

2.1 뷰 이름 명시적 지정

뷰이름을 명시적으로 지정하려면 ModelAndView나 String을 리턴해야 한다.

```
@RequestMapping("/index.do")
public ModelAndView index( ){
  ModelAndVew mav = new ModelAndView( "index");
  //...
  return mav;
}
```

ModelAndView를 리턴할 경우 ModelAndView 클래스의 생성자나 setViewName( ) 메서드를 이용해서 뷰 이름을 지정할 수 있다

```
ModelAndVew mav = new ModelAndView( );
mav.setViewName("search/game");
```

String 타입을 리턴할 경우 문자열 값이 뷰 이름으로 사용된다

```
@RequestMapping("/help/main.do")
public String helpMain(ModelMap model) {
  //...
  return "help/main";
}
```

2.2 뷰 이름 자동 지정

- 리턴 타입이 Model이나 Map인 경우
- 리턴 타입이 void이면서 ServletResponse나 HttpServletResponse 타입의 파라미터가 없는 경우

RequestToViewNameTranslator를 이용해서 URL로부터 뷰 이름을 결정한다. 스프링 설정 파일에 RequestToViewNameTranslator 빈이 존재하지 않을 경우 기본적으로 DefaultRequestToViewName Translator를 사용한다

- 요청 URI로부터 맨 앞의 슬래시와 확장자를 제외한 나머지 부분을 뷰 이름으로 사용

```
@RequestMapping("/search/game.do")
public Map<String, Object> search( ) {
  HashMap<String, Object> model = new HashMap<String, Object>();
  //...
  return model;
}
```

/search/game.do = = = = > search/game

2.3 리다이렉트 뷰

뷰 이름에 "redirect:" 접두어를 붙이면 지정한 페이지로 리다이렉트 된다.

```
redirect : /bbs/list - 현재 서블릿 컨텍스트에 대한 상대적인 경로로 리다이렉트
redirect : http://host//bbs/list - 지정한 절대 URL로 리다이렉트
```

ex) mav.setViewName("redirect : /error.do"); // 에러페이지로 이동

2-5. Spring View와 ViewResolver

1. 스프링(Spring) MVC : View

1.1 Outline

View 까지의 흐름

-> 클라이언트로부터의 HTTP 요청을 [DispatcherServlet]이 받음

-> [DispatcherServlet]은 [HandlerMapping]을 통해 요청 URL을 처리할 [Controller]를 검색

-> [DispatcherServlet]은 요청 URL을 처리할 [Controller]에게 처리를 요청

-> [Controller]은 요청을 적젒히 처리한 후 [ModelAndView]를 [DispatcherServlet]으로 젂달

-> [DispatcherServlet]은 [ViewResolver]를 통해 처리 결과를 보여줄 [View]를 검색

-> [DispatcherServlet]은 처리 결과를 보여줄 [View]를 출력

View

- DispatcherServlet은 ViewResolver를 사용하여 결과 출력 View 객체를 구함
- 해당 View 객체를 통해 출력 내용을 생성

2. ViewResolver

컨트롤러는 결과 생성 뷰의 이름을 지정

```
1    public class SomeController extends AbstraceController{
2    . . .
3    @Override
4    protected ModelAndView handleRequestInternal(HttpServletRequest request,
5    HttpServletResponse response) throws Exception{
6    . . .
7    ModelAndView modelAndView = new ModelAndView();
8    . . .
9    modelAndView.setViewName("someView");
10   return modelAndView;
11   }
```

ViewResolver

- 컨트롤러에 지정한 뷰 이름으로 ViewResolver를 통해 View 객체를 구함

ViewResolver 구현 클래스

- InternalResourceViewResolver

-> 뷰 이름으로 JSP, Tiles 연동을 위한 View 객체 리턴

- VelocityViewResolver

-> 뷰 이름으로 Velocity 연동을 위한 View 객체 리턴

- BeanNameViewResolver

-> 뷰 이름과 동일한 이름을 가지는 빈 객체를 View 객체로 사용

- ResourceBundleViewResolver

-> 뷰 이름과 View 객체 매핑 정보 저장을 위해 파일 사용

- XmlViewResolver

-> 뷰 이름과 View 객체 매핑 정보 저장을 위해 XML 파일 사용

ViewResolver 인터페이스

- 뷰 이름과 지역화 설정을 위한 Locale을 파라미터로 젂달받음
- 매핑된 View 객체를 리턴 (존재하지 않을 시 null)

```
1       package org.springframework.web.servlet;
2       import java.util.Locale;
3       public interface ViewResolver{
4       View resolveViewName(String viewName, Locale locale) throws
5       Exception;
6       }
```

Spring MVC 에서 View 는 사용자에게 결과를 랜더링하여 보여주는 Bean 이다.

- InternalResourceViewResolver

: 논리적인 View 이름을 사용하여 템플릿 파일 장원을 사용하여 랜더링 되는 View 객체를 결정한다.

- BeanNameViewResolver

: 논리적인 View 이름을 사용해 DispatcherServlet 의 어플리케이션 컨텍스트에 있는 View Bean 을 결정한다.

- ResourceBundleViewResolver

: 논리적인 View 이름을 사용해 ResourceBundle 에 포함돼있는 View 를 결정한다.

- XmlViewResolver

: DispatcherServlet 의 어플리케이션 컨텍스트와는 별도로, XML 파일로부터 View Bean 을 결정한다.

1) 템플릿 뷰의 사용

InternalResourceViewResolver 는
웹 어플리케이션 컨텍스트에 존재하는 템플릿에게 랜더링에 대한 책임을 위임하는 View 객체를
논리적으로 View 이름을 사용하여 결정한다.
이는 ModelAndView 객체로부터 리턴된 논리적인 이름을 취하고,
여기에 웹 어플리케이션 내의 템플릿의 경로에 도달하기 위한 접두어와 접미어를 붙이는 것으로 가능하다.

```
<bean id="viewResolver"
      class="org.springframework.web.servlet.view.InternalResourceViewResolver">
    <property name="viewClass">
        <value>org.springframework.web.servlet.view.InternalResourceView</value>
    </property>
    <property name="prefix"><value>/WEB-INF/jsp/</value></property>
    <property name="suffix"><value>.jsp</value></property>
</bean>
```

/WEB-INF/jsp/courseDetail.jsp
/WEB-INF/jsp/ : 접두어, courseDetail : 논리적인 뷰 이름, .jsp : 접미어

View 객체는 기본적으로 InternalResourceViewResolver 인데,
이 객체는 실제 랜더링을 수행할 JSP 에 단순히 요청을 전달하는 일을 한다.
만약 JSTL 태그를 사용한다면 viewClass 특성을 설정함으로써
InternalResourceView 를 JstlView 로 대체해야 한다.
JstlView 도 요청을 JSP 에 전달한다.

```
<bean id="viewResolver"
      class="org.springframework.web.servlet.view.InternalResourceViewResolver">
    <property name="viewClass">
        <value>org.springframework.web.servlet.view.JstlView</value>
    </property>
```

```
    <property name="prefix"><value>/WEB-INF/jsp/</value></property>
    <property name="suffix"><value>.jsp</value></property>
</bean>
```

2) 뷰 빈 결정

JSP 나 그 밖의 자원에 의해 표현되는 View 가 아닐 경우 InternalResourceViewResolver 대신 다른 뷰 리졸버를 선택하는 것이 좋다.
BeanNameViewResolver 는 논리적인 뷰 이름을 어플리케이션 컨텍스트의 Bean 이름과 부합시키는 뷰 리졸버이다.

```
<bean id="beanNameViewResolver"
      class="org.springframework.web.servlet.view.BeanNameViewResolver" />
```

위의 설정 후 컨트롤러가 특정 논리적인 View 이름과 함께 ModelAndView 를 리턴하면, BeanNameViewResolver 는 논리적인 View 이름의 Bean 을 찾는다.

```
<bean id="courseList" class="com.springinaction.training.mvc.CourseListPdfView" />
```

■ 별도의 XML 파일에서 뷰 빈 선언
View Bean 을 메인 어플리케이션 컨텍스트에서 찾는 대신 별도의 XML 파일에 의뢰한다.

```
<bean id="viewResolver" class="org.springframework.web.servlet.view.XmlViewResolver">
    <property name="location">
        <value>/WEB-INF/training-views.xml</value>
    </property>
</bean>
```

XmlViewResolver 는 기본적으로 WEB-INF/views.xml 에서 View 의 정의를 찾는다.
그러나 여기서는 location 특성에 /WEB-INF/training-views.xml 을 지정하여 기본 값을 재정의 했다.
XmlViewResolver 는 DispatcherSevlet 의 컨텍스트 설정 파일에 다수의 View Bean 을 선언하는 경우에 유용하다.

메인 컨텍스트 설정 파일을 깨끗하고 깔끔하게 유지하기 위해서는 View 의 선언을 나머지 Bean 으로부터 분리해야 할 필요가 있다.

■ 리소스 번들을 사용한 뷰 결정

프로퍼티 파일에서 View 의 정의를 관리한다.
프로퍼티 파일을 사용함으로써 ResourceBundleViewResolver 는 국제화와 관련한 이점을 갖는다.
다른 View 리졸버는 항상 논리적인 View 이름을 사용하여 단일한 View 구현 객체를 결정하는 반면, ResourceBundleViewResolver 는 사용자의 Locale 을 기초로 하여 동일한 논리적인 뷰 이름으로 서로 다른 View 구현 객체를 리턴할 수 있다.

```
<bean id="bundleViewResolver"
      class="org.springframework.web.servlet.view.ResourceBundleViewResolver">
    <property name="basename">
        <value>views</value>
    </property>
</bean>
```

basename 특성은 View 정의를 담고 있는 프로퍼티 파일의 이름을 어떻게 구성할지를 ResourceBundleViewResolver 에게 알려준다.
여기서는 views 라는 값을 지정했으므로 기본적으로는 views.properties 에, 미국은 views_en_US.properties, 프랑스는 views_fr_FR.properties 등으로 정의할 수 있다.
views.properties 에 JSP 기반 템플릿을 설정할 경우,

```
courseList.class=org.springframework.web.servlet.view.JstlView
courseList.url=/WEB-INF/jsp/courseList.jspz
```

views_en_US.properties 에 PDF 기반 템플릿을 설정할 경우,

```
courseList.class=com.paringinaction.training.mvc.CourseListPdfViewz
```

views_fr_FR.properties 에 EXCEL 기반 템플릿을 설정할 경우,

```
courseList.class=com.paringinaction.training.mvc.CourseListExcelView
```

3) 뷰 리졸버 선택

일반적으로 사용하기 쉬우며 간결한 InternalResourceViewResolver 사용을 권한다.
BeanNameViewResolver 와 XmlViewResolver 사이의 선택에 있어서는,
DispatcherServlet 의 컨텍스트 파일의 크기를 심각하게 증가시키지 않는
View Bean 을 가지고 있는 경우에 는 BeanNameViewResolver 를
그 반대일 경우엔 XmlViewResolver 를 선택하도록 한다.
국제화에 따른 뷰 랜더링의 경우엔 ResourceBundleViewResolver.

■ 다중 리졸버 사용

다중 뷰 리졸버를 사용하기 위해서는 단순히 컨텍스트 설정 파일에서 필요한 뷰 리졸버 Bean 을 선언해 준다.

[Spring MVC 에러처리]

Spring MVC 는 Controller 내에서 발생하는 에러 처리를 지원하기 위하여 ExceptionResolver 를 두고 있다.

ExceptionResolver 를 이용하여 에러를 처리하고자 한다면 먼저 Bean 설정 파일에 다음과 같이 설정해줘야 한다.

<< 설정 파일 : action-servlet.xml >>

```
<bean id="exceptionResolver"
class="org.springframework.web.servlet.handler.SimpleMappingExceptionResolver">
    <property name="exceptionMappings">
        <props>
            <prop key="com.xxx.user.ExistedUserException">defaultErrorMessage</prop>
            <prop key="com.xxx.user.UserNotFoundException">
                defaultErrorMessage
            </prop>
        </props>
    </property>
    <property name="exceptionAttribute" value="sampleException" />
    <property name="defaultErrorView" value="error" />
</bean>
```

2-6. Spring과 Mybatis 연동

1 Spring Mybatis 연동

이번 챕터에서는 spring과 Mybatis를 연동하여 Database Transection에 대한 의존성을 주입한다.

1.1 Maven에 라이브러리 등록

먼저 메이븐 라이브러리 관리 파일인 pom.xml에 Mybatis와 관련된 라이브러리를 등록한다.

```
<!-- ojdbc6의 경우 제공하는 서버의 파일을 읽어 오지 못하는 경우
     하단의 소스를 추가해야 한다.
  <repositories>
          <repository>
              <id>oracle</id>
              <name>ORACLE JDBC Repository</name>
              <url>https://maven.atlassian.com/3rdparty/</url>
          </repository>
   </repositories> -->
    <!-- ojdbc 라이브러리 -->
        <dependency>
            <groupId>com.oracle</groupId>
            <artifactId>ojdbc6</artifactId>
            <version>11.2.0.3</version>
        </dependency>
    <!-- MyBatis -->
        <dependency>
            <groupId>org.mybatis</groupId>
            <artifactId>mybatis</artifactId>
            <version>3.4.5</version>
        </dependency>
        <dependency>
```

```
        <groupId>org.mybatis</groupId>
        <artifactId>mybatis-spring</artifactId>
        <version>1.3.1</version>
    </dependency>
    <dependency>
        <groupId>org.springframework</groupId>
        <artifactId>spring-jdbc</artifactId>
        <version>${org.springframework-version}</version>
    </dependency>
    <!-- DBCP -->
    <dependency>
        <groupId>commons-dbcp</groupId>
        <artifactId>commons-dbcp</artifactId>
        <version>1.4</version>
    </dependency>
```

1.2 root-context.xml / mybatis-config.xml 설정

spring과 Database를 연동하기 위해서는 root-context.xml에 Database 연동과 관련된, Session 정보를 설정하고, mybatis-config.xml을 생성하여 mybatis 사용에 관한 세부 내용을 작성한다.

src/main/webapp/WEB-INF/spring/root-context.xml

```
01 <?xml version="1.0" encoding="UTF-8"?>
02 <beans xmlns="http://www.springframework.org/schema/beans"
03    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
04    xsi:schemaLocation="http://www.springframework.org/schema/beans
05 http://www.springframework.org/schema/beans/spring-beans.xsd">
06  <!-- DataSource로 database 커넥션 설정을 정의한다. -->
07  <!-- DataSource 등록은 서버연동과 같이 이루어져야 하므로, root-context.xml에
08 서 선언 -->
09    <bean id="dataSource" class="org.apache.commons.dbcp.BasicDataSource"
10       destroy-method="close">
11       <!-- destroy-method 옵션으로 사용한 SqlSession을 자동으로 Close 시킨다 -
12 ->
13       <property name="driverClassName" value="oracle.jdbc.driver.OracleDriver" />
14       <property name="url" value="jdbc:oracle:thin:@127.0.0.1:1521:xe" />
15       <property name="username" value="student" />
16       <property name="password" value="student" />
17    </bean>
18
19    <bean id="transactionManager"
20       class="org.springframework.jdbc.datasource.DataSourceTransactionManager">
21       <property name="dataSource" ref="dataSource" />
22    </bean>
23    <bean id="sqlSession" class="org.mybatis.spring.SqlSessionFactoryBean">
24       <property name="configLocation" value="classpath:mybatis-config.xml" />
25       <property name="dataSource" ref="dataSource"/>
26    </bean>
27
28    <bean class="org.mybatis.spring.SqlSessionTemplate">
```

```
29          <constructor-arg ref="sqlSession"/>
        </bean>
    </beans>
```

src/main/resources/mybatis-config.xml

```
01  <?xml version="1.0" encoding="utf-8"?>
02  <!DOCTYPE configuration PUBLIC "-//mybatis.org//DTD Config 3.0//EN"
03  "http://mybatis.org/dtd/mybatis-3-config.dtd">
04  <configuration>
05      <settings>      <!-- 입력 값이 Null일 경우 ""이 아닌 Null로 처리한다 -->
06          <setting name="jdbcTypeForNull" value="NULL"/>
07      </settings>
08
09      <typeAliases>   <!-- 사용하고자 하는 클래스의 별칭을 선언한다 -->
10          <typeAlias                  type="com.kh.testSpring.member.model.vo.Member"
11  alias="Member" />
12      </typeAliases>
13
14      <mappers>       <!-- 사용하고자 하는 쿼리를 정의한 mapper.xml을 등록한다 --
15  >
16          <mapper resource="mappers/member-mapper.xml" />
        </mappers>
    </configuration>
```

src/main/resources/mappers/member-mapper.xml

```
01  <?xml version="1.0" encoding="utf-8"?>
02  <!DOCTYPE mapper PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
03  "http://mybatis.org/dtd/mybatis-3-mapper.dtd">
04  <mapper namespace="Member">
05  <!-- config에서 선언한 별칭을 namespace로 사용하여 해당 mapper를 접근한다 -
06  ->
07      <resultMap id="resultMember" type="Member"> <!-- 원하는 포맷 결과로 출력
08  할 resultMap을 선언한다 -->
09          <id property="member_id" column="MEMBER_ID" />
10          <result property="member_password" column="MEMBER_PASSWORD" />
```

```
11          <result property="member_name" column="MEMBER_NAME" />
12          <result property="member_email" column="MEMBER_EMAIL" />
13          <result property="member_phone" column="MEMBER_PHONE" />
14          <result property="member_address" column="MEMBER_ADDRESS" />
15      </resultMap>
16
17      <select id="selectMember" parameterType="Member" resultType="Member"
18          resultMap="resultMember">
19          SELECT * FROM MEMBER WHERE MEMBER_ID = #{member_id}
20          AND MEMBER_PASSWORD = #{member_password}
21      </select>
22
23      <select id="selectMemberAll" resultType="arraylist" resultMap="resultMember">
24          SELECT * FROM MEMBER
25      </select>
26
27      <insert                 id="insertMember"                 parameterType="Member"
28  statementType="PREPARED">
29          INSERT INTO MEMBER VALUES(#{member_id}, #{member_password},
30          #{member_name},    #{member_email})
31      </insert>
32
33      <update                 id="updateMember"                 parameterType="Member"
34  statementType="PREPARED">
35          UPDATE MEMBER SET MEMBER_PASSWORD = #{member_password},
36          MEMBER_EMAIL = #{member_email}
37          WHERE MEMBER_ID = #{member_id}
38      </update>
39
40      <delete id="deleteMember" parameterType="string" statementType="PREPARED">
41          DELETE FROM MEMBER WHERE MEMBER_ID = #{userid}
        </delete>
    </mapper>
```

1.3 Dao 클래스 구현

com.kh.testSpring.member.model.dao.MemberDao.java

```
01  package com.kh.testSpring.member.model.dao;
02
03  import java.util.ArrayList;
04
05  import org.apache.ibatis.session.RowBounds;
06  import com.kh.testSpring.member.model.vo.Member;
07  import org.mybatis.spring.SqlSessionTemplate;
08  import org.springframework.beans.factory.annotation.Autowired;
09  import org.springframework.stereotype.Repository;
10
11  @Repository("memberDao")
12  public class MemberDao {
13
14      @Autowired    // bean으로 등록한 sqlSession의 의존성을 주입한다
15      private SqlSessionTemplate sqlSession;
16
17      public MemberDao(){}
18
19      public Member selectMember(Member m) {
20          // selectOne ( " Namespace . 쿼리ID ", Object )
21          return sqlSession.selectOne("Member.selectMember", m);
22      }
23
24      public ArrayList<Member> selectMemberAll() {
25          return                                                    new
26  ArrayList<Member>(sqlSession.selectList("Member.selectMemberAll"));
27      }
28    // Insert, Update, Delete는 수행한 행의 결과를 리턴한다.
29    // commit 이나 rollback의 처리는 TransactionManager에서 처리한다
30      public int insertMember(Member m) {
31          return sqlSession.insert("Member.insertMember", m);
```

```
32      }
33      public int updateMember(Member m) {
34          return sqlSession.update("Member.updateMember", m);
35      }
36      public int deleteMember(String userid) {
37          return sqlSession.delete("Member.deleteMember", userid);
38      }
    }
```

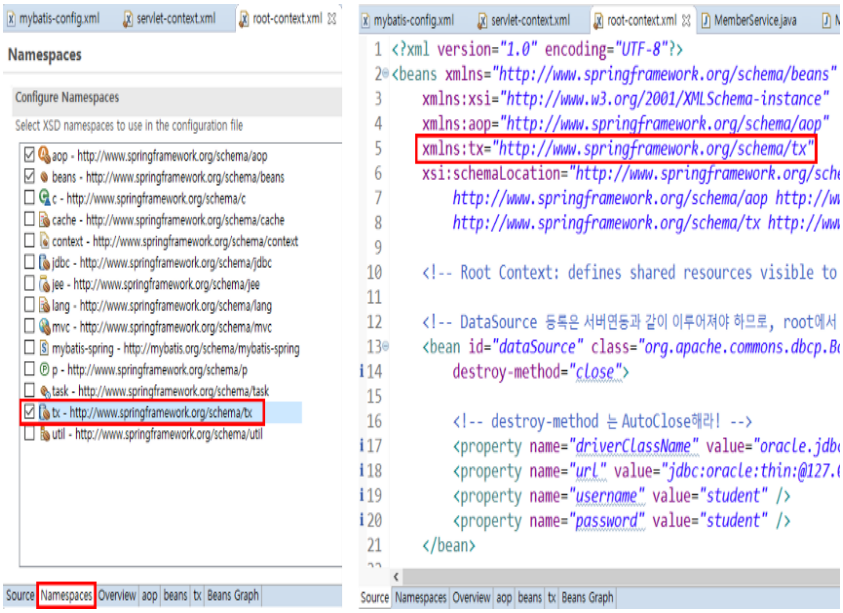
2 AOP Transaction Namespace 적용

Spring과 Mybatis를 연동하여 사용할 경우, 트랜잭션에 대한 commit(), rollback(), close() 처리는 반드시 <tx> 엘리먼트를 통해 정의해 주어야 한다. 그 이유는 Spring-mybatis를 통해 트랜잭션을 구현할 경우 AOP로 작동하며 가상의 Proxy를 통해 구동되면서 해당 메소드들을 지원할 수 없게 되기 때문이다. 만약, spring을 연동하기 이전 웹 프로젝트와 같이 SqlSession을 통해 Commit과 Rollback, Close를 구현하려고 시도하면, UnsupportedOperationException이 발생하게 될 것이다.

따라서, AOP의 namespace로 제공되는 Transaction 엘리먼트를 사용해 데이터베이스 트랜잭션에 대한 commit(), rollback(), close()를 advice로 처리하는 로직을 구현하여야 한다.

2.1 <tx> 엘리먼트를 통한 트랜잭션 적용

기존에 DataSource가 정의되어 있는 root-context.xml을 열고 namespace 탭에서 <tx> 엘리먼트를 선택하여 추가한다.



<tx> 엘리먼트는 다음과 같이 설정한다.

src/main/webapp/WEB-INF/spring/root-context.xml

```
01 <?xml version="1.0" encoding="UTF-8"?>
02 <beans xmlns="http://www.springframework.org/schema/beans"
03     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
04     xmlns:aop="http://www.springframework.org/schema/aop"
05     xmlns:tx="http://www.springframework.org/schema/tx"
06     xsi:schemaLocation="http://www.springframework.org/schema/beans
07 http://www.springframework.org/schema/beans/spring-beans.xsd
08         http://www.springframework.org/schema/aop
09 http://www.springframework.org/schema/aop/spring-aop-4.3.xsd
10         http://www.springframework.org/schema/tx
11 http://www.springframework.org/schema/tx/spring-tx-4.3.xsd">
12
13 <!-- DataSource를 통해 커넥션을 정의한다 -->
14   <bean id="dataSource" class="org.apache.commons.dbcp.BasicDataSource"
15     destroy-method="close">
16       <property name="driverClassName" value="oracle.jdbc.driver.OracleDriver" />
17       <property name="url" value="jdbc:oracle:thin:@127.0.0.1:1521:xe" />
18       <property name="username" value="student" />
19       <property name="password" value="student" />
20   </bean>
21
22  <!-- Mybatis가 사용할 Database Session을 정의한다  -->
23   <bean id="sqlSession" class="org.mybatis.spring.SqlSessionFactoryBean">
24       <property name="configLocation" value="classpath:mybatis-config.xml" />
25       <property name="dataSource" ref="dataSource"/>
26   </bean>
27
28     <!-- Mybatis가 제공하는 CRUD 구문을 사용할 수 있도록 설정한다 -->
29   <bean class="org.mybatis.spring.SqlSessionTemplate">
30       <constructor-arg ref="sqlSession"/>
31   </bean>
```

```
32
33    <!-- TransactionManager를 정의하여 Commit, Rollback을 제어한다 -->
34    <bean id="transactionManager"
35          class="org.springframework.jdbc.datasource.DataSourceTransactionManager">
36          <property name="dataSource" ref="dataSource" />
37    </bean>
38
39    <!-- Transaction 의 대상 설정 -->
40    <tx:advice id="txAdvice" transaction-manager="transactionManager">
41          <tx:attributes>
42           <!-- Transaction의 대상이 되는 메소드를 정의한다 -->
43             <tx:method name="select*" read-only="true" rollback-for="Exception"/>
44             <tx:method name="insert*" rollback-for="Exception"/>
45             <tx:method name="update*" rollback-for="Exception"/>
46             <tx:method name="delete*" rollback-for="Exception"/>
47          </tx:attributes>
48    </tx:advice>
49    <!-- <tx:annotation-driven/> -->
50
51    <!-- AOP를 통해 Transaction의 pointcut 설정 -->
52    <aop:config proxy-target-class="true">
53          <aop:pointcut id="daoMethod" expression="execution(*
54 com.kh.testSpring..*Impl.*(..)))" />
55          <aop:advisor id="transactionAdvisor" pointcut-ref="daoMethod" advice-
56 ref="txAdvice"/>
      </aop:config>
    </beans>
```

- <tx:method> 주요 속성 정리

속성	설명	
name	- 트랜잭션이 적용될 메소드 이름 - get*, *select 등, ' * ' 기호를 사용하여 범위설정이 가능하다.	
propagation	여러 메소드의 수행을 하나의 트랜잭션으로 처리할 것인지 설정(트랜잭션 전파)	
	REQUIRED	부모 트랜잭션 내에서 실행하며, 부모가 없을 경우 새로운 트랜잭션을 생성한다.
	REQUIRES_NEW	부모 트랜잭션을 무시하고 새로운 트랜잭션을 생성
	SUPPORT	부모 트랜잭션 내에서 실행하나, 부모 트랜잭션이 없을 경우에도 정상적으로 수행.
	MANDATORY	부모 트랜잭션 내에서 실행하며, 부모 트랜잭션이 없을 경우, 예외가 발생한다.
	NOT_SUPPORT	메소드가 트랜잭션을 필요로 하지 않음을 의미, 진행 중인 트랜잭션이 존재할 경우 해당 트랜잭션을 일시중지 시킨다.
	NEVER	메소드가 트랜잭션을 필요로 하지 않으며, 트랜잭션이 존재할 경우 예외가 발생한다.
	NESTED	메소드가 부모의 트랜잭션에서 실행될 경우 중첩되어 실행, 기존 트랜잭션이 존재하지 않으면 REQUIRED와 동일하다
read-only	읽기 전용 설정, 즉 트랜잭션의 Insert, Update, Delete를 금지한다.	
rollback-for	특정 예외가 발생 할 경우 rollback 처리(기본값 : RuntimeException)	
no-rollback-for	특정 예외가 발생하더라도 rollback 처리를 하지 않음(기본값 없음)	
timeout	트랜잭션의 제한시간을 설정	

2.2 @Transactional 어노테이션을 통한 Transaction 제어

root-context.xml에서 <tx:annotation-driven/>을 선언하고, 트랜잭션 제어가 필요한 클래스나 메소드 위에 @Transactional 어노테이션을 표시하여 제어한다.

```
@Transactional(
                                        // default   = REQUIRED
propagation=Propagation.REQUIRED,       // default = false
      readOnly = true,
      rollbackFor={Exception.class},
      noRollbackFor=Exception.class,    // default = -1 ( No TimeOut )
      timeout=10
)
```

- Service를 통한 Transaction 제어

com.kh.testSpring.member.model.service.MemberServiceImpl.java

```
01  package com.kh.testSpring.member.model.service;
02
03  import org.springframework.beans.factory.annotation.Autowired;
04  import org.springframework.stereotype.Service;
05  import org.springframework.transaction.annotation.Propagation;
06  import org.springframework.transaction.annotation.Transactional;
07
08  import com.kh.testSpring.member.model.dao.MemberDao;
09  import com.kh.testSpring.member.model.vo.Member;
10
11  @Service("memberService")
12  // 클래스 범위의 트랜잭션 선언
13  @Transactional(propagation=Propagation.REQUIRED, rollbackFor={Throwable.class})
14  public class MemberServiceImpl implements MemberService {
15
16      @Autowired
17      MemberDao memberDao;
18
19      @Override
20      public Member loginMember(Member m) {
21          return memberDao.memberSelect(m);
22      }
23
24      @Override
25     // 메소드 단위의 트랜잭션 선언
26      @Transactional(propagation=Propagation.REQUIRED,
27  rollbackFor={Throwable.class})
28      public int enrollMember(Member m) {
29          return memberDao.memberInsert(m);
30      }
    }
```

- Controller로 넘어온 Exception을 try-catch 로 처리.

com.kh.testSpring.member.model.service.MemberServiceImpl.java

```
01  package com.kh.myWeb.member.controller;
02
03  import org.springframework.beans.factory.annotation.Autowired;
04  import org.springframework.stereotype.Controller;
05  import org.springframework.web.bind.annotation.RequestMapping;
06  import org.springframework.web.bind.annotation.RequestMethod;
07  import org.springframework.web.servlet.ModelAndView;
08
09  import com.kh.myWeb.member.model.service.MemberService;
10  import com.kh.myWeb.member.model.vo.Member;
11
12  @Controller
13  public class MemberController {
14   @Autowired
15   MemberService memberService;
16
17   @RequestMapping(value = "txSelect.do", method = RequestMethod.POST)
18   public ModelAndView memberLoginService(Member m, ModelAndView model) {
19       try {
20          m = memberService.loginMember(m);
21          model.addObject("user", m);
22          model.setViewName("loginConfirm");
23       }catch(Exception e) {
24          System.out.println(e.getMessage()+" 가 발생하였습니다.");
25          model.setViewName("errorPage");
26       }
27       return model;
28    }
29  }
```

2-7. Spring Service와 Service implements

1. 스프링 MVC 수행 흐름

1) 클라이언트로부터의 모든 ".do" 요청을 DispatcherServlet 이 받는다.
(프로젝트명/src/main/sebapp/WEB-INF/spring/appServlet/servlet-context.xml)

2) DispatcherServlet 은 내장된 HandlerMapping 기능을 통해 요청을 처리할 Controller 를 검색(component-scan)한다.

3) DispatcherServlet 은 검색된 Controller 를 실행하여 클라이언트의 요청을 처리한다.

4) Controller 는 전달받은 클라이언트 데이터를 추출하고, 서비스의 메소드로 전달한 다음 비즈니스 로직의 수행 결과로 리턴된 정보를 Model 객체에 저장하고 Model을 보여줄 View 정보를 ModelAndView 객체에 저장 또는 뷰파일명을 DispatcherServlet 으로 리턴한다.

5) DispatcherServlet 은 ModelAndView 로부터 View 정보를 추출하거나 리턴된 뷰파일명을 내장된 ViewResolver 를 이용하여 응답할 View를 지정된 뷰 폴더 아래에서 찾는다.

6) DispatcherServlet은 ViewResolver를 통해 찾아낸 View를 실행하여 응답을 전송한다.

7) 응답된 뷰페이지는 jsp, el, jstl, ajax, javascript, html 을 이용해 브라우저에 결과를 출력한다.

2. DispatcherServlet 등록 및 스프링 컨테이너 구동

WEB-INF/web.xml 파일에 등록된 정보를 수정한다.
DispatcherServlet 파일명을 설정한 내용에 맞춰 action-servlet.xml 로 수정하거나 있는 그대로 사용해도 된다.
참고로 DispatcherServlet 파일은 여러 개를 작성할 수 있으며, web.xml 에 등록하면 된다.

다음은 폴더명과 파일명을 수정한 경우의 예이다.

```
<context-param>
	<param-name>contextConfigLocation</param-name>
	<param-value>/WEB-INF/config/root-context.xml</param-value>
</context-param>

<servlet>
	<servlet-name>action</servlet-name>
	<servlet-class>
org.springframework.web.servlet.DispatcherServlet</servlet-class>
	<init-param>
		<param-name>contextConfigLocation</param-name>
		<param-value>/WEB-INF/config/*-servlet.xml</param-
value>
	</init-param>
	<load-on-startup>1</load-on-startup>
</servlet>

<servlet-mapping>
	<servlet-name>action</servlet-name>
	<url-pattern>*.do</url-pattern>
</servlet-mapping>
```

3. 어노테이션 관련 설정

DispatcherServlet 에 <annotation-driven /> 등록 확인함.

그 다음 Controller 클래스가 검색 범위에 포함되도록 하고자 <context:component-scan> 엘리먼트의 base-package 속성에 Controller 클래스들이 있는 가장 상위 패키지를 등록한다.

<context:component-scan base-package="com.kh.hello//controller" />**

또는

<context:component-scan base-package="com.kh.hello" />

4. @Controller 사용하기

기존의 스프링 컨테이너가 Controller 클래스를 생성하려면 Controller 클래스들을 스프링 설정 파일(DispatcherServlet)에 <bean> 등록해야 했다. 어노테이션 방식으로 진행할 경우에는 컨트롤러 클래스들을 일일이 <bean> 으로 등록하지 않고 클래스 선언부 위에 "@Controller" 를 붙이면 된다.

스프링 설정 파일에 작성된 <context:component-scan> 이 컨트롤러 객체들을 자동으로 생성해 준다.

Ctrl + Shift + O 키를 눌러 import를 자동 추가한다.

5. @RequestMapping 사용하기

클라이언트의 "login.do" 요청에 대해 loginMethod() 가 실행되도록 하려면 @RequestMapping 을 이용하여 HandlerMapping 설정을 대체한다.

HandlerMapping 은 서블릿에서의 @WebServlet("login.do") 즉, 서블릿의 url-pattern 을 의미한다.

@RequestMapping 은 메소드 위에 설정한다. value 속성은 대부분 생략한다.

```
@RequestMapping(value="/login.do", method=RequestMethod.POST)
public ModelAndView loginMethod(){
        return null;
}
```

6. 클라이언트 요청 처리

대부분 Controller 는 사용자의 입력 정보를 추출하여 VO(Value Object) 에 저장한다. 그리고 비즈니스 컴포넌트의 메소드를 호출할 때 VO 를 인자로 전달한다. 사용자 입력 정보는 HttpSerevletRequest 의 getParameter( ) 메소드를 사용하여 추출한다. 그런데 사용자 입력 정보가 많을 경우와 입력 정보가 변경될 때마다 Controller 의 추출 코드가 수정되어야 할 것이다.
Command 객체를 이용하면 이런 문제를 해결할 수 있다. Command 객체는 Controller 의 메소드 매개변수로 받은 VO 객체라고 보면 된다.

```
@RequestMapping("/login.do")
public ModelAndView loginMethod(MemberVO vo){
        return null;
}
```

스프링 컨테이너가 해당 메소드를 실행할 때 Command 객체를 생성하고 사용자가 입력한 값들을 Command 객체에 셋팅까지 해서 넘겨준다. 즉, 사용자 입력 정보 추출과 VO 객체 생성, 추출값 셋팅 모두를 스프링 컨테이너가 자동으로 처리한다는 것이다.

여기서 중요한 것은 FORM 태그 안의 **INPUT 태그의 NAME 속성의 이름과 VO 객체의 필드명이 반드시 일치**해야 한다. 그리고 각 필드의 Setter 메소드가 반드시 존재해야 한다.

7. 비즈니스 컴포넌트 사용

Spring IoC 에서의 비즈니스 컴포넌트는 VO 클래스, DAO 클래스, Service 인터페이스, Service 구현 클래스 4개의 파일로 구성되어 있다. 그리고 Controller 는 DAO 를 직접 이용해서는 안되며 반드시 비즈니스 컴포넌트를 이용해야 한다.
이유는 비즈니스 컴포넌트에 다형성을 적용하여 클라이언트가 인터페이스를 통해서 비즈니스 컴포넌트를 이용하면 컴포넌트의 구현 클래스(인터페이스 후손 클래스)를 수정하거나, 다른 구현 클래스로 대체해도 이를 사용하는 클라이언트는 수정하지 않아도 되기 때문이다. 즉 비즈니스 컴포넌트가 수정되더라도 이를 사용하는 Controller 는 수정하지 않아도 되게 하려는 것이다.
Service 인터페이스를 작성하고 나서, Service 인터페이스를 상속받은 Service 구현 클래스 이름 위에 @Service("서비스이름") 어노테이션을 표시한다.

서비스 인터페이스 구현은 다음과 같다.

```
package com.kh.hello.member.service;
public interface MemberService {  추상메소드 선언  }
```

서비스 인터페이스 구현체 작성은 다음과 같다.

```
package com.kh.hello.member.service;

import org.springframework.stereotype.Service;

@Service("memberService")
public class MemberServiceImpl implements MemberService {
//DAO 사용
@Autowired
MemberDAO memberDAO;

 @Override
Public …… 상속받은 서비스 메소드 오버라이딩함.
DAO 클래스의 사용 메소드로 vo 객체 넘기고 받은 결과 리턴함
}
```

DAO 클래스 구현은 다음과 같다.

```
@Repository("memberDAO")
Public class MemberDAO{

//마이바티스 스프링 연동 모듈이 제공하는 클래스 사용
@Autowired
private SqlSessionTemplate sqlSession;

//각 서비스별 메소드 구현
}
```

컨트롤러는 서비스를 사용한다. @AutoWired 어노테이션을 사용하여 MemberService 타입의 MemberServiceImpl 객체가 의존성 주입된다.

```
@Controller
public class MemberController {

        @Autowired
        private MemberService memberService;
```

8. 비즈니스 컴포넌트 의존성 주입하기

DispatcherServlet 에서 **<context:component-scan base-package="com.kh.hello" />** 로 설정하지 않고

<context:component-scan base-package="com.kh.hello//controller" />**

으로 설정했을 경우에는 실행하면 에러가 발생할 것이다. 이 에러 메시지는 Controller 가 @Autowired 로 의존성 주입을 하려고 하는 데 Service 타입의 객체가 메모리에 없어서 의존성 주입을 할 수 없다는 에러 메시지이다.

@Autowired 어노테이션을 사용하려면 의존성 주입 대상이 되는 객체가 반드시 메모리에 먼저 로딩되어 있어야 한다.

결론은 Controller 보다 의존성 주입이 될 ServiceImpl 객체가 먼저 생성되어 있어야 하는 것이다. 그런데 DispatcherServlet (즉, action-servlet.xml) 파일에는 Controller 객체들만 컴포넌트 스캔하도록 설정했기 때문에 ServiceImpl 객체는 생성되지 않는다.

결국 Controller 보다 의존성 주입 대상이 되는 비즈니스 컴포넌트를 먼저 생성하려면 비즈니스 컴포넌트를 먼저 생성하는 또 다른 스프링 컨테이너가 필요하다. 그리고 이 컨테이너를 서블릿 컨테이너(DispatcherServlet) 보다 먼저 구동하면 된다.

src/main/resources 폴더에 비즈니스 레이어에 해당하는 설정 파일로 root-context.xml(파일명은 임의대로 설정 가능) 파일을 두고, 이 파일을 읽어 비즈니스 컴포넌트들을 메모리에 생성되게 해야 한다. 이 때 사용하는 클래스가 스프링에서 제공하는 ContextLoaderListener 이다.

web.xml 에 <listener> 태그로 등록한다.

중요한 것은 서블릿 컨테이너가 web.xml 파일을 읽어서 구동될 때 ContextLoaderListener 클래스가 자동으로 메모리에 생성된다.

즉 ContextLoaderListener 는 클라이언트의 요청이 없어도 컨테이너가 구동될 때 Pre-Loading 되는 객체이다. 그러므로 roo-context.xml 에 설정된 내용이 먼저 구동된다는 것이다.

web.xml 설정 추가

```
<!-- The definition of the Root Spring Container shared by all Servlets and Filters -->
<context-param>
        <param-name>contextConfigLocation</param-name>
        <param-value>classpath:root-context.xml</param-value>
</context-param>

<!-- Creates the Spring Container shared by all Servlets and Filters -->
<listener>
        <listener-class>
        org.springframework.web.context.ContextLoaderListener
        </listener-class>
</listener>
```

root-context.xml 파일 설정

NameSpace 에서 context 체크한 다음, 다음의 내용을 추가한다.

```
<context:component-scan base-package="com.kh.hello.**.service">
    </context:component-scan>
<context:component-scan base-package="com.kh.hello.**.dao">
    </context:component-scan>
```

1 파일 업로드 처리

파일 업로드가 필요한 경우 HTML폼의 enctype 속성을 "multipart/form-data"로 설정한다.

```
<form method="post" enctype="multipart/form-data">
```

인코딩 타입이 Multipart인 경우 파라미터나 업로드 한 파일을 구하려면 전송 데이터를 알맞게 처리해 주어야 한다. 이를 위해서 스프링은 Multipart 지원 기능을 제공하고 있기 때문에 추가적인 처리 없이 Multipart 형식으로 전송된 파라미터와 파일정보를 쉽게 구할 수 있다.

1.1 MultipartResolver 설정

Multipart 지원 기능을 사용하려면 먼저 MultipartResolver를 스프링 설정파일에 등록해 주어야 한다. MultipartResolver는 Multipart 형식으로 데이터가 전송된 경우 스프링 MVC가 사용할 수 있도록 변환해 준다.

스프링이 기본적으로 제공하는 MultipartResolver는 CommonsMultipartResolver이다. Commons MultipartResolver는 Commons FileUpload API를 이용한다.

```
<bean id="multipartResolver">
  class="org.springframework.web.multipart.commons.CommonsMutipartResolver">
</bean>
```

DispatcherServlet은 이름이 "multipartResolver"인 빈을 사용하기 때문에 다른 이름을 지정할 경우 MultipartResolver로 사용되지 않는다.

- CommonsMultipartResolver 클래스의 프로퍼티

프로퍼티	타입	설명
maxUploadSize	long	최대 업로드 가능한 바이트 크기 기본 값은 -1이다 (제한 없음)
maxInMemorySize	int	디스크에 임시 파일을 생성하기 전에 메모리에 보관할 수 있는 최대 바이트 크기 기본 값은 10240 바이트
defaultEncoding	String	요청을 파싱할 때 사용할 캐릭터 인코딩 지정하지 않을 경우 HttpServlerRequest.setCharacterEncoding( ) 메소드로 지정한 캐릭터 셋을 사용, 기본값은 ISO-8859-1(UTF-8)이다

1.2 첫번째 방법 : @RequestParam 어노테이션을 이용한 업로드 파일의 접근

업로드 한 파일을 전달받는 첫 번째 방법은 @RequestParam 어노테이션이 적용된 MultipartFile 타입의 파라미터를 사용하는 것이다.

- FORM HTML

```
<form action="submitReport1.do" method="post" enctype="multipart/form-data">
학번:<input type="text" name="studentNumber" /><br>
리포트파일:<input type="file" name="report" /><br>
<input type="submit" />
</form>
```

위 HTML 코드에서 파일은 report 파라미터를 통해서 전달되며 아래의 Controller에서와 같이@RequestParam 어노테이션과 MultipartFile 타입의 파라미터를 이용해서 업로드 파일 데이터를 전달 받을 수 있다.

- Controller

```
package com.mvc.file.controller;
// 중 략 ...
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.multipart.MultipartFile;

@Controller
public class ReportSubmissionController {

  @RequestMapping( value="/report/submission.do", method = RequestMethod.GET )
  public String form( ) {
    return "report/submissionForm";
  }

  @RequestMapping( value = "/report/submitReportl.do", method =
```

```
RequestMethod.POST )
    public String submitReport1(
        @RequestParam("studentNumber") String studentNumber,
        @RequestParam("report") MultipartFile report) {
      //
      MultipartFile이 제공하는 메소드를 이용해서 업로드 데이터 접 근
      printInfo(studentNumber, report);

      return "report/submissionComplete";
    }
    // 중 략 ...
  }
```

1.3 두 번째 방법 : MultipartHttpServletRequest를 이용한 업로드파일 접근

MultipartHttpServletRequest 인터페이스는 스프링이 제공하는 인터페이스로서 Multipart 요청이 들어올 때 내부적으로 원본 HttpServletRequest 대신 사용되는 인터페이스이다

- Controller

```
// 중략...
import org.springframework.web.multipart.MultipartHttpServletRequest;
@Controller
public class ReportSubmissionController {
  @RequestMapping( value = "/report/submitReport2.do", method = RequestMethod.POST)
  public String submitReport2(MultipartHttpServletRequest request) {

    String studentNumber = request.getParameter("studentNumber");
    MultipartFile report = request.getFile ("report");
    printInfo(studentNumber, report);
    return "report/submissionComplete";
  }
  // 중략 ...
}
```

MultipartHttpServletRequest는 인터페이스이며 실제로 어떤 메서드도 선언되어 있지 않다

MultipartHttpServlerRequest는 HttpServletRequest를 상속받기 때문에 웹 요청 정보를 구하기 위한 getParameter( )와 같은 메서드를 사용할 수 있으며 MultipartRequest가 제공하는 메서드도 사용할 수 있다

- MultipartRequest 인터페이스의 파일 관련 주요 메서드

메소드	설명
Iterator<String> getFileNames( )	업로드 된 파일의 이름 목록을 제공하는 Iterator를 구함
MultipartFile getFile(String name)	파라미터 이름이 name인 업로드 파일 정보를 구함
List<MultipartFile> getFiles(String name)	파라미터 이름이 name인 업로드 파일 정보 목록을 구함
Map <String, MultipartFile> getFile( )	파라미터 이름을 키로 파라미터에 해당하는 파일 정보를 값으로 하는 Map을 구함

1.4 세 번째 방법 : 커맨드 객체를 통한 업로드 파일 접근

커맨드 클래스에 파라미터와 동일한 이름의 MultipartFile 타입 프로퍼티를 추가해주기만 하면 커맨드 객체를 이용해 업로드 한 파일을 전달 받을 수 있게 된다.

- Command 클래스

```
import org.springframework.web.multipart.MultipartFile;

public class ReportCommand {
  private String studentNumber;
  private MultipartFile report;

  String getStudentNumber( ) {
    return studentNumber;
  }
  void setStudentNumber(String studentNumber) {
    this.studentNumber = studentNumber;
  }
  MultipartFile getReport( ) {
    return report;
  }
  void setReport(MultipartFile report) {
    this.report = report;
  }
}
```

위와 같이 MultipartFile 타입의 프로퍼티를 커맨드 클래스에 추가해 주었다면 @RequestMapping 메서드의 커맨드 객체로 사용함으로써 업로드 파일 정보를 커맨드 객체를 통해서 전달받을 수 있다.

- Controller

```
@Controller
public class ReportSubmissionController {
  // 중략 ...
  @RequestMapping( value = "/report/submitReport3.do", method = RequestMethod.POST)
  public String submitReport3 (ReportCommand report Command) {
    printInfo(reportCommand.getStudentNumber(), reportCommand.getReport());
    return "report/submissionComplete";

    // 중 략 ...
  }
}
```

2 파일 업로드 처리

2.1 web.xml에 DispatcherServlet을 등록

WEB-INF/web.xml

```
01 <?xml version="1.0" encoding="UTF-8"?>
02 <web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
03 xmlns="http://xmlns.jcp.org/xml/ns/javaee"
04 xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
05 http://xmlns.jcp.org/xml/ns/javaee/web-app_3_1.xsd" id="WebApp_ID" version="3.1">
06   <display-name>FileUp</display-name>
07   <welcome-file-list>
08     <welcome-file>index.html</welcome-file>
09   </welcome-file-list>
10
11   <filter>
12     <filter-name>encodingFilter</filter-name>
13     <filter-class>
14       org.springframework.web.filter.CharacterEncodingFilter
15     </filter-class>
16
17     <init-param>
18       <param-name>encoding</param-name>
19       <param-value>UTF-8</param-value>
20     </init-param>
21   </filter>
22
23   <filter-mapping>
24     <filter-name>encodingFilter</filter-name>
25     <url-pattern>/*</url-pattern>
26   </filter-mapping>
27
28   <servlet>
29     <servlet-name>spring</servlet-name>
```

```
30      <servlet-class>
31        org.springframework.web.servlet.DispatcherServlet
32      </servlet-class>
33    </servlet>
34
35    <servlet-mapping>
36      <servlet-name>spring</servlet-name>
37      <url-pattern>*.do</url-pattern>
38    </servlet-mapping>
      </web-app>
```

2.2 스프링 설정파일을 작성

WEB-INF/spring-servlet.xml

```
01  <?xml version="1.0" encoding="UTF-8"?>
02  <beans xmlns="http://www.springframework.org/schema/beans"
03    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
04    xmlns:context="http://www.springframework.org/schema/context"
05    xmlns:p="http://www.springframework.org/schema/p"
06    xsi:schemaLocation="http://www.springframework.org/schema/beans
07      http://www.springframework.org/schema/beans/spring-beans.xsd
08      http://www.springframework.org/schema/context
09      http://www.springframework.org/schema/context/spring-context-3.1.xsd">
10
11    <bean id="multipartResolver"
12
13  class="org.springframework.web.multipart.commons.CommonsMultipartResolver">
14    </bean>
15
16  <bean id="viewResolver"
17      class="org.springframework.web.servlet.view.InternalResourceViewResolver">
18
19      <property name="prefix" value="/WEB-INF/" />
20      <property name="suffix" value=".jsp" />
21    </bean>
    </beans>
```

2.3 리포트 제출 폼 작성

WEB-INF/report/reportForm.jsp

```
01 <?xml version="1.0" encoding="UTF-8" ?>
02 <%@ page language="java" contentType="text/html;charset=UTF-8"
03 pageEncoding="UTF-8"%>
04
05 <%@taglib uri="http://www.springframework.org/tags" prefix="spring" %>
06 <%@taglib uri="http://www.springframework.org/tags/form" prefix="form" %>
07
08 <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
09  "http://www.w3.org/TR/xhtml1l/DTD/xhtml1l-transitional.dtd">
10
11 <html xmlns="http://www.w3.org/1999/xhtml">
12 <head>
13 <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
14 <title>리포트 제출</title>
15 </head>
16 <body>
17
18   <h3>@RequestParam 사용</h3>
19   <form action="submitReport1.do" method="post"
20     enctype="multipart/form-data">
21     학번:<input type="text" name="studentNumber" /><br/>
22     리포트파일:<input type="file" name="report" /> <br/>
23     <input type="submit" />
24   </form>
25
26   <h3>MultipartHttpServletRequest 사용</h3>
27   <form action="submitReport2.do" method="post" enctype="multipart/form-data">
28     학번:<input type ="text" name="studentNumber" /><br/>
29      리포트파일:<input type="file" name="report" /> <br/>
30     <input type="submit" />
31   </form>
```

```
32
33    <h3>커맨드 객체 사용</h3>
34    <form action="submitReport3.do" method="post" enctype="multipart/form-data">
35      학번:<input type="text" name="studentNumber" /> <br/>
36       리포트파일:<input type="file" name="report" /><br/>
37      <input type="submit" />
38    </form>
39  </body>
    </html>
```

2.4 컨트롤러 작성

com.kh.testMySpring.files.controller.ReportController.java

```
01 package com.kh.testMySpring.files.controller;
02
03 import java.io.File;
04
05 import javax.servlet.http.HttpServletRequest;
06
07 import org.springframework.stereotype.Controller;
08 import org.springframework.web.bind.annotation.RequestMapping;
09 import org.springframework.web.bind.annotation.RequestMethod;
10 import org.springframework.web.bind.annotation.RequestParam;
11 import org.springframework.web.multipart.MultipartFile;
12 import org.springframework.web.multipart.MultipartHttpServletRequest;
13
14 import com.kh.testMySpring.files.model.vo.ReportCommand;
15
16 @Controller
17 public class ReportController {
18   @RequestMapping(value = "/report/reportForm.do", method =
19 RequestMethod.GET)
20   public String form() {
21     return "report/reportForm";
22   }
23   // 1. @RequestParam 사용
24   @RequestMapping(value = "/report/submitReport1.do", method =
25 RequestMethod.POST)
26   public String submitReport1(@RequestParam("studentNumber") String
27 studentNumber,
28       @RequestParam("report") MultipartFile report, HttpServletRequest request) {
29     printInfo(studentNumber, report);
30     saveFile(report, request);
31     return "report/reportComplete";
```

```
32    }
33    // 2. MultipartHttpServletRequest 사용
34    @RequestMapping(value = "/report/submitReport2.do", method =
35  RequestMethod.POST)
36    public String submitReport2(MultipartHttpServletRequest request) {
37      String studentNumber = request.getParameter("studentNumber");
38      MultipartFile report = request.getFile("report");
39
40      printInfo(studentNumber, report);
41      saveFile(report, request);
42      return "report/reportComplete";
43    }
44
45    // 3. 커맨드 객체 사용
46    @RequestMapping(value = "/report/submitReport3.do", method =
47  RequestMethod.POST)
48    public String submitReport3(ReportCommand reportCommand, HttpServletRequest
49  request) {
50      printInfo(reportCommand.getStudentNumber(), reportCommand.getReport());
51      saveFile(reportCommand.getReport(), request);
52      return "report/reportComplete";
53    }
54    // 파일 업로드 정보 출력
55    private void printInfo(String studentNumber, MultipartFile report) {
56      System.out.println(studentNumber + "가 업로드 한 파일: " +
57  report.getOriginalFilename());
58    }
59    // 실제 파일 업로드 메소드 구현
60    private void saveFile(MultipartFile report, HttpServletRequest request) {
61
62      String root = request.getSession().getServletContext().getRealPath("resources");
63      String savePath = root + "₩₩uploadFiles";
64     String filePath = null;
```

```
65      File folder = new File(savePath);
66
67      if (!folder.exists()) {
68         folder.mkdir(); // 폴더가 없다면 생성한다.
69      }
70
71      try {
72         // 파일 저장
73         System.out.println(report.getOriginalFilename() + "을 저장합니다.");
74         System.out.println("저장 경로 : " + savePath);
75
76         filePath = folder + "₩₩" + report.getOriginalFilename();
77
78         report.transferTo(new File(filePath)); // 파일을 저장한다
79         System.out.println("파일 명 : " + report.getOriginalFilename());
           System.out.println("파일 경로 : " + filePath);
           System.out.println("파일 전송이 완료되었습니다.");
        } catch (Exception e) {
           System.out.println("파일 전송 에러 : " + e.getMessage());
        }
     }
  }
```

2.5 커맨드 클래스 작성

com.kh.testMySpring.files.model.vo.ReportCommand.java

```
01  package com.kh.testMySpring.files.model.vo;
02
03  import org.springframework.web.multipart.MultipartFile;
04
05  public class ReportCommand {
06     private String studentNumber;
07     private MultipartFile report;
08
09     public String getStudentNumber() {
10        return studentNumber;
11     }
12     public void setStudentNumber(String studentNumber) {
13        this.studentNumber = studentNumber;
14     }
15
16     public MultipartFile getReport() {
17        return report;
18     }
19     public void setReport(MultipartFile report) {
20        this.report = report;
21     }
22  }
```

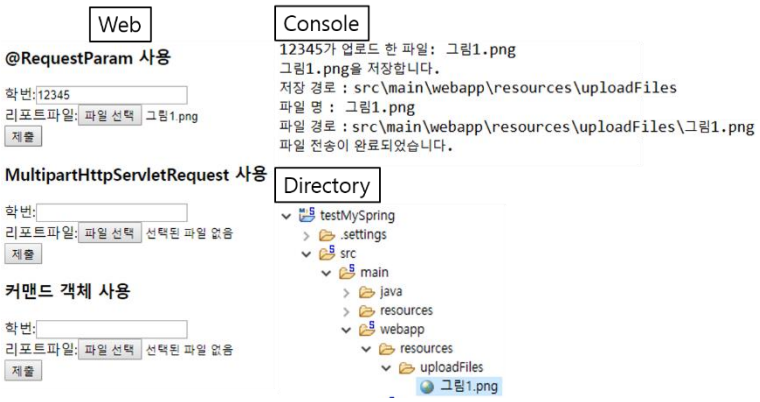
2.6 전송 완료 화면 작성

WEB-INF/report/reportComplete.jsp

```
01 <?xml version="1.0" encoding="UTF-8"?>
02 <%@ page language="java" contentType="text/html; charset=UTF-8"
03 pageEncoding="UTF-8"%>
04
05 <!DOCTYPE html>
06 <html>
07 <head>
08 <meta charset="UTF-8" />
09 <title>리포트 제출 완료</title>
10 </head>
11 <body>
12     리포트 제출 완료
   </body>
   </html>
```

2.7 확인

http://loclahost:port/[projectName]/report/reportForm.do 로 접속하여 확인할 수 있다



3 Spring Ajax 연동하기

spring에서 비동기화 통신을 처리해야 하는 경우, ajax를 통해 처리할 수 있다.

3.1 JSON 라이브러리 등록

Maven 라이브러리 설정파일인 pom.xml에 ajax에 사용될 JSON 라이브러리를 추가한다.

```
  <dependency>
        <groupId>net.sf.json-lib</groupId>
        <artifactId>json-lib-ext-spring</artifactId>
        <version>1.0.2</version>
    </dependency>

    <!-- https://mvnrepository.com/artifact/com.googlecode.json-simple/json-simple --
>
    <dependency>
        <groupId>com.googlecode.json-simple</groupId>
        <artifactId>json-simple</artifactId>
        <version>1.1.1</version>
    </dependency>
```

3.2 서버쪽 컨트롤러로 값을 보내는 방법

Client가 전달하는 데이터를 Controller로 전송하는 과정을 Ajax로 처리해본다.

JSP : <script>

```
$("#test1").on("click", function(){
      $.ajax({
        url : "test1.do",
        data : { name : "신사임당", age : 47 },
        type : "post",
        success : function(result){
          if(result == "ok"){
              alert("전송 성공!");
          } else
              alert("전송 실패!");
        },
        error : function(request, status, errorData){
            alert("error code : " + request.status + "₩n"
                + "message : " + request.responseText + "₩n"
                + "error : " + errorData);
        }
      });
});
```

Controller

```
@RequestMapping(value="test1.do", method=RequestMethod.POST)
  public void test1Method(Sample vo,
          HttpServletResponse response, HttpSession session) throws IOException{
      response.setContentType("text/html; charset=utf-8");
      // PrintWriter 객체를 생성하여 통신에 대한 응답 결과를 전달한다.
      PrintWriter out = response.getWriter();

      System.out.println("vo : " + vo);
      if(vo.getName().equals("신사임당")){
          session.setAttribute("samp", vo);
          out.append("ok");
```

```
            out.flush();
        }else{
            out.append("fail");
            out.flush();
        }
        out.close();
}
```

3.3 컨트롤러에서 전송하는 응답 결과를 화면에 출력하는 방법

Controller에서 전달하는 응답 결과를 별도의 새로고침 없이 화면에 바로 출력해보자.

JSP : <script>

```
$("#test2").on("click", function(){
    // test2.do 로부터 json 객체를 리턴받아, div 에 출력한다
    $.ajax({
      url : "test2.do",
      type : "post",
      dataType : "json",
      success : function(data){
        // 전달받은 JSONObject에 담은 Value를 Key로 접근하여 출력한다
        $("#d2").html("번호 : " + data.no
          + "<br>제목 : " + data.title
          + "<br>작성자 : " + decodeURIComponent(data.writer)
          + "<br>내용 : " + decodeURIComponent(data.content.replace(/₩+/g, " ")));
      },
      error : function(request, status, errorData){
        alert("error code : " + request.status + "₩n"
          + "message : " + request.responseText + "₩n"
          + "error : " + errorData);
      }
    });
});
```

Controller

```
@RequestMapping(value="test2.do", method=RequestMethod.POST)
  @ResponseBody  //결과를 response 객체에 담아서 보내는 어노테이션
  public String test2Method(HttpServletResponse response) throws IOException{
     response.setContentType("application/json; charset=utf-8");

    JSONObject job = new JSONObject();
    // Map 형식의 JSONObject 객체를 생성하여 출력할 값을 Key와 Value 형태로 담는다
    job.put("no", 123);
```

```
    job.put("title", "test return json object");
    // 한글 전송 시 깨질 우려가 있으므로, URLEncoder로 UTF-8방식의 인코딩을 처리한다
    job.put("writer", URLEncoder.encode("홍길동", "utf-8"));
    job.put("content", URLEncoder.encode("json 객체를 뷰로 리턴하는 테스트", "utf-8"));

    // JSONObject 를 string 형태로 리턴한다.
    return job.toJSONString();
  }
```

3.4 컨트롤러에서 전송하는 객체 배열을 화면에 출력하는 방법

Controller에서 전달하는 객체 배열을 받아 화면에 표시하는 로직을 구현해보자.

JSP : <script>

```
$("#test3").on("click", function(){
    // 컨트롤러로 부터 리스트를 받아서 출력한다
    $.ajax({
        url : "test3.do",
        type : "post",
        dataType : "json",
        success : function(data){
          // 전달받은 data를 JSON 문자열 형태로 바꾼다
          var jsonStr = JSON.stringify(data);
           // 바꾼 문자열을 json 객체로 변환한다
           var json = JSON.parse(jsonStr);

           var values = $("#d3").html();

           for(var i in json.list){
               values += json.list[i].userId + ", " + json.list[i].userPwd
                  + ", " + decodeURIComponent(json.list[i].userName)
                  + ", " + json.list[i].age
                  + ", " + json.list[i].email + "<br>";
           }

           // values에 담은 값을 d3이라는 id의 div에 출력한다.
           $("#d3").html(values);
        },
        error : function(request, status, errorData){
           alert("error code : " + request.status + "₩n"
                 + "message : " + request.responseText + "₩n"
                 + "error : " + errorData);
        }
    });
```

```
});
```

Controller

```
@RequestMapping(value="test3.do", method=RequestMethod.POST)
  public void test3Method(HttpServletResponse response) throws IOException{
    logger.info("test3Method() run...");
    //List를 json 배열로 만들어서, 뷰로 리턴 처리한다.
    ArrayList<User> list = new ArrayList<User>();
    list.add(new User("u1234", "p1234", "홍길동", 25, "h1234@kh.org"));
    list.add(new User("u2345", "p2345", "박문수", 33, "p2345@kh.org"));
    list.add(new User("u3456", "p3456", "장영실", 45, "j3456@kh.org"));

    // 전송용 최종 json 객체
    JSONObject sendJson = new JSONObject();
    // JSONArray 객체를 생성하여 JSONObject 객체를 하나씩 담는다
    JSONArray jarr = new JSONArray();

    //list 를 jarr 에 저장 처리
    for(User user : list){
      //user 정보 저장할 json 객체 선언
      JSONObject juser = new JSONObject();
      juser.put("userId", user.getUserId());
      juser.put("userPwd", user.getUserPwd());
      juser.put("userName", URLEncoder.encode(user.getUserName(), "utf-8"));
      juser.put("age", user.getAge());
      juser.put("email", user.getEmail());
      jarr.add(juser);
    }
    // 전송할 객체 배열을 JSONObject에 담아 한 번에 전달한다
    sendJson.put("list", jarr);
    response.setContentType("application/json; charset=utf-8");
    PrintWriter out = response.getWriter();
    out.println(sendJson.toJSONString());
    out.flush();
```

```
        out.close();
}
```

3.5 컨트롤러에서 맵(Map) 객체를 리턴받아서 출력하는 방법

구현에 앞서, DIspatcherServlet 설정 XML에 Map 객체를 담아 줄 jsonView라는 bean 객체를 선언해주어야 한다.

WEB-INF/spring/appServlet/servlet-context.xml

```
<!-- JsonView를 사용하기 위한 bean 객체 등록 -->
<beans:bean id="jsonView" class="net.sf.json.spring.web.servlet.view.JsonView">
</beans:bean>

<!-- jsonView 인식을 위한 viewResolver 등록 -->
<beans:bean id="viewResolver"
class="org.springframework.web.servlet.view.BeanNameViewResolver">
    <beans:property name="order" value="0"></beans:property>
</beans:bean>
```

설정이 완료되었다면, 이후 ajax 사용 시 setViewName("JSONVIEW_ID") 메소드를 사용하여, 원하는 값을 ModelAndView 객체에 담아 전송할 수 있다.

선언한 jsonView를 사용해 Controller에서 Map 객체를 전송하여 화면에 출력해보자.

JSP : <script>

```
$("#test4").on("click", function(){
    // 컨트롤러에서 맵 객체를 jsonView를 사용해 json 객체로 리턴받아서 출력처리
    $.ajax({
      url : "test4.do",
      type : "post",
      dataType : "json",
      success : function(data){
        $("#d4").html("받은 맵 안의 samp 객체 정보 확인<br>"
          + "이름 : " +  decodeURIComponent(data.samp.name)
          + ", 나이 : " + data.samp.age);
      },
```

```
        error : function(request, status, errorData){
          alert("error code : " + request.status + "₩n"
              + "message : " + request.responseText + "₩n"
              + "error : " + errorData);
        }
    });
});
```

Controller

```
@RequestMapping(value="test4.do", method=RequestMethod.POST)
  public ModelAndView test4Method(ModelAndView mv) throws
    UnsupportedEncodingException{
      logger.info("test4Method() run...");
      //map 객체를 ModelAndView 에 담아서 리턴한다
      System.out.println("samp : " + samp);
      samp.setName(URLEncoder.encode(samp.getName(), "utf-8"));

      Map<String, Sample> map = new HashMap<String, Sample>();
      map.put("samp", samp);

      mv.addAllObjects(map);
      //뷰지정 : jsonView를 빈으로 등록하고, goekd 객체의 id를 뷰이름으로 지정
해야 함
      mv.setViewName("jsonView");

      return mv;      //ajax 는 json 객체로 받음
  }
```

3.6 JSON 객체를 뷰에서 Controller로 보내는 방법

Client 화면에서 json 객체를 생성하여 Controller로 전달하는 로직은 다음과 같다.

JSP : <script>

```
$("#test5").on("click", function(){
    // 자바스크립트에서 json 객체를 생성해 서버 컨트롤러로 전송한다
    var job = new Object();
    job.name = "강감찬";
    job.age = 33;

    $.ajax({
      url : "test5.do",
      data : JSON.stringify(job),
      type : "post",
      contentType : "application/json; charset=utf-8",
      success : function(result){
        alert("전송 성공!");
        $("#d5").html("전송한 json 객체의 값 : " + job.name + ", " + job.age);
      },
      error : function(request, status, errorData){
        alert("error code : " + request.status + "₩n"
            + "message : " + request.responseText + "₩n"
            + "error : " + errorData);
      }
  });
});
```

Controller

```
@RequestMapping(value="test5.do", method=RequestMethod.POST)
  public ResponseEntity<String> test5Method(HttpServletRequest request,
@RequestBody String param) throws Exception {
    logger.info("test5Method() run...");
    request.setCharacterEncoding("utf-8");

    //전송온 문자열을 json 객체로 변환 처리
```

```
    JSONParser parser = new JSONParser();
    JSONObject job = (JSONObject)parser.parse(param);

    String name = (String)job.get("name");
    int age = ((Long)job.get("age")).intValue();

    System.out.println("name : " + name + ", age : " + age);

    //정상 완료됨을 클라이언트로 성공값을 보내야 함
    return new ResponseEntity<String>("success", HttpStatus.OK);
}
```

3.7 JSON 배열을 뷰에서 Controller로 보내는 방법

Client가 전달하는 데이터를 Controller로 전송하는 과정을 Ajax로 처리해보자.

JSP : <script>

```
$("#test6").on("click", function(){
        // 자바스크립트에서 jsonArray 객체를 만들어서, 서버 컨트롤러로 보내기
        var jArray = [{"name" : "이 이", "age" : 30 },
          {"name" : "신사임당", "age" : 47},
          {"name" : "황진이", "age" : 25}];

        $.ajax({
          url : "test6.do",
          data : JSON.stringify(jArray),
          type : "post",
          contentType : "application/json; charset=utf-8",
          success : function(result){
            alert("전송성공!");
            var values = $("#d6").html();
            for(var i in jArray){
              values += "이름 : " + jArray[i].name + ", 나이 : " + jArray[i].age + "<br>";
            }

            $("#d6").html(values);
          },
          error : function(request, status, errorData){
            alert("error code : " + request.status + "₩n"
                + "message : " + request.responseText + "₩n"
                + "error : " + errorData);
          }
      });
});
```

Controller

```
@RequestMapping(value="test6.do", method=RequestMethod.POST)
    @ResponseBody
```

```
public ResponseEntity<String> test6Method(HttpServletRequest request,
        @RequestBody String param) throws Exception {
    logger.info("test6Method() run...");
    request.setCharacterEncoding("utf-8");

    System.out.println("param : " + param);
    JSONArray jarr = (JSONArray)new JSONParser().parse(param);

    System.out.println("jarr : " + jarr.size());

    for(int i = 0; i < jarr.size(); i++){
        JSONObject job = (JSONObject)jarr.get(i);
        String name = (String)job.get("name");
        int age = ((Long)job.get("age")).intValue();

        System.out.println("name : " + name + ", age : " + age);
    }

    // 정상적으로 처리가 되었다면, 클라이언트로 성공값을 보내야 한다
    return new ResponseEntity<String>("success", HttpStatus.OK);
}
```

4 스프링 MVC 게시판 구현

기존의 기능을 활용하여 댓글 기능을 가지고 있는 게시판을 구현한다

4.1 데이터베이스 테이블 구현

4.1.1 게시글 저장 테이블

```
DROP TABLE BOARD CASCADE CONSTRAINTS;

CREATE TABLE BOARD (
   BOARD_NUM CHAR (10),
   BOARD_WRITER VARCHAR2(12) NOT NULL,
   BOARD_TITLE VARCHAR2(50) NOT NULL,
   READ_COUNT NUMBER(5 , 0) DEFAULT 0 NOT NULL,
   REGDATE DATE DEFAULT SYSDATE NOT NULL,
   BOARD_CONTENT VARCHAR2(4000) NOT NULL,
   BOARD_FILE VARCHAR2(500),
   BOARD_PWD VARCHAR2(100),
   CONSTRAINT BOARD_PK PRIMARY KEY (BOARD_NUM)
);
```

4.1.2 댓글 테이블

```
DROP TABLE BOARD_COMMENT CASCADE CONSTRAINTS;

CREATE TABLE BOARD_COMMENT (
   COMMENT_ID CHAR (10),
   BOARD_NUM CHAR (10) NOT NULL,
   COMMENT_NAME VARCHAR2(20),
   COMMENT_PWD VARCHAR2(100),
   COMMENTS VARCHAR2(4000),
   REGDATE DATE DEFAULT SYSDATE NOT NULL,
   CONSTRAINT BOARD_COMPK PRIMARY KEY (COMMENT_ID),
   CONSTRAINT FK_BNO FOREIGN KEY (BOARD_NUM) REFERENCES BOARD
    ON DELETE CASCADE
```

```
);
```

4.1.3 게시글 시퀀스

```
DROP SEQUENCE BOARD_SEQ;

CREATE SEQUENCE BOARD_SEQ START WITH 1 INCREMENT BY 1
MAXVALUE 99 NOCACHE NOCYCLE NOORDER;
```

4.1.4 댓글 시퀀스

```
DROP SEQUENCE BOARD_COMMENT_SEQ;

CREATE SEQUENCE BOARD_COMMENT_SEQ START WITH 1 INCREMENT BY 1
MAXVALUE 99 NOCACHE NOCYCLE NOORDER;
```

4.2 pom.xml 에 Library 추가

```
<repositories>
    <!-- ojdbc6 설정시 라이브러리 참조 위치 등록 -->
    <repository>
      <id>codelds</id>
      <url>https://code.lds.org/nexus/content/groups/main-repo</url>
    </repository>
 </repositories>
 <dependencies>
    <!-- Spring -->
    <dependency>
      <groupId>org.springframework</groupId>
      <artifactId>spring-context</artifactId>
      <version>${org.springframework-version}</version>
      <exclusions>
        <!-- Exclude Commons Logging in favor of SLF4j -->
        <exclusion>
          <groupId>commons-logging</groupId>
          <artifactId>commons-logging</artifactId>
        </exclusion>
      </exclusions>
    </dependency>
    <dependency>
      <groupId>org.springframework</groupId>
      <artifactId>spring-webmvc</artifactId>
      <version>${org.springframework-version}</version>
    </dependency>

    <!-- AspectJ RunTime -->
    <dependency>
      <groupId>org.aspectj</groupId>
      <artifactId>aspectjrt</artifactId>
      <version>${org.aspectj-version}</version>
```

```
    </dependency>

    <!-- AspectJ Weaver -->
    <dependency>
      <groupId>org.aspectj</groupId>
      <artifactId>aspectjweaver</artifactId>
      <version>${org.aspectj-version}</version>
    </dependency>

    <!-- Logging -->
    <dependency>
      <groupId>log4j</groupId>
      <artifactId>log4j</artifactId>
      <version>1.2.15</version>
      <scope>runtime</scope>
    </dependency>

  <!-- Servlet -->
    <dependency>
      <groupId>javax.servlet</groupId>
      <artifactId>servlet-api</artifactId>
      <version>2.5</version>
      <scope>provided</scope>
    </dependency>
    <dependency>
      <groupId>javax.servlet.jsp</groupId>
      <artifactId>jsp-api</artifactId>
      <version>2.1</version>
      <scope>provided</scope>
    </dependency>
    <dependency>
      <groupId>javax.servlet</groupId>
      <artifactId>jstl</artifactId>
```

```
    <version>1.2</version>
</dependency>

<!-- Test -->
<dependency>
    <groupId>junit</groupId>
    <artifactId>junit</artifactId>
    <version>4.7</version>
    <scope>test</scope>
</dependency>

<!-- FileUpload -->
<dependency>
    <groupId>commons-fileupload</groupId>
    <artifactId>commons-fileupload</artifactId>
    <version>1.3.3</version>
</dependency>
<dependency>
    <groupId>commons-io</groupId>
    <artifactId>commons-io</artifactId>
    <version>2.6</version>
</dependency>

<!-- Database 설정 -->
<dependency>
    <groupId>com.oracle</groupId>
    <artifactId>ojdbc6</artifactId>
    <version>11.2.0.3</version>
</dependency>

<!-- MyBatis -->
<dependency>
    <groupId>org.mybatis</groupId>
```

```
    <artifactId>mybatis</artifactId>
    <version>3.4.5</version>
  </dependency>
  <dependency>
    <groupId>org.mybatis</groupId>
    <artifactId>mybatis-spring</artifactId>
    <version>1.3.1</version>
  </dependency>
  <dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-jdbc</artifactId>
    <version>${org.springframework-version}</version>
  </dependency>

  <!-- DBCP -->
  <dependency>
    <groupId>commons-dbcp</groupId>
    <artifactId>commons-dbcp</artifactId>
    <version>1.4</version>
  </dependency>

  <!--  JSON  -->
  <dependency>
    <groupId>net.sf.json-lib</groupId>
    <artifactId>json-lib-ext-spring</artifactId>
    <version>1.0.2</version>
  </dependency>
  <dependency>
    <groupId>com.googlecode.json-simple</groupId>
    <artifactId>json-simple</artifactId>
    <version>1.1.1</version>
  </dependency>
</dependencies>
```

기본 환경 설정

4.2.1 WebApplication 환경 설정 - web.xml

WEB-INF/web.xml

```
01  <?xml version="1.0" encoding="UTF-8"?>
02  <web-app version="2.5" xmlns="http://java.sun.com/xml/ns/javaee"
03      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
04      xsi:schemaLocation="http://java.sun.com/xml/ns/javaee
05  http://java.sun.com/xml/ns/javaee/web-app_2_5.xsd">
06
07      <welcome-file-list>
08          <welcome-file>/WEB-INF/views/index.jsp</welcome-file>
09      </welcome-file-list>
10
11      <!-- 서버 연동을 위해 필요한 Root 설정 파일 등록 -->
12      <context-param>
13          <param-name>contextConfigLocation</param-name>
14          <param-value>/WEB-INF/spring/root-context.xml</param-value>
15      </context-param>
16
17      <!-- 스프링 컨테이너 등록-->
18      <listener>
19          <listener-
20  class>org.springframework.web.context.ContextLoaderListener</listener-class>
21      </listener>
22
23      <!-- DispatcherServlet 등록 -->
24      <servlet>
25          <servlet-name>appServlet</servlet-name>
26          <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-
27  class>
28          <init-param>
29              <param-name>contextConfigLocation</param-name>
```

```
30              <param-value>/WEB-INF/spring/appServlet/servlet-context.xml</param-
31  value>
32          </init-param>
33          <load-on-startup>1</load-on-startup>
34      </servlet>
35
36      <servlet-mapping>
37          <servlet-name>appServlet</servlet-name>
38          <url-pattern>*.do</url-pattern>
39      </servlet-mapping>
40
41      <!-- UFT-8 인코딩 -->
42      <filter>
43          <filter-name>encodingFilter</filter-name>
44          <filter-class>org.springframework.web.filter.CharacterEncodingFilter</filter-class>
45          <init-param>
46              <param-name>encoding</param-name>
47              <param-value>UTF-8</param-value>
48          </init-param>
49      </filter>
50
51      <filter-mapping>
52          <filter-name>encodingFilter</filter-name>
            <url-pattern>/*</url-pattern>
      </filter-mapping>
  </web-app>
```

4.2.2 스프링 설정파일 - root-context.xml

WEB-INF/spring/root-context.xml : aop, tx

```
01 <?xml version="1.0" encoding="UTF-8"?>
02 <beans xmlns="http://www.springframework.org/schema/beans"
03     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
04     xmlns:aop="http://www.springframework.org/schema/aop"
05     xmlns:tx="http://www.springframework.org/schema/tx"
06     xsi:schemaLocation="http://www.springframework.org/schema/beans
07 http://www.springframework.org/schema/beans/spring-beans.xsd
08         http://www.springframework.org/schema/aop
09 http://www.springframework.org/schema/aop/spring-aop-4.3.xsd
10         http://www.springframework.org/schema/tx
11 http://www.springframework.org/schema/tx/spring-tx-4.3.xsd">
12
13     <!-- 서버 기반 설정이 될 Root Context 등록 -->
14     <bean id="dataSource" class="org.apache.commons.dbcp.BasicDataSource"
15         destroy-method="close">
16         <!-- destroy-method 옵션으로 사용한 SqlSession을 자동으로 Close 시킨다 -
17 ->
18         <property name="driverClassName" value="oracle.jdbc.driver.OracleDriver" />
19         <property name="url" value="jdbc:oracle:thin:@127.0.0.1:1521:xe" />
20         <property name="username" value="student" />
21         <property name="password" value="student" />
22     </bean>
23
24     <bean id="transactionManager"
25         class="org.springframework.jdbc.datasource.DataSourceTransactionManager">
26         <property name="dataSource" ref="dataSource" />
27     </bean>
28
29     <bean id="sqlSession" class="org.mybatis.spring.SqlSessionFactoryBean">
30         <property name="configLocation" value="classpath:mybatis-config.xml" />
31         <property name="dataSource" ref="dataSource" />
```

```
32      </bean>
33
34      <bean class="org.mybatis.spring.SqlSessionTemplate">
35          <constructor-arg ref="sqlSession" />
36      </bean>
37
38      <!-- Transaction 시점 등록 -->
39      <tx:advice id="txAdvice" transaction-manager="transactionManager">
40          <tx:attributes>
41              <tx:method name="select*" read-only="true" rollback-for="Exception" />
42              <tx:method name="insert*" rollback-for="Exception" />
43              <tx:method name="update*" rollback-for="Exception" />
44              <tx:method name="delete*" rollback-for="Exception" />
45          </tx:attributes>
46      </tx:advice>
47      <tx:annotation-driven />
48
49      <!-- Transaction 관련된 AOP 설정 -->
50      <aop:config proxy-target-class="true">
51          <aop:pointcut id="daoMethod" expression="execution(*
52  com.kh.testMySpring..*Impl.*(..)))" />
53          <aop:advisor id="transactionAdvisor" pointcut-ref="daoMethod"
54              advice-ref="txAdvice" />
        </aop:config>
    </beans>
```

4.2.3 스프링 설정파일 - servlet-context.xml

WEB-INF/spring/appServlet/servlet-context.xml : bean, context, aop

```
01  <?xml version="1.0" encoding="UTF-8"?>
02  <beans:beans xmlns="http://www.springframework.org/schema/mvc"
03      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
04      xmlns:beans="http://www.springframework.org/schema/beans"
05      xmlns:context="http://www.springframework.org/schema/context"
06      xmlns:aop="http://www.springframework.org/schema/aop"
07      xsi:schemaLocation="http://www.springframework.org/schema/mvc
08  http://www.springframework.org/schema/mvc/spring-mvc.xsd
09          http://www.springframework.org/schema/beans
10  http://www.springframework.org/schema/beans/spring-beans.xsd
11          http://www.springframework.org/schema/context
12  http://www.springframework.org/schema/context/spring-context.xsd
13          http://www.springframework.org/schema/aop
14  http://www.springframework.org/schema/aop/spring-aop-4.3.xsd">
15
16      <!-- DispatcherServlet 설정 등록 -->
17
18      <!-- Spring MVC @Controller 어노테이션 자동 등록 -->
19      <annotation-driven />
20
21     <!-- Spring MVC AOP 어노테이션 자동 등록 -->
22      <aop:aspectj-autoproxy/>
23
24      <!-- Multipart 파일 업로드 관련 bean 등록 -->
25      <beans:bean id="multipartResolver"
26        class="org.springframework.web.multipart.commons.CommonsMultipartResolv
27  er"/>
28
29      <!-- /resources/** 폴더 절대 경로 등록 -->
30      <resources mapping="/resources/**" location="/resources/" />
31
```

```
32      <!-- @Controllers 와 *.jsp 파일을 잇기 위한 ViewResolver 등록 -->
33      <beans:bean
34  class="org.springframework.web.servlet.view.InternalResourceViewResolver">
35          <beans:property name="prefix" value="/WEB-INF/views/" />
36          <beans:property name="suffix" value=".jsp" />
37      </beans:bean>
38
39      <context:component-scan base-package="com.kh.testMySpring" />


        </beans:beans>
```

4.2.4 MyBatis 환경 설정 - mybatis-config.xml

src/main/resources/mybatis-config.xml

```
01 <?xml version="1.0" encoding="utf-8"?>
02 <!DOCTYPE configuration PUBLIC "-//mybatis.org//DTD Config 3.0//EN"
03 "http://mybatis.org/dtd/mybatis-3-config.dtd">
04 <configuration>
05     <settings>      <!-- 입력 값이 Null일 경우 ""이 아닌 Null로 처리 -->
06         <setting name="jdbcTypeForNull" value="NULL"/>
07     </settings>
08
09     <typeAliases>   <!-- 사용하고자 하는 클래스의 별칭을 선언 -->
10         <typeAlias type="com.kh.testMySpring.board.model.vo.Board" alias="Board" />
11         <typeAlias                type="com.kh.testMySpring.board.model.vo.BoardReply"
12 alias="BoardReply" />
13     </typeAliases>
14
15     <mappers>      <!-- 사용하고자 하는 쿼리를 정의한 mapper.xml을 등록 -->
16         <mapper resource="mappers/board-mapper.xml" />
17         <mapper resource="mappers/boardReply-mapper.xml" />
18     </mappers>
   </configuration>
```

4.2.5 쿼리 mapper 설정 - board-mapper.xml

src/main/resources/mappers/board-mapper.xml

```
01 <?xml version="1.0" encoding="utf-8"?>
02 <!DOCTYPE mapper PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
03 "http://mybatis.org/dtd/mybatis-3-mapper.dtd">
04
05 <mapper namespace="Board">
06 <!-- config에서 선언한 별칭을 사용한 namespace를 통해 해당 mapper를 접근한
07 다 -->
08
09 <resultMap type="Board" id="resultBoard">
10 </resultMap>
11
12    <!-- 게시글 전체 개수 조회 -->
13    <select id="listCount" resultType="int">
14        SELECT count(*) FROM BOARD
15    </select>
16
17    <!-- 페이지당 게시글 조회 -->
18    <select id="selectList" resultType="arraylist" resultMap="resultBoard">
19        SELECT * FROM BOARD
20    </select>
21
22    <!-- 게시글 단일 조회 -->
23    <select id="selectOne" parameterType="string" resultType="Board">
24        SELECT * FROM BOARD
25        WHERE BOARD_NUM = #{board_num}
26    </select>
27
28    <!-- 검색한 게시글 조회 -->
29    <select id="searchList" parameterType="string" resultType="arraylist"
30 resultMap="resultBoard">
31        SELECT * FROM BOARD
```

```
32          WHERE BOARD_TITLE LIKE '%'|| #{keyword} ||'%'
33              OR BOARD_CONTENT LIKE '%'|| #{keyword} ||'%'
34          ORDER BY REGDATE DESC
35      </select>
36
37    <!-- 게시글 삽입 -->
38      <insert id="insertBoard" parameterType="string" flushCache="true"
39      statementType="PREPARED">
40          INSERT INTO BOARD VALUES(
41            'BO'||TO_CHAR(SYSDATE,'RRMMDD')||LPAD(BOARD_SEQ.NEXTVAL,2,'0'),
42            #{board_writer}, #{board_title}, 0,
43            SYSDATE, #{board_content}, #{board_file},
44           #{board_pwd})
45      </insert>
46
47    <!-- 게시글 조회 수 증가 -->
48      <update id="addReadCount" parameterType="string" flushCache="true"
49    statementType="PREPARED">
50          UPDATE BOARD SET READ_COUNT = READ_COUNT+1
51          WHERE BOARD_NUM = #{board_num}
52      </update>
53
54    <!-- 게시글 수정 -->
55      <update id="updateBoard" parameterType="Board" flushCache="true"
56    statementType="PREPARED">
57          UPDATE BOARD SET BOARD_TITLE = #{board_title},
58          BOARD_CONTENT = #{board_content}, BOARD_FILE = #{board_file}
59          WHERE BOARD_NUM = #{board_num} AND BOARD_PWD = #{board_pwd}
60      </update>
61
62      <!-- 게시글 삭제 -->
63      <delete id="deleteBoard" parameterType="string" flushCache="true"
64    statementType="PREPARED">
```

```
65          DELETE FROM BOARD WHERE BOARD_NUM = #{board_num}
        </delete>
    </mapper>
```

4.2.6 쿼리 mapper 설정 - boardReply-mapper.xml

src/main/resources/mappers/boardReply-mapper.xml

```
01  <?xml version="1.0" encoding="utf-8"?>
02  <!DOCTYPE mapper PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
03  "http://mybatis.org/dtd/mybatis-3-mapper.dtd">
04  <mapper namespace="BoardReply">
05      <!-- 원하는 포맷에 맞춰 출력하기 위해 resultMap 등록 -->
06      <resultMap id="resultBoardReply" type="BoardReply">
07       <!--  property는 필드명, column 은 DB 컬럼 명을 입력한다 -->
08          <id property="comment_id" column="COMMENT_ID"/>
09          <result property="board_num" column="BOARD_NUM"/>
10          <result property="comment_name" column="COMMENT_NAME"/>
11          <result property="comment_pwd" column="COMMENT_PWD"/>
12          <result property="comments" column="COMMENTS"/>
13          <result property="regdate" column="REGDATE"/>
14      </resultMap>
15
16      <!-- 해당 게시글에 대한 댓글 리스트 조회 -->
17      <select id="selectBoardReplyAll" parameterType="string" resultType="arraylist"
18   resultMap="resultBoardReply">
19          SELECT * FROM BOARD_COMMENT WHERE BOARD_NUM = #{board_num}
20      </select>
21
22      <!-- 댓글 단일 조회 -->
23      <select id="selectBoardReply" parameterType="string" resultType="BoardReply"
24   resultMap="resultBoardReply">
25          SELECT * FROM BOARD_COMMENT WHERE COMMENT_ID = #{comment_id}
26      </select>
27
```

```
28      <!-- 댓글 입력 -->
29      <insert id="insertBoardReply" parameterType="BoardReply" flushCache="true"
30   statementType="PREPARED" timeout="20">
31          INSERT INTO BOARD_COMMENT VALUES(
32
33   'BR'||TO_CHAR(SYSDATE,'RRMMDD')||LPAD(BOARD_COMMENT_SEQ.NEXTVAL,2,'0'),
34              #{board_num}, #{comment_name}, #{comment_pwd}, #{comments},
35   SYSDATE)
36      </insert>
37
38       <!-- 댓글 수정 -->
39      <update id="updateBoardReply" parameterType="BoardReply" flushCache="true"
40   statementType="PREPARED" timeout="20">
41          UPDATE BOARD_COMMENT SET COMMENTS = #{comments}
42          WHERE COMMENT_ID = #{comment_id}
43          AND COMMENT_PWD = #{comment_pwd}
44      </update>
45
46       <!-- 댓글 삭제 -->
47      <delete id="deleteBoardReply" parameterType="BoardReply" flushCache="true"
48   statementType="PREPARED" timeout="20">
49          DELETE FROM BOARD_COMMENT
50          WHERE COMMENT_ID = #{comment_id}
51          AND COMMENT_PWD = #{comment_pwd}
      </delete>
    </mapper>
```

4.3 모델 클래스 작성

4.3.1 Board VO

com.kh.testMySpring.board.model.vo.Board.java

```
01  package com.kh.testMySpring.board.model.vo;
02
03  import java.sql.Date;
04
05  import org.springframework.stereotype.Component;
06
07  @Component
08  public class Board implements java.io.Serializable{
09
10      private static final long serialVersionUID = 20001L;
11      private String board_num;
12      private String board_writer;
13      private String board_title;
14      private String board_content;
15      private String board_file;
16      private String board_pwd;
17      private int read_count;
18      private Date regDate;
19
20      public Board() {}
21
22      public Board(String board_num, String board_pwd) {
23       this.board_num = board_num;
24       this.board_pwd = board_pwd;
25      }
26
27      public Board(String board_num, String board_writer, String board_title, String
28  board_content, String board_file,
29              int read_count, Date regDate, String board_pwd) {
```

```
30      this.board_num = board_num;
31      this.board_writer = board_writer;
32      this.board_title = board_title;
33      this.board_content = board_content;
34      this.board_file = board_file;
35      this.read_count = read_count;
36      this.regDate = regDate;
37      this.board_pwd = board_pwd;
38    }
39
40    public Board(String board_writer, String board_title, String board_content, String
41 board_file,String board_pwd) {
42      this.board_writer = board_writer;
43      this.board_title = board_title;
44      this.board_content = board_content;
45      this.board_file = board_file;
46      this.board_pwd = board_pwd;
47    }
48
49    @Override   // toString() 메소드 오버라이딩
50    public String toString() {
51     return "Board [board_num=" + board_num + ", board_writer=" + board_writer + ",
52 board_title=" + board_title
53               + ", board_content=" + board_content + ", board_file=" + board_file + ",
54 read_count=" + read_count
55               + ", regDate=" + regDate + "]";
56    }
57
58 /* ***** 필드에 대한 Getter & Setter 구현  **** */
59
60 }
```

4.3.2 BoardReply VO

com.kh.testMySpring.board.model.vo.BoardReply.java

```
01  package com.kh.testMySpring.board.model.vo;
02
03  import java.sql.Date;
04
05  import org.springframework.stereotype.Component;
06
07  @Component
08  public class BoardReply implements java.io.Serializable {
09
10   private static final long serialVersionUID = 30001L;
11   private String comment_id;
12   private String board_num;
13   private String comment_name;
14   private String comment_pwd;
15   private String comments;
16   private Date regdate;
17
18   public BoardReply() {}
19
20   public BoardReply(String comment_id, String comment_pwd, String comments) {
21      super();
22      this.comment_id = comment_id;
23      this.comment_pwd = comment_pwd;
24      this.comments = comments;
25   }
26
27   public BoardReply(String comment_id, String comment_pwd) {
28      this.comment_id = comment_id;
29      this.comment_pwd = comment_pwd;
30   }
31
```

```
32   public BoardReply(String comment_id, String board_num, String comment_name,
33  String comment_pwd, String comments,
34           Date regdate) {
35      this.comment_id = comment_id;
36      this.board_num = board_num;
37      this.comment_name = comment_name;
38      this.comment_pwd = comment_pwd;
39      this.comments = comments;
40      this.regdate = regdate;
41   }
42
43   public BoardReply(String board_num, String comment_name, String comment_pwd,
44  String comments) {
45      this.board_num = board_num;
46      this.comment_name = comment_name;
47      this.comment_pwd = comment_pwd;
48      this.comments = comments;
49   }
50
51   @Override   // toString() 메소드 오버라이딩
52   public String toString() {
53      return "BoardReply [comment_id=" + comment_id + ", board_num=" + board_num
54  + ", comment_name=" + comment_name
55              + ", comment_pwd=" + comment_pwd + ", comments=" + comments + ",
56  regdate=" + regdate + "]";
57   }
58
59  /* ***** 필드에 대한 Getter & Setter 구현   **** */
60
61  }
```

4.4 게시판 DAO 구현

4.4.1 BoardDao.java

com.kh.testMySpring.board.model.dao.BoardDao.java

```
01  package com.kh.testMySpring.board.model.dao;
02
03  import java.util.List;
04
05  import org.apache.ibatis.session.RowBounds;
06  import org.apache.ibatis.session.SqlSession;
07  import org.springframework.beans.factory.annotation.Autowired;
08  import org.springframework.stereotype.Repository;
09
10  import com.kh.testMySpring.board.model.vo.Board;
11
12  @Repository("bDao")
13  public class BoardDao {
14
15      @Autowired
16      private SqlSession sqlSession;
17
18      public int listCount() { //전체 글 수 조회
19          return sqlSession.selectOne("Board.listCount");
20      }
21
22      public Board selectOne(String board_num) {  //글 가져오기
23          return sqlSession.selectOne("Board.selectOne", board_num);
24      }
25
26      public List<Board> searchList(String keyword) {  //  게시글 검색 조회
27          return sqlSession.selectList("Board.searchList", keyword);
28      }
29
```

```
30      public List<Board> selectList(int startPage, int limit){  // 특정 페이지 단위의 게시
31  글 조회
32          int startRow = (startPage-1)*limit;
33          RowBounds row = new RowBounds(startRow, limit);
34          return sqlSession.selectList("Board.selectList", null, row);
35      }
36
37      public int insertBoard(Board b) {  // 글 입력
38          return sqlSession.insert("Board.insertBoard",b);
39      }
40
41      public int addReadCount(String board_num) { // 글 조회 수 증가
42          return sqlSession.update("Board.addReadCount", board_num);
43      }
44
45      public int updateBoard(Board b) {  // 글 수정
46          return sqlSession.update("Board.updateBoard", b);
47      }
48
49      public int deleteBoard(String board_num) {  // 글 삭제
50          return sqlSession.delete("Board.deleteBoard", board_num);
51      }
    }
```

4.4.2 BoardReplyDao.java

com.kh.testMySpring.board.model.dao.BoardReplyDao.java

```
01  package com.kh.testMySpring.board.model.dao;
02
03  import java.util.*;
04
05  import org.apache.ibatis.session.SqlSession;
06  import org.springframework.beans.factory.annotation.Autowired;
07  import org.springframework.stereotype.Repository;
```

```
08
09  import com.kh.testMySpring.board.model.vo.BoardReply;
10
11  @Repository("brDao")
12  public class BoardReplyDao {
13
14      @Autowired
15      private SqlSession sqlSession;
16
17      public List<BoardReply> selectList(String board_num) {  // 게시글에 해당하는 댓
18  글 조회
19          return sqlSession.selectList("BoardReply.selectBoardReplyAll", board_num);
20      }
21
22      public BoardReply selectOne(String comment_id) { // 단일 댓글 조회
23          return sqlSession.selectOne("BoardReply.selectBoardReply", comment_id);
24      }
25
26      public int insertBoardReply(BoardReply br) { // 댓글 입력
27          return sqlSession.insert("BoardReply.insertBoardReply", br);
28      }
29
30      public int updateBoardReply(BoardReply br) {  // 댓글 수정
31          System.out.println(br);
32          return sqlSession.update("BoardReply.updateBoardReply", br);
33      }
34
35      public int deleteBoardReply(BoardReply br) { // 댓글 삭제
36          System.out.println(br);
37          return sqlSession.delete("BoardReply.deleteBoardReply", br);
38      }
    }
```

4.5 게시판용 서비스

4.5.1 BoardService.java

com.kh.testMySpring.board.model.service.BoardService.java

```
01 package com.kh.testMySpring.board.model.service;
02
03 import java.util.List;
04
05 import com.kh.testMySpring.board.model.vo.Board;
06
07 public interface BoardService {
08     int totalCount();
09     Board selectBoard(int chk, String board_num);
10     List<Board> selectList(int startPage, int limit);
11     List<Board> selectSearch(String keyword);
12     void insertBoard(Board b);
13     Board updateBoard(Board b);
14     void deleteBoard(String board_num);
15 }
```

4.5.2 BoardServiceImpl.java

com.kh.testMySpring.board.model.service.BoardServiceImpl.java

```
01 package com.kh.testMySpring.board.model.service;
02
03 import java.util.List;
04
05 import org.springframework.beans.factory.annotation.Autowired;
06 import org.springframework.stereotype.Service;
07
08 import com.kh.testMySpring.board.model.dao.BoardDao;
09 import com.kh.testMySpring.board.model.vo.Board;
10
11 @Service("bService")
```

```
12  public class BoardServiceImpl implements BoardService {
13
14      @Autowired
15      private BoardDao bDao;
16
17      @Override
18      public int totalCount() {
19          return bDao.listCount();
20      }
21      @Override
22      public Board selectBoard(int chk, String board_num) {
23          if (chk == 0) bDao.addReadCount(board_num);
24          return bDao.selectOne(board_num);
25      }
26
27      @Override
28      public List<Board> selectList(int startPage, int limit) {
29          return bDao.selectList(startPage, limit);
30      }
31
32      @Override
33      public List<Board> selectSearch(String keyword) {
34          return bDao.searchList(keyword);
35      }
36
37      @Override
38      public void insertBoard(Board b) {
39          bDao.insertBoard(b);
40      }
41
42      @Override
43      public Board updateBoard(Board b) {
44          int result = bDao.updateBoard(b);
```

```
45          if (result > 0) {
46              b = bDao.selectOne(b.getBoard_num());
47          } else {
48              b = null;
49          }
50          return b;
51      }
52
53      @Override
54      public void deleteBoard(String board_num) {
55          bDao.deleteBoard(board_num);
56      }
57  }
```

4.5.3 BoardReplyService.java

com.kh.testMySpring.board.model.service.BoardReplyService.java

```
01  package com.kh.testMySpring.board.model.service;
02
03  import java.util.List;
04
05  import com.kh.testMySpring.board.model.vo.BoardReply;
06
07  public interface BoardReplyService {
08      BoardReply selectBoardReply(String comment_id);
09      List<BoardReply> selectList(String board_num);
10      int insertBoardReply(BoardReply br);
11      int updateBoardReply(BoardReply br);
12      int deleteBoardReply(BoardReply br);
13  }
```

4.5.4 BoardReplyServiceImpl.java

com.kh.testMySpring.board.model.service.BoardReplyServiceImpl.java

```
01  package com.kh.testMySpring.board.model.service;
02
03  import java.util.List;
04
05  import org.springframework.beans.factory.annotation.Autowired;
06  import org.springframework.stereotype.Service;
07
08  import com.kh.testMySpring.board.model.dao.BoardReplyDao;
09  import com.kh.testMySpring.board.model.vo.BoardReply;
10
11  @Service("brService")
12  public class BoardReplyServiceImpl implements BoardReplyService {
13      @Autowired
14      private BoardReplyDao brDao;
15
16      public BoardReply selectBoardReply(String comment_id) {
17          return brDao.selectOne(comment_id);
18      }
19
20      public List<BoardReply> selectList(String board_num){
21          return brDao.selectList(board_num);
22      }
23
24      public int insertBoardReply(BoardReply br) {
25          return brDao.insertBoardReply(br);
26      }
27
28      public int updateBoardReply(BoardReply br) {
29          return brDao.updateBoardReply(br);
30      }
31
```

```
32      public int deleteBoardReply(BoardReply br) {
33          return brDao.deleteBoardReply(br);
34      }
35  }
```

4.6 서비스 로깅 처리용 AOP

4.6.1 log4j.xml

src/main/resources/log4j.xml

```
01 <?xml version="1.0" encoding="UTF-8"?>
02 <!DOCTYPE log4j:configuration PUBLIC "-//APACHE//DTD LOG4J 1.2//EN" "log4j.dtd">
03 <log4j:configuration xmlns:log4j="http://jakarta.apache.org/log4j/">
04
05     <!-- Appenders -->
06     <appender name="console" class="org.apache.log4j.ConsoleAppender">
07         <param name="Target" value="System.out" />
08         <layout class="org.apache.log4j.PatternLayout">
09             <param name="ConversionPattern" value="%-5p: %c - %m%n" />
10         </layout>
11     </appender>
12     <appender name="fileLogger" class="org.apache.log4j.DailyRollingFileAppender">
13         <param name="file"
14          value="D:/springWorkspace/testMySpring/src/main/resources/logs/system.log"
15 />
16         <param name="Append" value="true"/>
17         <param name="DataPattern" value="'.'yyyy-MM-dd"/>
18         <layout class="org.apache.log4j.PatternLayout">
19             <param   name="ConversionPattern"   value="[%d{HH:mm:ss}][%-5p](%F:%L)
20 - %m%n" />
21         </layout>
22     </appender>
23
24     <!-- Application Loggers -->
25     <logger name="com.kh.testMySpring">
26         <level value="info" />
27     </logger>
28
29     <!-- 3rdparty Loggers -->
```

```
30      <logger name="org.springframework.core">
31          <level value="info" />
32      </logger>
33
34      <logger name="org.springframework.beans">
35          <level value="info" />
36      </logger>
37      <logger name="org.springframework.context">
38          <level value="info" />
39      </logger>
40      <logger name="org.springframework.web">
41          <level value="info" />
42      </logger>
43
44      <!-- Root Logger -->
45      <root>
46          <priority value="warn" />
47          <appender-ref ref="console" />
48          <appender-ref ref="fileLogger" />
        </root>
    </log4j:configuration>
```

4.6.2 LogPointcut.java

com.kh.testMySpring.common.servic.LogPointcut.java

```
01 package com.kh.testMySpring.common.service;
02
03 import org.aspectj.lang.annotation.Aspect;
04 import org.aspectj.lang.annotation.Pointcut;
05
06 @Aspect
07 public class LogPointcut {
08     @Pointcut("execution(* com.kh.testMySpring..*Impl.*(..))")
09     public void allPointcut() {}
10
11     @Pointcut("execution(* com.kh.testMySpring..*Impl.get*(..))")
12     public void getPointcut() {}
13 }
```

4.6.3 AroundLog.java

com.kh.testMySpring.common.service.AroundLog.java

```
01 package com.kh.testMySpring.common.service;
02
03 import org.aspectj.lang.ProceedingJoinPoint;
04 import org.aspectj.lang.annotation.Around;
05 import org.aspectj.lang.annotation.Aspect;
06 import org.slf4j.Logger;
07 import org.slf4j.LoggerFactory;
08 import org.springframework.stereotype.Service;
09 import org.springframework.util.StopWatch;
10
11 @Service
12 @Aspect
13 public class AroundLog {
14     private static final Logger logger = LoggerFactory.getLogger(AroundLog.class);
```

```
15
16      @Around("LogPointcut.allPointcut()")
17      public Object aroundLosg(ProceedingJoinPoint pp) throws Throwable{
18          logger.info("============ 사전 공통 코드 동작! ==============");
19          String methodName = pp.getSignature().getName();
20
21          StopWatch stopWatch = new StopWatch();
22          stopWatch.start();
23
24          Object obj = pp.proceed();
25
26          stopWatch.stop();
27          System.out.println(methodName + "() 메소드 수행에 걸린 시간 : " +
28              stopWatch.getTotalTimeMillis() + "(ms)초");
29          logger.info("============ 사후 공통 코드 동작! ==============");
30          return obj;
31      }
32  }
```

4.6.4 AfterThrowingLog.java

com.kh.testMySpring.common.service.AfterThrowingLog.java

```
01  package com.kh.testMySpring.common.service;
02
03  import org.aspectj.lang.JoinPoint;
04  import org.aspectj.lang.annotation.AfterThrowing;
05  import org.aspectj.lang.annotation.Aspect;
06  import org.slf4j.Logger;
07  import org.slf4j.LoggerFactory;
08  import org.springframework.stereotype.Service;
09
10  @Service
11  @Aspect
12  public class AfterThrowingLog {
13
14      private static final Logger logger =
15  LoggerFactory.getLogger(AfterThrowingLog.class);
16
17      @AfterThrowing(pointcut="LogPointcut.allPointcut()", throwing="exceptObj")
18      public void exceptionLog(JoinPoint jp, Exception exceptObj) {
19
20          String methodName = jp.getSignature().getName();
21          logger.warn(methodName + "() 메소드 수행 중 예외 발생!");
22
23          logger.warn("예외가 발생했습니다 : "+exceptObj.getMessage());
24      }
    }
```

4.7 게시판 컨트롤러 클래스

4.7.1 BoardController.java

com.kh.testMySpring.board.controller.BoardController.java

```
01  package com.kh.testMySpring.board.controller;
02
03  import java.io.File;
04
05  import javax.servlet.http.HttpServletRequest;
06
07  import org.springframework.beans.factory.annotation.Autowired;
08  import org.springframework.stereotype.Controller;
09  import org.springframework.web.bind.annotation.RequestMapping;
10  import org.springframework.web.bind.annotation.RequestMethod;
11  import org.springframework.web.bind.annotation.RequestParam;
12  import org.springframework.web.multipart.MultipartFile;
13  import org.springframework.web.servlet.ModelAndView;
14
15  import com.kh.testMySpring.board.model.service.BoardReplyService;
16  import com.kh.testMySpring.board.model.service.BoardService;
17  import com.kh.testMySpring.board.model.vo.Board;
18
19  @Controller
20  public class BoardController {
21
22     @Autowired
23     private BoardService bService;
24     @Autowired
25     private BoardReplyService brService;
26
27     public static final int LIMIT = 10;
28
29     @RequestMapping(value = "blist.do", method = RequestMethod.GET)
```

```
30    public ModelAndView boardListService(
31          @RequestParam(name = "page", defaultValue = "1") int page,
32          @RequestParam(name = "keyword", required = false) String keyword,
33    ModelAndView mv) {
34       try {
35          int currentPage = page;
36          // 한 페이지당 출력할 목록 갯수
37          int listCount = bService.totalCount();
38          int maxPage = (int) ((double) listCount / LIMIT + 0.9);
39
40          if(keyword != null && !keyword.equals(""))
41             mv.addObject("list", bService.selectSearch(keyword));
42          else mv.addObject("list", bService.selectList(currentPage, LIMIT));
43          mv.addObject("currentPage", currentPage);
44          mv.addObject("maxPage", maxPage);
45          mv.addObject("listCount", listCount);
46          mv.setViewName("board/blist");
47       } catch (Exception e) {
48          mv.addObject("msg", e.getMessage());
49          mv.setViewName("errorPage");
50       }
51       return mv;
52    }
53
54    @RequestMapping(value = "bDetail.do", method = RequestMethod.GET)
55    public ModelAndView boardDetail(@RequestParam(name = "board_num") String
56    board_num,
57          @RequestParam(name = "page", defaultValue = "1") int page, ModelAndView
58    mv) {
59       try {
60          int currentPage = page;
61          // 한 페이지당 출력할 목록 갯수
62
```

```
63              mv.addObject("board", bService.selectBoard(0, board_num));
64              mv.addObject("commentList", brService.selectList(board_num));
65              mv.addObject("currentPage", currentPage);
66
67              mv.setViewName("board/boardDetail");
68          } catch (Exception e) {
69              mv.addObject("msg", e.getMessage());
70              mv.setViewName("errorPage");
71          }
72          return mv;
73      }
74
75      @RequestMapping(value = "bRenew.do", method = RequestMethod.GET)
76      public ModelAndView boardDetail(@RequestParam(name = "board_num") String
77  board_num, ModelAndView mv) {
78          try {
79              mv.addObject("board", bService.selectBoard(1, board_num));
80              mv.setViewName("board/boardRenew");
81          } catch (Exception e) {
82              mv.addObject("msg", e.getMessage());
83              mv.setViewName("errorPage");
84          }
85          return mv;
86      }
87
88      @RequestMapping(value = "writeForm.do", method = RequestMethod.GET)
89      public String boardInsertForm(ModelAndView mv) {
90
91          return "board/writeForm";
92      }
93
94      @RequestMapping(value = "bInsert.do", method = RequestMethod.POST)
95      public ModelAndView boardInsert(Board b, @RequestParam(name = "upfile",
```

```
96  required = false) MultipartFile report,
97          HttpServletRequest request, ModelAndView mv) {
98      try {
99          if (report != null && !report.equals(""))
100             saveFile(report, request);
101         b.setBoard_file(report.getOriginalFilename());
102         bService.insertBoard(b);
103         mv.setViewName("redirect:blist.do");
104     } catch (Exception e) {
105         mv.addObject("msg", e.getMessage());
106         mv.setViewName("errorPage");
107     }
108     return mv;
109  }
110
111  @RequestMapping(value = "bUpdate.do", method = RequestMethod.POST)
112  public ModelAndView boardUpdate(Board b, @RequestParam(name = "page",
113 defaultValue = "1") int page,
114         @RequestParam("upfile") MultipartFile report, HttpServletRequest request,
115 ModelAndView mv) {
116     try {
117         if (report != null && !report.equals("")) {
118             removeFile(b.getBoard_file(),request);
119             saveFile(report, request);
120         }
121         b.setBoard_file(report.getOriginalFilename());
122         mv.addObject("board_num", bService.updateBoard(b).getBoard_num());
123         mv.addObject("currentPage", page);
124         mv.setViewName("redirect:bDetail.do");
125     } catch (Exception e) {
126         mv.addObject("msg", e.getMessage());
127         mv.setViewName("errorPage");
128     }
```

```
129         return mv;
130     }
131 
132     @RequestMapping(value = "bDelete.do", method = RequestMethod.GET)
133     public ModelAndView boardDelete(@RequestParam(name = "board_num") String
134 board_num,
135             @RequestParam(name = "page", defaultValue = "1") int page,
136             HttpServletRequest request, ModelAndView mv) {
137         try {
138             Board b = bService.selectBoard(1, board_num);
139             removeFile(b.getBoard_file(),request);
140 
141             bService.deleteBoard(board_num);
142 
143             mv.addObject("currentPage", page);
144             mv.setViewName("redirect:blist.do");
145 
146         } catch (Exception e) {
147             mv.addObject("msg", e.getMessage());
148             mv.setViewName("errorPage");
149         }
150         return mv;
151     }
152 
153     private void saveFile(MultipartFile report, HttpServletRequest request) {
154 
155         String root = request.getSession().getServletContext().getRealPath("resources");
156         String savePath = root + "₩₩uploadFiles";
157         File folder = new File(savePath);
158 
159         if (!folder.exists()) {
160             folder.mkdir(); // 폴더가 없다면 생성한다.
161         }
```

```
162
163         String filePath = null;
164
165         try {
166             // 파일 저장
167             System.out.println(report.getOriginalFilename() + "을 저장합니다.");
168             System.out.println("저장 경로 : " + savePath);
169
170             filePath = folder + "₩₩" + report.getOriginalFilename();
171
172             report.transferTo(new File(filePath)); // 파일을 저장한다
173             System.out.println("파일 명 : " + report.getOriginalFilename());
174             System.out.println("파일 경로 : " + filePath);
175             System.out.println("파일 전송이 완료되었습니다.");
176         } catch (Exception e) {
177             System.out.println("파일 전송 에러 : " + e.getMessage());
178         }
179     }
180
181     private void removeFile(String board_file, HttpServletRequest request) {
182         String root = request.getSession().getServletContext().getRealPath("resources");
183         String savePath = root + "₩₩uploadFiles";
184
185         String filePath = savePath + "₩₩" + board_file;
186
187         try {
188             // 파일 저장
189             System.out.println(board_file + "을 삭제합니다.");
190             System.out.println("기존 저장 경로 : " + savePath);
191
192             File delFile = new File(filePath);
193             delFile.delete();
194
```

```
195             System.out.println("파일 삭제가 완료되었습니다.");
196         } catch (Exception e) {
197             System.out.println("파일 삭제 에러 : " + e.getMessage());
        }
    }
}
```

4.7.2 BoardReplyController.java

com.kh.testMySpring.board.controller.BoardReplyController.java

```
01  package com.kh.testMySpring.board.controller;
02
03  import java.io.PrintWriter;
04
05  import javax.servlet.http.HttpServletResponse;
06
07  import org.json.simple.JSONObject;
08  import org.springframework.beans.factory.annotation.Autowired;
09  import org.springframework.stereotype.Controller;
10  import org.springframework.web.bind.annotation.RequestMapping;
11  import org.springframework.web.bind.annotation.RequestMethod;
12  import org.springframework.web.bind.annotation.RequestParam;
13  import org.springframework.web.servlet.ModelAndView;
14
15  import com.kh.testMySpring.board.model.service.BoardReplyService;
16  import com.kh.testMySpring.board.model.vo.BoardReply;
17
18  @Controller
19  public class BoardReplyController {
20
21      @Autowired
22      private BoardReply br;
23
24      @Autowired
25      private BoardReplyService brService;
26
27      @RequestMapping(value = "brInsert.do", method = RequestMethod.GET)
28      public ModelAndView boardReplyInsert(
29              @RequestParam(name = "board_num") String board_num,
30              @RequestParam(name = "page", defaultValue = "1") int page,
31              BoardReply br, ModelAndView mv) {
```

```
32
33          try {
34              brService.insertBoardReply(br);
35              mv.addObject("board_num", board_num);
36              mv.addObject("page", page);
37              mv.setViewName("redirect:bDetail.do");
38          } catch (Exception e) {
39              mv.addObject("msg", e.getMessage());
40              mv.setViewName("errorPage");
41          }
42          return mv;
43      }
44
45      @RequestMapping(value = "brUpdate.do", method = RequestMethod.POST)
46      public void boardReplyUpdate(HttpServletResponse response, BoardReply br) {
47          PrintWriter out = null;
48          JSONObject job = new JSONObject();
49          try {
50              job.put("ack", brService.updateBoardReply(br));
51              out = response.getWriter();
52              out.append(job.toJSONString());
53          } catch (Exception e) {
54              job.put("ack", -1);
55          } finally {
56              out.flush();
57              out.close();
58          }
59      }
60
61      @RequestMapping(value = "brDelete.do", method = RequestMethod.POST)
62      public void boardReplyDelete(HttpServletResponse response, BoardReply br) {
63
64          PrintWriter out = null;
```

```
65          JSONObject job = new JSONObject();
66          try {
67              job.put("ack", brService.deleteBoardReply(br));
68              out = response.getWriter();
69              out.append(job.toJSONString());
70          } catch (Exception e) {
71              job.put("ack", -1);
72          } finally {
73              out.flush();
74              out.close();
75          }
76      }
77  }
```

4.8 뷰 페이지

4.8.1 style.css

src/main/webapp/resources/css/style.css

```
01  @CHARSET "UTF-8";
02
03  BODY {
04     FONT-SIZE: 9pt;
05     COLOR: black;
06     LINE-HEIGHT: 160%;
07     FONT-FAMILY: 굴림, verdana, tahoma
08  }
09
10  TD {
11     FONT-SIZE: 9pt;
12     COLOR: black;
13     LINE-HEIGHT: 160%;
14     FONT-FAMILY: 굴림, verdana, tahoma
15  }
16
17  SELECT {
18     FONT-SIZE: 9pt;
19     COLOR: black;
20     LINE-HEIGHT: 160%;
21     FONT-FAMILY: 굴림, verdana, tahoma
22  }
23
24  DIV {
25     FONT-SIZE: 9pt;
26     COLOR: black;
27     LINE-HEIGHT: 160%;
28     FONT-FAMILY: 굴림, verdana, tahoma
29  }
```

```
30
31  FORM {
32     FONT-SIZE: 9pt;
33     COLOR: black;
34     LINE-HEIGHT: 160%;
35     FONT-FAMILY: 굴림, verdana, tahoma
36  }
37
38  TEXTAREA {
39     BORDER-RIGHT: l p x solid #999999;
40     BORDER-TOP: 1px solid #999999;
41     FONT-SIZE: 9pt;
42     BORDER-LEFT: l p x solid #999999;
43     COLOR: BLACK;
44     BORDER-BOTTOM: 1px solid #999999;
45     FONT-FAMILY: 굴림, verdana;
46     BACKGROUND-COLOR: white
47  }
48
49  INPUT {
50     BORDER-RIGHT: 1px solid #999999;
51     BORDER-TOP: 1px solid #999999;
52     FONT-SIZE: 9pt;
53     BORDER-LEFT: 1px solid #999999;
54     COLOR: BLACK;
55     BORDER-BOTTOM: 1px solid #999999;
56     FONT-FAMILY: 굴림, verdana;
57     HEIGHT: 19px;
58     BACKGROUND-COLOR: white
59  }
60
61  A:link {
62     text-decoration: none;
```

```
63      color: #696969
64  }
65
66  A:hover {
67      text-decoration: yes;
68      color: #66CCFF
69  }
70
71  A:visited {
72      text  ldecoration: none;
73      color: #330066
74  }
```

4.8.2 boardList.jsp

src/main/webapp/WEB-INF/views/board/blist.jsp

```
01  <%@ page language="java" contentType="text/html; charset=UTF-8"
02      pageEncoding="UTF-8"%>
03  <%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core"%>
04  <!DOCTYPE html>
05  <html>
06  <head>
07  <meta charset="UTF-8">
08  <title>게시판 목록</title>
09  <link                href="${pageContext.request.contextPath}/resources/css/style.css"
10  rel="stylesheet" type="text/css" />
11  <script                                                    type="text/javascript"
12  src="${pageContext.request.contextPath}/resources/js/jquery-3.2.1.min.js"></script>
13  <script type="text/javascript">
14  $(function(){
15      $('form[name=listForm]').on('submit',function(e){
16          if($('input[name=keyword]').val() == null || $('input[name=keyword]').val() == ""){
17              alert("검색어를 입력해 주세요");
18              e.preventDefault();
```

```
19          } else {
20              return true;
21          }
22      });
23  });
24      function showInsertForm() {
25          location.href = "writeForm.do";
26      }
27  </script>
28  </head>
29  <body bgcolor="#FFEFD5">
30      <h1>Spring MVC 서버</h1>
31      <hr>
32      <p align="center">
33      <h3>게 시 판</h3>
34      <form method="get" name="listForm" action="blist.do">
35          <input type="hidden" name="page" value="${currentPage}">
36          <input type="text" name="keyword">
37          <input type="submit" value="검색">
38      </form>
39      <table>
40          <tr>
41              <td align="right" colspan="5">
42                  <input                  type="button"                  value="전체목록"
43  onclick="window.location='blist.do'">
44                  <input                  type="button"                  value="글쓰기"
45  onclick="window.location='writeForm.do'">
46              </td>
47          </tr>
48          <tr bgcolor="#FFD1B7">
49              <td align="center" width="60">번호</td>
50              <td align="center" width="380">제목</td>
51              <td align="center" width="100">작성자</td>
```

```
52              <td align="center" width="100">작성일</td>
53              <td align="center" width="60">조회수</td>
54          </tr>
55          <!-- 글이 없을 경우 -->
56          <c:if test="${listCount eq 0}">
57              <tr>
58                <td colspan="6" align="center"><br><br>
59                    게시판에 저장된 글이 없습니다.<br><br></td>
60              </tr>
61          </c:if>
62          <c:if test="${listCount ne 0}">
63              <c:forEach var="vo" items="${list}" varStatus="status">
64                <tr>
65                  <td align="center">${status.count}</td>
66                  <td align="left">
67                    <a
68  href="bDetail.do?board_num=${vo.board_num}&page=${currentPage}">
69                         ${vo.board_title}
70                    </a>
71                  </td>
72                  <td align="center">${vo.board_writer}</td>
73                  <td align="center">${vo.regDate}</td>
74                  <td align="center">${vo.read_count}</td>
75                </tr>
76              </c:forEach>
77          </c:if>
78          <!-- 앞 페이지 번호 처리 -->
79          <tr align="center" height="20">
80              <td colspan="5"><c:if test="${currentPage <= 1}">
81          [이전] 
82      </c:if>
83                <c:if test="${currentPage > 1}">
84                  <c:url var="blistST" value="blist.do">
```

```
85                  <c:param name="page" value="${currentPage-1}" />
86                </c:url>
87                <a href="${blistST}">[이전]</a>
88      </c:if> <!-- 끝 페이지 번호 처리 -->
89         <c:set var="endPage" value="${maxPage}" /> <c:forEach
90                var="p" begin="${startPage+1}" end="${endPage}">
91                <c:if test="${p eq currentPage}">
92                 <font color="red" size="4"><b>[${p}]</b></font>
93                </c:if>
94                <c:if test="${p ne currentPage}">
95                 <c:url var="blistchk" value="blist.do">
96                    <c:param name="page" value="${p}" />
97                 </c:url>
98                 <a href="${blistchk}">${p}</a>
99                </c:if>
10              </c:forEach> <c:if test="${currentPage >= maxPage}">
0          [다음]
10      </c:if>
1               <c:if test="${currentPage < maxPage}">
10                <c:url var="blistEND" value="blist.do">
2                  <c:param name="page" value="${currentPage+1}" />
10                </c:url>
3                 <a href="${blistEND}">[다음]</a>
10              </c:if></td>
4          </tr>
10      </table>
5   </body>
    </html>
```

4.8.3 boardDetail.jsp

src/main/webapp/WEB-INF/views/board/boardDetail.jsp

```
01 <%@ page language="java" contentType="text/html; charset=UTF-8"
02     pageEncoding="UTF-8"%>
03 <%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
04 <!DOCTYPE html>
05 <html>
06 <head>
07 <meta charset="UTF-8">
08 <title>게시글 상세 보기</title>
09 <link href="${pageContext.request.contextPath}/resources/css/style.css"
10 rel="stylesheet" type="text/css" />
11 </head>
12 <body bgcolor="#FFEFD5">
13 <h1 align="center">게시글 상세보기</h1>
14 <table align="center" cellpadding="10" cellspacing="0" border="1" width="70%">
15     <tr align="center" valign="middle">
16         <th colspan="2">${board.board_num} 번글 상세보기</th>
17     </tr>
18     <tr><td height="15" width="100">제 목</td>
19         <td>${board.board_title}</td>
20     </tr>
21     <tr><td>내 용</td>
22         <td>${board.board_content}</td>
23     </tr>
24     <tr><td>첨부파일</td>
25         <td>
26         <c:if test="${empty board.board_file}">
27          첨부파일 없음
28         </c:if><c:if test="${!empty board.board_file}">
29          <a
30 href="${pageContext.request.contextPath}/resources/uploadFiles/${board.board_file}"
31 download>${board.board_file}</a>
```

```
32            </c:if>
33           </td>
34       </tr>
35       <tr align="center" valign="middle">
36           <td colspan="2">
37            <c:url var="bupview" value="bRenew.do">
38             <c:param name="board_num" value="${board.board_num}"/>
39             <c:param name="page" value="${currentPage}"/>
40            </c:url>
41            <c:url var="bdelete" value="bDelete.do">
42             <c:param name="board_num" value="${board.board_num}"/>
43            </c:url>
44               <a href="${bupview}"> [수정 페이지로 이동] </a>   
45               <a href="${bdelete}"> [글 삭제] </a>   
46            <c:url var="blist" value="blist.do">
47             <c:param name="page" value="${currentPage}"/>
48            </c:url>
49               <a href="${blist}">[목록]</a>
50           </td>
51       </tr>
52   </table>
53   <br>
54   <h4><span>댓글 (${commentList.size()})</span></h4>
55   <c:if test="${!empty commentList}">
56   <c:forEach var="rep" items="${commentList}">
57   <div id="comment">
58   <hr>
59   <input type="hidden" id="rep_id" name="rep_id" value="${rep.comment_id}">
60   <input type="hidden" id="rep_pwd" name="rep_pwd"
61   value="${rep.comment_pwd}">
62   <h4 class="comment-head">작성자 : ${rep.comment_name}    작성일 :
63   ${rep.regdate}</h4>
64   <div class="comment-body"><p>${rep.comments}</p></div>
```

```
65  <div class="comment-confirm" style="display: none;">비밀번호 확인 : <input
66  type="password" name="pwd_chk"></div>
67  <p align="right">
68  <button type="button" class="updateConfirm" name="updateConfirm"
69  id="updateConfirm" style="display: none;">수정완료</button>
70     <button type="button" class="delete" name="delete"
71  id="delete" style="display: none;">삭제하기</button>
72     <button type="button" class="update" name="update"
73  id="update">수정 및 삭제</button></p>
74  </div>
75  <br><br>
76  </c:forEach>
77  </c:if>
78  <hr>
79  <div class="comment-box">
80   <form action="brInsert.do" id="replyForm" method="get">
81       <input type="hidden" id="board_num" name="board_num"
82  value="${board.board_num}">
83       <input type="hidden" id="page" name="page" value="${currentPage}">
84       <input type="hidden" id="comments" name="comments" value="">
85     <p align="center">
86      작성자 : <input type="text" name="comment_name" size="23">   
87  비밀번호 : <input type="password" name="comment_pwd" size="23"><br><br>
88          <textarea id="reply_contents" class="form-control" rows="6" cols="70%"
89  onfocus="if(this.value == 'Message') { this.value = ''; }" onblur="if(this.value == '')
90  { this.value = 'Message'; }" placeholder="Message"></textarea>
91          <br><br>
92          <input type="submit" value="댓글쓰기">
93     </p>
94     </form>
95  </div>
96  </body>
97  <script
```

```
98  src="https://ajax.googleapis.com/ajax/libs/jquery/3.2.1/jquery.min.js"></script>
99  <script type="text/javascript">
10  $(function(){
0    // 댓글 Insert Script
10   $('#replyForm').on('submit',function(event){
1        if($('#reply_contents').val() == ""){
10           alert("내용을 입력해주세요.");
2            event.preventDefault();
10       } else {
3            $('#comments').val($('#reply_contents').val());
10           return true;
4        }
10   });
5
10   //기존 댓글 수정 & 삭제
6    $(".update").on('click',function(){
10       var parentP = $(this).parent();
7            var parentDiv = parentP.parent();
10           var commBody = parentDiv.children('.comment-body');
8        var content = commBody.children('p').text().trim();
10
9        if($(this).text() == "수정 및 삭제"){
11           commBody.append('<textarea style="margin-top:7px;" rows="4" cols="70%"
0   class="updateContent"                                    name="updateContent"
11  id="updateContent">'+content+'</textarea>');
1            parentDiv.children('.comment-confirm').show();
11           parentP.children(".delete").toggle("fast");
2            parentP.children(".updateConfirm").toggle("fast");
11           $(this).text("수정취소");
3        } else {
11           commBody.children(".updateContent").remove();
4            parentDiv.children('.comment-confirm').hide();
11           $(this).text("수정 및 삭제");
```

```
5            parentP.children(".delete").toggle("fast");
11           parentP.children(".updateConfirm").toggle("fast");
6         }
11     });
7
11        $(".updateConfirm").on('click',function(){
8          var parentP = $(this).parent();
11            var parentDiv = parentP.parent();
9             if(parentDiv.find('input[name=pwd_chk]').val()                    !=
12 parentDiv.children('input[name=rep_pwd]').val()){
0              alert("비밀번호가 일치하지 않습니다.");
12             return false;
1             } else {
12             $.ajax({
2                     url : "${pageContext.request.contextPath}/brUpdate.do",
12                    method : "POST",
3                     async : false,
12                    data: {
4                      comment_id : parentDiv.find("input[name=rep_id]").val(),
12                     comment_pwd : parentDiv.find('input[name=pwd_chk]').val(),
5                      comments : parentDiv.find('.updateContent').val()
12                    },
6                     success : function(data) {
12                     alert(data);
7                      parentDiv.find(".comment-body
12 p").text(parentDiv.find('.updateContent').val());
8                     }, error : function(request,status,error) {
12
9  alert("code:"+request.status+"₩n"+"message:"+request.responseText+"₩n"+"error:"+
13 error);
0                     }
13                 });
1             }
```

```
13
2           parentDiv.find(".updateContent").remove();
13          parentDiv.find(".comment-confirm").val("");
3           parentDiv.find(".comment-confirm").hide();
13          parentP.children(".updateConfirm").toggle("fast");
4           parentP.children(".delete").toggle("fast");
13          parentP.children('.update').text("수정 및 삭제");
5    });
13
6    $(".delete").on('click',function(){
13       var parentP = $(this).parent();
7           var parentDiv = parentP.parent();
13
8           if(parentDiv.find('input[name=pwd_chk]').val()                    !=
13   parentDiv.children('input[name=rep_pwd]').val()){
9            alert("비밀번호가 일치하지 않습니다.");
14           return false;
0           } else {
14           $.ajax({
1                   url : "${pageContext.request.contextPath}/brDelete.do",
14                  method : "POST",
2                   data: {
14                   comment_id : parentDiv.find("input[name=rep_id]").val(),
3                    comment_pwd : parentDiv.find('input[name=pwd_chk]').val()
14                  },
4                   success : function(data) {
14                      alert(data);
5                       parentDiv.remove();
14                  }, error : function(request,status,error) {
6
14   alert("code:"+request.status+"₩n"+"message:"+request.responseText+"₩n"+"error:"+
7    error);
14                  }
```

```
8                 });
14              }
9           });
15  });
0   </script>
    </html>
```

4.8.4 writeForm.jsp

src/main/webapp/WEB-INF/views/board/writeForm.jsp

```
01 <%@ page language="java" contentType="text/html; charset=UTF-8"
02     pageEncoding="UTF-8"%>
03 <%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core"%>
04 <!DOCTYPE html>
05 <html>
06 <head>
07 <meta charset="UTF-8">
08 <title>게시글 등록</title>
09 <link href="${pageContext.request.contextPath}/resources/css/style.css"
10 rel="stylesheet" type="text/css" />
11 </head>
12 <body bgcolor="#FFEFD5">
13 <form action="blnsert.do" method="post" enctype="multipart/form-data">
14 <table align="center">
15     <tr><td>제목</td><td><input type="text" name="board_title"></td></tr>
16     <tr><td>작성자</td><td><input type="text" name="board_writer"></td></tr>
17     <tr><td>첨부파일</td><td><input type="file" name="upfile"></td></tr>
18     <tr><td>글 비밀번호</td><td><input type="password"
19 name="board_pwd"></td></tr>
20     <tr><td>내용</td><td><textarea cols="50" rows="7"
21 name="board_content"></textarea></td></tr>
22     <tr><td colspan="2" align="center"><input type="submit" value="등록하기">
23  
24     <c:url var="blist" value="blist.do">
25         <c:param name="page" value="1"/>
26     </c:url>
27         <a href="${blist}">목록으로</a></td></tr>
28 </table>
29 </form>
   </body>
   </html>
```

4.8.5 boardRenew.jsp

src/main/webapp/WEB-INF/views/board/boardRenew.jsp

```
01 <%@ page language="java" contentType="text/html; charset=UTF-8"
02     pageEncoding="UTF-8"%>
03 <%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
04 <!DOCTYPE html>
05 <html>
06 <head>
07 <meta charset="UTF-8">
08 <title>게시글 수정하기</title>
09 </head>
10 <link href="${pageContext.request.contextPath}/resources/css/style.css"
11 rel="stylesheet" type="text/css" />
12 <body bgcolor="#FFEFD5">
13 <form name="renewForm" action="bUpdate.do" method="post"
14 enctype="multipart/form-data">
15 <input type="hidden" name="board_num" value="${board.board_num}">
16 <input type="hidden" name="board_file" value="${board.board_file}">
17 <table align="center">
18    <tr><td>제목</td><td><input type="text" name="board_title"
19 value="${board.board_title}"></td></tr>
20    <tr><td>작성자</td><td>${board.board_writer}</td></tr>
21    <tr><td>글 비밀번호</td><td><input type="password"
22 name="board_pwd"></td></tr>
23    <tr><td>이전 첨부파일</td>
24        <td>
25        <c:if test="${empty board.board_file}">
26           첨부파일 없음
27        </c:if><c:if test="${!empty board.board_file}">
28           <a
29 href="${pageContext.request.contextPath}/resources/uploadFiles/${board.board_file}"
30 download>${board.board_file}</a>
31           </c:if>
```

```
32              </td>
33          </tr>
34          <tr><td>변경할 첨부파일</td>
35              <td>
36              <input type="file" name="upfile">
37              </td>
38          </tr>
39          <tr><td>내용</td><td><textarea cols="50" rows="7"
40              name="board_content">${board.board_content}</textarea></td></tr>
41          <tr><td colspan="2" align="center"><input type="submit" id="renew" value="수
42      정하기">  
43          <c:url var="blist" value="blist.do">
44              <c:param name="page" value="1"/>
45          </c:url>
46              <a href="${blist}">목록으로</a></td></tr>
47      </table>
48      </form>
49      </body>
50      <script                 src="${pageContext.request.contextPath}/resources/js/jquery-
51      3.2.1.min.js"></script>
52      <script type="text/javascript">
53      $(function(){
54          $('form[name=renewForm]').on('submit',function(event){
55              if($('input[name=board_pwd]').val() != "${board.board_pwd}"){
56                  alert("비밀번호가 일치하지 않습니다.");
57                  event.preventDefault();
58              } else {
59                  return true;
60              }
61          });
        });
        </script>
        </html>
```

Part 3. 인터페이스 구현 검증하기

Chapter 8. SqlSessionTemplate

1 IoC, Inversion of Control - 제어의 역행

IoC란 프로그램의 제어 흐름 구조가 바뀌는 것을 뜻한다.

일반적인 프로그램은 main( ) 같은 프로그램이 시작되는 지점에서 사용할 객체를 생성하고, 만들어진 객체를 통해 메소드를 호출하는 작업을 통해 프로그램이 수행된다. 객체는 프로그램 흐름을 결정하거나 사용할 객체를 구성하는 작업을 수행하며 프로그램의 흐름에 직접적으로 관여한다. 이러한 방식은 프로그램의 모든 작업을 사용하는 쪽에서 제어하는 구조이다.

이와 달리 IoC는 제어 흐름의 개념을 거꾸로 뒤집은 것이다. 객체는 자신이 어떻게 만들어지고 어디서 사용되는지 알 수 없으며 사용할 객체를 생성하거나 선택하지 않는다. 객체의 제어 권한을 객체 자신이 아닌 다른 대상에게 위임하는 것이다. 객체의 생성부터 생명주기의 관리까지 모든 객체에 대한 제어 권한을 다른 대상에게 위임한 것이 제어권의 역행(IoC)이라는 개념이다.

- 작업을 수행하는 쪽에서 객체를 생성하는 기존의 제어 흐름 개념을 뒤집은 것이다
- 사용할 객체를 직접 생성하거나 선택하지 않는다
- 모든 객체는 제어 권한을 위임받는 특별한 객체에 의해서 만들어 지고 사용된다
- 객체는 자신이 어떻게 생성되고 어떻게 사용되는지 알 수 없다

Chapter 9. AOP Transaction 관리

1 DI, Dependency Injection - 의존성 주입

IoC(제어의 역행)의 핵심 기능으로써 객체의 레퍼런스를 외부로부터 제공 받고, 이를 통해 다른 객체들과 의존 관계가 다양하게 형성되는 것을 말한다. 설명을 어렵게 느낄 수도 있으나, 사실 우리는 지금까지 이러한 의존 관계 주입을 통한 DI를 사용해왔다. 바로 'new'를 통한 객체 생성이다.

이러한 객체 생성에 대한 예로써 A가 B Class의 객체 bob을 생성하여 사용한다고 하자. 만약 B에 새로운 메소드가 추가되거나 변경되는 사항이 발생한다면, A의 내용도 그에 따른 영향을 받게 될 것이다. 그럼 결과적으로 A의 기능 수행에도 영향을 미칠 수 있는데, 이러한 관계를 의존 관계라고 한다.

```
public class A {
    // 이렇게 객체를 직접 생성하여 사용하는 것을 종속 객체 의존성 주입이라고 한다.
    private B bob = new B( );
}
```

1.1 직접 객체를 생성하여 의존성 주입

new를 통해 객체를 생성함으로써 'PersonManager has Person'의 관계가 되고, PersonManager는 Person의 내용이 변경되었을 때 수정이 필요한, 의존적인 관계가 성립된다. 하지만 관련 메소드를 이용하고자 할 때에는 반드시 Person 객체를 생성해야 되므로 코드가 효율적이지 못하다.

com.kh.testSpring.dependency.PersonManager.java

```
01 package com.kh.testSpring.dependency;
02
03 import com.kh.testSpring.person.model.vo.Person;
04
05 public class PersonManager {
06     public PersonManager() {}
07     public void printPerson() {
08         Person p = new Person();
09         System.out.println(p.getName());
10     }
11 }
```

com.kh.testSpring.person.model.vo.Person.java

```
01 package com.kh.testSpring.person.model.vo;
02
03 public class Person {
04   private String name;
05
06   public Person( ) {
07       this.name = "Bob";
08   }
09   public String getName ( ) {
10     return this.name;
11   }
12   public void setName(String name){
13       this.name = name;
14   }
15 }
```

1.2 생성자를 통한 의존성 주입

Person 객체를 생성할 때 Person 객체에서 이름을 직접 생성하지 않고, name을 지정하여 담아 줄 수 있도록 함으로써 코드의 유연함이 생겼다.

com.kh.testSpring.dependency.PersonManager.java

```
01 package com.kh.testSpring.dependency;
02
03 import com.kh.testSpring.person.model.vo.Person;
04
05 public class PersonManager {
06     public PersonManager() {}
07     public void printPerson() {
08         String name = "Bob";
09         Person p = new Person(name);
10         System.out.println(p.getName());
11     }
12 }
```

com.kh.testSpring.person.model.vo.Person.java

```
01 package com.kh.testSpring.person.model.vo;
02
03 public class Person {
04   private String name;
05
06   public Person(String name) {
07       this.name = name;
08   }
09   public String getName ( ) {
10     return this.name;
11   }
12   public void setName(String name){
13       this.name = name;
14   }
15 }
```

1.3 Setter을 통한 의존성 주입

new로 생성한 이후, Setter 메소드를 호출하여 의존성 주입을 처리하는 방법이다.

com.kh.testSpring.dependency.PersonManager.java

```
01 package com.kh.testSpring.dependency;
02
03 import com.kh.testSpring.person.model.vo.Person;
04
05 public class PersonManager {
06     private Person p = new Person();
07     public PersonManager() {}
08     public void printPerson() {
09         p.setName("Bob");
10         System.out.println(p.getName());
11     }
12 }
```

com.kh.testSpring.person.model.vo.Person.java

```
01 package com.kh.testSpring.person.model.vo;
02
03 public class Person {
04   private String name;
05
06   public Person(String name) {
07       this.name = name;
08   }
09   public String getName ( ) {
10     return this.name;
11   }
12   public void setName(String name){
13       this.name = name;
14   }
15 }
```

2 스프링에서의 의존성 주입

스프링에서는 객체에 의존성을 주입하는 방법이 'new' 를 사용한 방법 외에 기본 생성자냐, 매개변수가 존재하는 생성자냐에 따라 그 종류가 나뉘는데 각각의 경우를 분류하여 설명하겠다.

2.1 XML을 통한 의존성 주입

XML에서 빈의 의존 관계 정보를 이용하여 의존성 주입을 하기 위해서는 별도의 xml 파일을 작성하여 bean 객체를 선언하고, 해당 객체를 사용하고자 하는 클래스에 GenericXmlApplication Context를 사용하여 XML 파일에 설정된 빈 정보를 가져온다. STS의 New - Other - Spring - Spring Bean Configuration File 을 선택하여 쉽게 생성이 가능하다.

com/kh/testSpring/sampleContext.xml

```
01 <?xml version="1.0" encoding="UTF-8"?>
02 <beans:beans xmlns="http://www.springframework.org/schema/mvc"
03     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
04     xmlns:beans="http://www.springframework.org/schema/beans"
05 xsi:schemaLocation="http://www.springframework.org/schema/mvc
06 http://www.springframework.org/schema/mvc/spring-mvc.xsd
07         http://www.springframework.org/schema/beans
08 http://www.springframework.org/schema/beans/spring-beans.xsd
09         http://www.springframework.org/schema/context
10 http://www.springframework.org/schema/context/spring-context.xsd
11         http://www.springframework.org/schema/aop
12 http://www.springframework.org/schema/aop/spring-aop-4.3.xsd">
13
14 <!-- Sample이라는 클래스를 samp이라는 ID로 객체 생성 -->
15
16 <!--<beans:bean id="samp" class="com.kh.testSpring.Sample">
17         생성자를 통해 의존성을 주입하려면 <constructor-arg> 엘리먼트를 사용한
18 다. -->
19         <beans:constructor-arg index="0" value="김유신" name="name" /> <!-- 첫번
```

```
20  째 인자 -->
21          <beans:constructor-arg index="1" value="35" name="age" />       <!-- 두번째
22  인자 -->
23      </beans:bean> -->
24
25
26  <beans:bean id="samp" class="com.kh.testSpring.Sample">
27          <!-- Setter를 통한 의존성을 주입하려면 property 엘리먼트를 통해 변경한다.
    -->
            <beans:property name="name" value="박문수" />
            <beans:property name="age" value="40" />
        </beans:bean>
    </beans:beans>
```

com.kh.testSpring.TestController.java

```
01  package com.kh.testSpring;
02
03  import org.slf4j.Logger;
04  import org.slf4j.LoggerFactory;
05  import org.springframework.context.support.AbstractApplicationContext;
06  import org.springframework.context.support.ClassPathXmlApplicationContext;
07  import org.springframework.context.support.GenericXmlApplicationContext;
08  import org.springframework.stereotype.Controller;
09  import org.springframework.ui.Model;
10
11  @Controller
12  public class TestController {
13
14      private static final Logger logger = LoggerFactory.getLogger(TestController.class);
15
16      //종속객체 주입(의존성 주입)
17      //private Sample samp = new Sample();
```

```
18      private Sample samp;  // 현재는 null 값을 보유하고 있다.
19
20      @RequestMapping(value = "testDI_1.do", method = RequestMethod.GET)
21      public String testDIMethod(Model model) {
22          logger.info("testDI_1 Method() run....");
23          AbstractApplicationContext cntx = new
24  GenericXmlApplicationContext("sampleContext.xml");
25      // 만약 XML 파일의 위치가 기본 클래스패스가 아니라면,
26      // ClassPathXmlApplicationContext 을 통해 참조할 수도 있다.
27      // AbstractApplicationContext cntx =
28       //         new ClassPathXmlApplicationContext("sampleContext.xml",Sample.class);
29
30    // XML에 설정되어 있는 bean을 통해 객체를 생성함으로써 의존성을 주입한다.
31          samp = cntx.getBean("samp", Sample.class);
32          System.out.println("samp : " + samp);
33          model.addAttribute("samp", samp);
34
35          return "test/testDI_1_Page";
36      }
    }
```

2.2 어노테이션을 통한 의존성 주입

스프링 2.5 이전 버전에서는 모든 빈 객체가 XML파일에 수동으로 설정되어야 했으며, 자바
에 대한 어노테이션 지원이 없었다. 때문에 XML 코딩에 손이 많이 가게 되었으며, 유지 보수에서도 문제가 발생했다. 이를 2.5 버전 이후로는 특정 어노테이션에 대한 XML 설정을 자동으로 해줌으로써, XML설정 파일을 더욱 간편화 시켜 주었다.

Spring에서 제공하는 Controller, Service, Repository, Component 어노테이션을 사용하게 되면 자동으로 bean 객체가 등록된다. 이렇게 자동 의존성 주입이 된 객체를 사용하려면 반드시 @Autowired 어노테이션을 레퍼런스 변수를 선언한 라인 바로 위에 표시해야 한다.
각각의 어노테이션의 특징을 정리하면 다음과 같다.

Annotation	설명
@Controller	spring MVC의 Controller 객체임을 명시, 스프링 설정 파일에 <context:component-scan>을 자동 등록
@Service	비즈니스 로직을 담당하는 Service 객체임을 명시 @Service("OO")에 등록된 OO과 변수명이 같아야 한다.
@Repository	데이터의 접근을 담당하는 DAO 객체임을 명시 @Repository("OO")에 등록된 OO과 변수명이 같아야 한다.
@Component	<context:component-scan>을 정의하고 적용할 기본 패키지를 base-package 속성으로 등록, @Scope를 사용하여 범위 지정이 가능하다.

3.2.1 Controller

com.kh.testSpring.sample.controller.SampleController.java

```
01  package com.kh.testSpring.sample.controller;
02
03  import org.slf4j.Logger;
04  import org.slf4j.LoggerFactory;
05  import org.springframework.beans.factory.annotation.Autowired;
06  import org.springframework.stereotype.Controller;
07  import org.springframework.ui.Model;
08  import org.springframework.web.bind.annotation.RequestMapping;
09  import org.springframework.web.bind.annotation.RequestMethod;
10
11  import com.kh.testSpring.sample.model.service.SampleService;
12  import com.kh.testSpring.sample.model.vo.Sample;
13
14  // spring MVC의 controller 역할에 의존성을 주입한다
15  // @Controller 어노테이션은 스프링 내부 설정파일에서
16  // <context:component-scan base-package="패키지 명"> 을 자동으로 추가한다
17  @Controller
18  public class SampleController {
19
20      private static final Logger logger = LoggerFactory.getLogger(SampleController.class);
21
22     // @Service 어노테이션으로 선언한 SampleService 객체를
23     // 찾아 자동으로 DI 처리한다.
24      @Autowired
25      private SampleService sampleService;
26
27     // @Component 어노테이션으로 선언한 Sample 객체를
28     // 찾아 자동으로 DI 처리한다.
29     @Autowired
30      private Sample samp;
```

```
31
32      @RequestMapping(value = "sample.sa", method = RequestMethod.GET)
33      public String selesctSample(Model model) {
34          logger.info("selectSample() run...", locale);
35          samp = sampleService.selectName("김유신");
36
37          model.addAttribute("sample", samp);
38          return "testDI";
39      }
40  }
```

3.2.2 Service

com.kh.testSpring.sample.model.service.SampleService.java

```
01  package com.kh.testSpring.sample.model.service;
02
03  import org.springframework.beans.factory.annotation.Autowired;
04  import org.springframework.stereotype.Service;
05
06  import com.kh.testSpring.sample.model.dao.SampleDao;
07  import com.kh.testSpring.sample.model.vo.Sample;
08
09  // @Service 어노테이션으로 해당 클래스가 Service객체임을 명시한다.
10  @Service("sampleService")
11  public class SampleService {
12
13     // @Repository를 통해 선언한 SampleDao 객체를 찾아 자동으로 DI 처리한다.
14      @Autowired
15      private SampleDao sampleDao;
16
17      public Sample selectName(String name) {
18          return sampleDao.selectOne(name);
19      }
20  }
```

3.2.3 Dao

com.kh.testSpring.sample.model.dao.SampleDao.java

```
01  package com.kh.testSpring.sample.model.dao;
02
03  import org.springframework.beans.factory.annotation.Autowired;
04  import org.springframework.stereotype.Repository;
05
06  import com.kh.testSpring.sample.model.vo.Sample;
07
08  // @Repository 어노테이션으로 해당 클래스가 Dao 객체임을 명시한다.
09  @Repository("sampleDao")
10  public class SampleDao {
11
12      @Autowired
13      Sample samp;
14
15      public SampleDao() {}
16
17     // DataBase에 연동하기 이전 소스를 간략하게 표현하였다.
18     // 이후 Dao코드 구현은Database연동에서 자세히 다루겠다.
19      public Sample selectOne(String name) {
20          if(name.equals("김유신")) {
21              samp.setId(100);
22              samp.setName("김유신");
23          } else {
24              samp.setId(200);
25              samp.setName("신사임당");
26          }
27          return samp;
28      }
29  }
```

2.3 @Autowired와 @Resource 어노테이션을 통한 의존성 주입

@Autowired 와 @Resource 어노테이션은 특정 객체에 대해 자동으로 의존성을 주입하지만 각각 동작하는 로직이 다르다. 그럼 두 어노테이션의 차이점을 알아보자.

Annotation	설명
@Autowired	- 스프링에서 지원하는 어노테이션 - 대상 객체를 Class 타입(type)으로 연결 - @Qualifier("객체명")으로 특정 객체를 사용하도록 설정 가능 - 반드시 필드 값을 선언할 필요가 없을 경우 (required=false) 옵션 설정 가능
@Resource	- 자바에서 지원하는 기존의 어노테이션 - 대상 객체를 레퍼런스 변수 명 (name)으로 연결 - 특정 객체와 매칭 시켜야할 경우 (name=OOO) 옵션 설정 가능

3.3.1 @Autowired 어노테이션으로 의존성을 주입한 경우

com/kh/testSpring/sampleContext.xml

```
01 <?xml version="1.0" encoding="UTF-8"?>
02 <beans:beans xmlns="http://www.springframework.org/schema/mvc"
03     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
04     xmlns:beans="http://www.springframework.org/schema/beans"
05     xmlns:context="http://www.springframework.org/schema/context"
06     xsi:schemaLocation="http://www.springframework.org/schema/mvc
07 http://www.springframework.org/schema/mvc/spring-mvc.xsd
08         http://www.springframework.org/schema/beans
09 http://www.springframework.org/schema/beans/spring-beans.xsd
10         http://www.springframework.org/schema/context
11 http://www.springframework.org/schema/context/spring-context.xsd">
12
13    <beans:bean class="com.kh.myWeb.sample.model.vo.Sample" id="sampleT">
14        <beans:property name="id" value="201"/>
15        <beans:property name="name" value="신성우"/>
16    </beans:bean>
17 </beans:beans>
```

com.kh.testSpring.sample.model.dao.SampleDao.java

```
01  package com.kh.testSpring.sample.model.dao;
02
03  import org.springframework.beans.factory.annotation.Autowired;
04  import org.springframework.beans.factory.annotation.Qualifier;
05  import org.springframework.stereotype.Repository;
06
07  import com.kh.testSpring.sample.model.vo.Sample;
08
09  // @Repository를 통한 자동 DI 설정
10  @Repository("sampleDao")
11  public class SampleDao {
12
13    // @Autowired로 의존성 자동 주입
14    // @Qualifier("sampleT") 를 통해 context에서 지정한 "sampleT"의
15    // 설정 정보를 불러와 가져와 의존성을 주입한다.
16    @Autowired
17     @Qualifier("sampleT")
18     Sample samp;
19
20     public SampleDao() {}
21
22     public Sample selectOne(String name) {
23
24         return samp;
25     }
26  }
```

3.3.2 @Resource 어노테이션으로 의존성을 주입한 경우

- com/kh/testSpring/sampleContext.xml 내용은 동일하다고 가정하자.

com.kh.testSpring.sample.model.dao.SampleDao.java

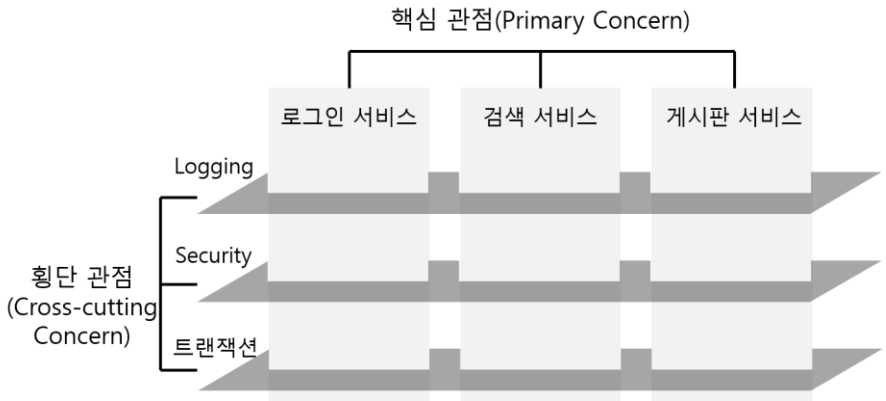
```
01  package com.kh.testSpring.sample.model.dao;
02
03  import javax.annotation.Resource;
04
05  import org.springframework.stereotype.Repository;
06
07  import com.kh.testSpring.sample.model.vo.Sample;
08
09  // @Repository를 통한 자동 DI 설정
10  @Repository("sampleDao")
11  public class SampleDao {
12
13    // @Resource로 의존성 자동 주입
14    // name 옵션을 통해 "sampleT" ID의 빈 객체를 불러와 의존성을 주입한다.
15     @Resource(name="sampleT")
16     Sample samp;
17
18      public SampleDao() {}
19
20      public Sample selectOne(String name) {
21
22          return samp;
23      }
24  }
```

3-3. Spring AOP

1 AOP, Aspect-Oriented Programming

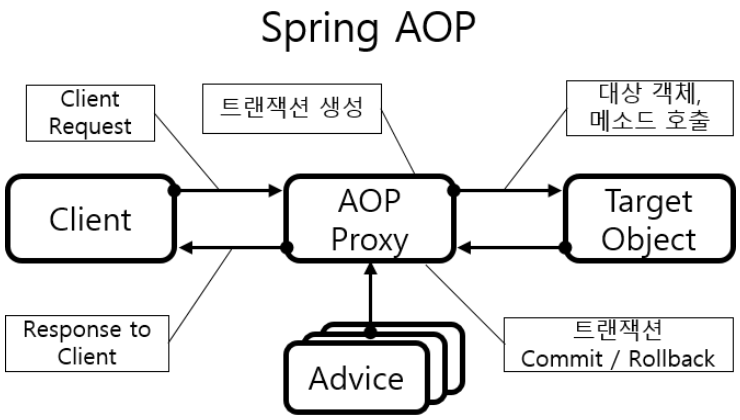
관점 지향 프로그래밍 - 로직(code) 에 대한 관점을 이해하고 적용하는 방법론이다

프로그램에서는 특정 코드가 반복적으로 나오게 된다. 즉, 로깅처리, 보안코드, 트랜잭션 처리 등 프로그램들을 횡단하는 공통적인 코드가 아래의 그림처럼 나타나게 되는데, 이를 횡단 관점(Cross-cutting Concern)이라고 한다.



프로그래머는 이러한 부분을 추출하여 클래스로 만들고 반복적으로 호출하여 사용할 수 있도록 공통 모듈로 작성하게 된다. 하지만 분리한 공통 모듈을 호출하는 코드가 반드시 프로그램 내에 존재하게 되는데, 이러한 호출 코드까지 분리하여 각 객체 별로 처리하던 것을 각 관점 별로 외부에서 접근하게 하는 것이 AOP의 목표이다.

즉, 공통 코드를 작성한 후 실행되어야 할 위치만 설정을 해주면 자동으로 코드가 끼워 넣어진 것처럼 소스 코드를 삽입되도록 하는 것이다.



이 때 공통적으로 뽑아낸 코드를 Advice라고 하며 코드를 끼워 넣을 곳을 Joinpoint, 조인포인트를 모아둔 것을 Pointcut이라고 한다. 어드바이스를 조인포인트에 실제로 끼워 넣는 작업을 Weaving이라고 한다. 스프링에서는 어드바이스와 포인트 컷을 묶어 Advisor라고 한다.

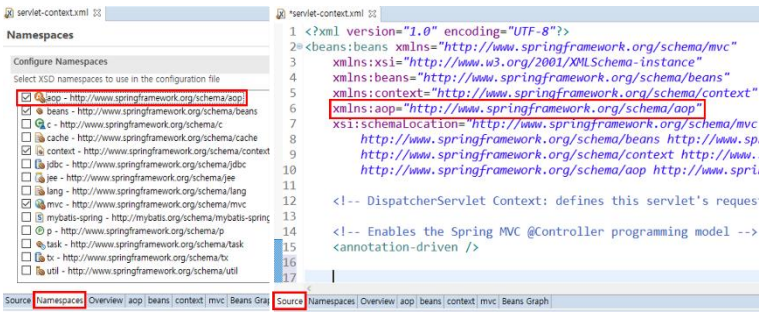
AOP 핵심 용어 정리

용어	설명	
Aspect	- 여러 객체에 공통으로 적용되는 기능을 분리하여 작성한 클래스 - 로깅, 트랜잭션, 보안 등을 대표적 예로 들 수 있다	
Joinpoint	- 클래스의 객체(인스턴스) 생성 지점, 메소드 호출 시점, 예외 발생 시점 등 특정 작업이 시작되는 시점을 말한다 - Advice 를 적용할 수 있는 시점이다	
Advice	- Joinpoint에 삽입되어 동작될 코드, 메소드를 말한다 - 동작 시점에 따라 5가지 로 나뉜다	
	Before advice	Joinpoint 앞에서 실행
	Around Advice	Joinpoint 앞과 뒤에서 실행
	After Advice	Joinpoint 호출이 리턴되기 직전에 실행
	After Returning Advice	Joinpoint 메소드 호출이 정상적으로 종료된 후에 실행
	After Throwing Advice	예외가 발생했을 때 실행
Pointcut	- 조인 포인트의 부분 집합을 의미 - 실제 Advice가 적용되는 Joinpoint를 말한다 - 정규 표현식이나 AspectJ 문법을 이용하여 범위를 정의할 수 있다	
Introduction	- 정적인 방식의 AOP 기술 - 기존의 클래스와 인터페이스에 필요한 메소드나 필드를 추가해준다	
Weaving	- 작성한 Advice (공통 코드)를 핵심 로직 코드에 삽입하는 것을 말한다 - 시점에 따라 컴파일 시, 클래스 로딩 시, 런 타임 시에 위빙하는 방식이 있다	
	컴파일시에 위빙	- AspectJ 에서 사용하는 방식으로, 컴파일 시 AOP가 적용된 클래스 파일이 새로 생성
	클래스 로딩시에 위빙	- JVM에서 로딩한 클래스의 바이트 코드를 AOP가 변경하여 사용
	런타임시에 위빙	- 스프링에서 자체적으로 지원하는 기능 - 런타임 시 소스코드나 클래스 정보 자체를 변경하지 않고, 중간에 프록시를 생성하여 경유 - 메소드 호출 Joinpoint 만 지원
Proxy	- 대상 객체에 Advice가 적용된 후 생성된 객체	
Target Object	- 핵심 로직을 구현하는 클래스 - Advice를 삽입할 대상 객체를 뜻한다	

2 AOP 작성 순서

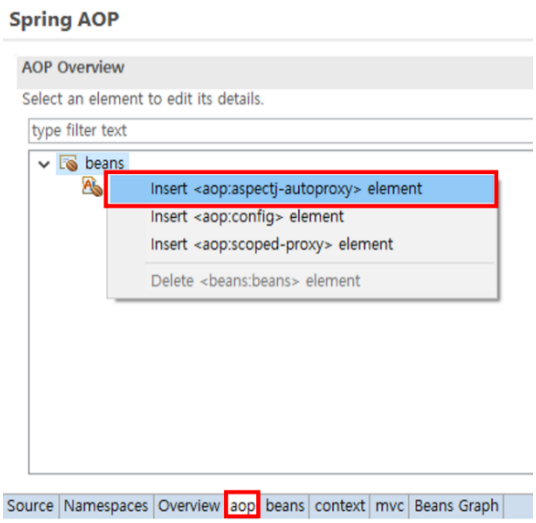
2.1 AOP 사용을 위한 환경 설정

AOP를 사용하기 위해서는 먼저 스프링 설정 파일(servlet-context.xml)에 <aop:aspectj-autoproxy> 앨리먼트를 선언해야 한다. 스프링 설정 파일을 클릭 후 하단의 Namespaces 탭을 클릭하여 aop을 선택하여 엘리먼트를 사용하기 위한 설정을 추가한다.



Source 탭을 클릭하여 상단에 aop 네임스페이스가 등록된 것을 확인하였다면, aop 탭으로 가서 beans를 우클릭하고 'Insert <aop:aspect-autoproxy> element'를 클릭하여 <aop:aspect-autoproxy> 엘리먼트를 추가하자.

<aop:aspect-autoproxy> 엘리먼트는 스프링 컨테이너에서 AOP 관련 어노테이션들을 인식하고 용도에 맞게 자동으로 처리해준다.



다음으로 메이븐 라이브러리 관리 파일인 pom.xml에 AOP 사용을 위한 라이브러리를 등록 한다.

```
<!-- AspectJ RunTime -->
    <dependency>
        <groupId>org.aspectj</groupId>
        <artifactId>aspectjrt</artifactId>
        <version>${org.aspectj-version}</version>
    </dependency>

<!-- AspectJ Weaver -->
<!-- weaver는 AOP에서 advice를 핵심 기능에 적용하는 설정 파일이다 -->
    <dependency>
        <groupId>org.aspectj</groupId>
        <artifactId>aspectjweaver</artifactId>
        <version>${org.aspectj-version}</version>
    </dependency>

<!-- AOP Proxy Library -->
<dependency>
        <groupId>cglib</groupId>
        <artifactId>cglib</artifactId>
        <version>2.2</version>
    </dependency>
```

해당 라이브러리가 없다면 톰캣 서버를 실행할 때 NoClassDefFoundError가 발생한다.

2.2 AOP 사용 구문

2.2.1 @Pointcut 어노테이션 표현식

Poincut 어노테이션을 사용할 때 execution 속성에 표현식을 설정해 주어야 하는데, 다음과 같은 형식으로 지정해 주어야 한다.

Pointcut 표현식 / execution(* com.kh.biz. .*Impl.*(..))			
*	com.kh.biz.	.*Impl.	*(..)
리턴 타입	패키지 경로	클래스명	메소드명 및 매개변수

: 예시의 표현식은 리턴 타입과 매개변수를 무시하고, com.kh.biz. 패키지로 시작하는 클래스 중 이름이 Impl로 끝나는 클래스의 모든 메소드를 뜻한다.

- 상세 형식 지정자

형식	설명	
리턴타입	*	모든 리턴 타입 허용
	void	리턴타입이 void인 메소드만 선택
	!void	리턴타입이 void가 아닌 메소드 선택
패키지	com.kh.test	정확하게 com.kh.test 패키지만 선택
	com.kh.test..	com.kh.test 로 시작하는 모든 패키지 선택
	com.kh.test..*impl	com.kh.test 로 시작해서 마지막 패키지 이름이 impl로 끝나는 패키지 선택
클래스	BoardServiceImpl	정확하게 BoardServiceImpl 클래스만 선택
	*Impl	클래스 이름이 Impl로 끝나는 클래스만 선택
	BoardService+	클래스 이름 뒤에 '+'가 붙으면 해당 클래스로부터 파생된 모든 자식 클래스 선택 인터페이스 뒤에 '+'가 붙으면 해당 인터페이스를 구현한 모든 클래스 선택
메소드	*(..)	가장 기본 설정으로 모든 메소드 선택
	get*(..)	메소드 이름이 get으로 시작하는 모든 메소드 선택
매개변수	(..)	가장 기본 설정으로 '..'은 매개변수의 개수와 타입에 제약이 없음을 의미
	(*)	반드시 1개의 매개변수를 가진 메소드만 선택
	(com.kh.user.UserVO)	매개변수로 UserVO를 가지는 메소드만 선택, 이때 클래스의 패키지 경로가 반드시 포함되어야 함
	(!com.kh.user.UserVO)	매개변수로 UserVO를 가지지 않는 메소드만 선택
	(Integer, ..)	한 개이상의 매개변수를 가지되, 첫번째 매개변수의 타입이 Integer 인 메소드만 선택
	(Integer, *)	반드시 두 개의 매개변수를 가지되, 첫번째 매개변수의 타입이 Integer 인 메소드만 선택

2.3 Joinpoint와 바인드 변수

횡단 관심에 해당하는 Advice 멧드를 의미있게 구현하려면 클라이언트가 호출한 비즈니스 메소드의 정보가 필요하다. 예를 들어, After Throwing 기능의 어드바이스 메소드를 구현한다고 가정하면, 예외가 발생한 비즈니스 메소드 이름이 무엇인지, 그 메소드가 속한 클래스와 패키지 정보는 무엇인지 알아야 정확한 예외 처리 로직을 구현할 수 있다.

스프링에서는 이런 정보를 이용할 수 있도록 JoinPoint 인터페이스를 제공한다.

```
import org.aspectj.lang.JoinPoint;


  private JoinPoint jp;
```

Joinpoint가 제공하는 메소드	
Signature getSignature()	클라이언트가 호출한 메소드의 시그니처 (리턴타입, 이름, 매개변수) 정보가 저장된 Signature 객체 리턴
Object getTarget()	클라이언트가 호출한 비즈니스 메소드를 포함하는 비즈니스 객체 리턴
Object[] getArgs()	클라이언트가 메소드를 호출할 때 넘겨준 인자 목록을 Object 배열로 리턴

Before, After Returning, After Throwing, After 어드바이스에서는 JoinPoint를 사용해야 하고, 유일하게 Around 어드바이스에서만 ProceedingJoinPoint 를 매개변수로 사용해야 한다. Around 어드바이스에서만 proceed() 메소드가 필요하기 때문이다.

ProceedingJoinPoint 인터페이스는 JoinPoint 의 후손으로 proceed() 메소드가 추가되었다.

2.4 @Pointcut 사용 구문

Poincut 은 공통 코드를 필요로 하는 시점을 표현식으로 작성한 곳에 개입하여 정의한 Advice를 적용한다. 작성하는 방법은 먼저 공통 기능으로 수행할 메소드를 구현한 뒤, @Pointcut 어노테이션으로 표시한다. 다음 해당 기능을 언제 수행할 것인지 표현식을 통해 내용을 작성하면 된다.

```
// com.kh.testSpring 패키지 안의 Controller 로 끝나는 클래스의 모든 메소드에 적용한다
    @Pointcut("execution(* com.kh.testSpring..*Controller.*(..))")
    public void commonPointcut() { }
```

이렇게 작성한 Pointcut은 구동 위치에 따라 Before, After, AfterReturning, AfterThrowing으로 나뉘어진 Advice에 적용하여 사용할 수 있다.

2.5 Advice 사용 구문

2.5.1 XML 설정을 통한 Advice 사용

src/main/webapp/WEB-INF/spring/appServlet/servlet-context.xml

```
01 <?xml version="1.0" encoding="UTF-8"?>
02 <beans:beans xmlns="http://www.springframework.org/schema/mvc"
03     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
04     xmlns:beans="http://www.springframework.org/schema/beans"
05     xmlns:context="http://www.springframework.org/schema/context"
06     xmlns:aop="http://www.springframework.org/schema/aop"
07     xsi:schemaLocation="http://www.springframework.org/schema/mvc
08 http://www.springframework.org/schema/mvc/spring-mvc.xsd
09         http://www.springframework.org/schema/beans
10 http://www.springframework.org/schema/beans/spring-beans.xsd
11         http://www.springframework.org/schema/context
12 http://www.springframework.org/schema/context/spring-context.xsd
13         http://www.springframework.org/schema/aop
14 http://www.springframework.org/schema/aop/spring-aop-4.3.xsd">
15
16     <!-- DispatcherServlet Context: defines this servlet's request-processing infrastructure
17 -->
18
19     <!-- Enables the Spring MVC @Controller programming model -->
20     <annotation-driven />
21
22     <!-- AOP : 공통 코드를 작성한 Advice를 구현한 클래스를 등록한다 -->
23     <beans:bean id="adviceTest" class="com.kh.testSpring.aop.common.AdviceTest" />
24
25         <!-- Aspect 설정: 어떤 Advice를 어느 Pointcut에 적용할 지 설정 -->
26     <aop:config>
27         <aop:aspect id="loggingAdvice" ref="adviceTest">
28             <aop:pointcut id="test"   <!-- 사용할 Pointcut과 해당 시점을 등록한다 -->
29                 expression="execution(* com.kh.testSpring..*Controller.*(..))" />
30
```

```
31          <!-- 해당 시점 이전에 적용할 기능을 가진 메소드를 등록한다 -->
32          <aop:before pointcut-ref="test" method="beforeMethod"/>
33
34          <!-- 해당 시점에 작업이 끝난 후 반환값을 확인하는 메소드를 등록한다
35  -->
36          <aop:after-returning pointcut-ref="test" method="afterReturningMethod"
37                  returning="returnString"/>
38
39          <!-- 해당 시점 이후에 적용할 기능을 가진 메소드를 등록한다 -->
40          <aop:after pointcut-ref="test" method="afterMethod"/>
41
42          <!-- 예외가 발생할 경우 구동시킬 메소드를 등록한다 -->
43          <aop:after-throwing pointcut-ref="test" method="afterThrowingMethod"
44                  throwing="exception"/>
45
46          <!-- 해당 시점의 직전, 직후에 구동할 기능의 메소드를 등록한다 -->
47          <aop:around pointcut-ref="test" method="aroundMethod" />
48      </aop:aspect>
49    </aop:config>
50      <aop:aspectj-autoproxy/>
51
    <context:component-scan base-package="com.kh.myWeb" />
    </beans:beans>
```

com.kh.testSpring.aop.common.AdviceTest.java

```
01  package com.kh.testSpring.aop.common;
02
03  import org.apache.commons.logging.Log;
04  import org.apache.commons.logging.LogFactory;
05  import org.aspectj.lang.JoinPoint;
06  import org.aspectj.lang.ProceedingJoinPoint;
07  import org.aspectj.lang.annotation.After;
08  import org.aspectj.lang.annotation.AfterReturning;
09  import org.aspectj.lang.annotation.AfterThrowing;
```

```
10  import org.aspectj.lang.annotation.Around;
11  import org.aspectj.lang.annotation.Aspect;
12  import org.aspectj.lang.annotation.Before;
13  import org.aspectj.lang.annotation.Pointcut;
14
15  public class AdviceTest {
16      // 로그를 기록하기 위한 logger 객체 생성
17       private static Log logger = LogFactory.getLog(AdviceTest.class);
18       public AdviceTest(){}
19       public void commonPointcut() { }
20
21      // Before Advice
22      public void beforeMethod(JoinPoint jp) throws Exception {
23          logger.info("beforeMethod () before called.....");
24       }
25
26      // After Advice
27      public void afterMethod(JoinPoint jp) throws Exception {
28          logger.info("afterMethod() called.....");
29      }
30
31      // After Returning Advice
32      public void afterReturningMethod(JoinPoint jp, String returnString) throws
33  Exception {
34          logger.info("afterReturningMethod() called.....");
35          // 호출된 메소드 반환값 출력
36          logger.info("afterReturningMethod() returnString : " + returnString);
37      }
38
39      // AfterThrowing Advice
40      public void afterThrowingMethod(JoinPoint jp, Exception exception){
41          logger.info("afterThrowingMethod() called.....");
42
```

```
43              // 발생한 예외의 메세지를 출력한다.
44              logger.warn(exception.toString());
45          }
46          // Around Advice
47          public Object aroundMethod(ProceedingJoinPoint pjp) throws Throwable {
48              logger.info("aroundMethod() before called.....");
49
50              Object result = pjp.proceed();
51              Logger.info("aroundMethod() after called.....");
52              return result;
53          }
    }
```

2.5.2 어노테이션 설정을 통한 Advice 사용

com.kh.testSpring.aop.common.AdviceTest.java

```
01  package com.kh.testSpring.aop.common;
02
03  import org.apache.commons.logging.Log;
04  import org.apache.commons.logging.LogFactory;
05  import org.aspectj.lang.JoinPoint;
06  import org.aspectj.lang.ProceedingJoinPoint;
07  import org.aspectj.lang.annotation.After;
08  import org.aspectj.lang.annotation.AfterReturning;
09  import org.aspectj.lang.annotation.AfterThrowing;
10  import org.aspectj.lang.annotation.Around;
11  import org.aspectj.lang.annotation.Aspect;
12  import org.aspectj.lang.annotation.Before;
13  import org.aspectj.lang.annotation.Pointcut;
14
15  @Aspect  // @Aspect 어노테이션을 통해 Advisor 객체를 자동으로 등록시킨다.
16  public class AdviceTest {
17       private static Log logger = LogFactory.getLog(AdviceTest.class);
18
19      // 공통으로 사용될 포인트컷을 지정
20      // com.kh.testSpring 패키지 안의 Controller 로 끝나는 클래스의 모든 메소드에
21  적용한다
22      @Pointcut("execution(* com.kh.testSpring..*Controller.*(..))")
23      public void commonPointcut() { }
24
25      // Before Advice, 위에서 정의한 commonPointcut()을 사용한다
26      @Before("commonPointcut()")
27      public void beforeMethod(JoinPoint jp) throws Exception {
28            logger.info("beforeMethod() called.....");
29       }
30
31      // After Advice
```

```
32      @After("commonPointcut()")
33      public void afterMethod(JoinPoint jp) throws Exception {
34          logger.info("afterMethod() called.....");
35      }
36
37      // After Returning Advice
38      // 반환 값을 returning으로 선언하여 받아 출력할 수 있다
39      @AfterReturning(pointcut="commonPointcut()", returning="returnString")
40      public void afterReturningMethod(JoinPoint jp, String returnString) throws
41  Exception {
42          logger.info("afterReturningMethod() called.....");
43          // 호출된 메소드 반환값 출력
44          logger.info("afterReturningMethod() returnString : " + returnString);
45      }
46
47      // AfterThrowing Advice
48     // 예외가 발생 했을 때 구동, 예외 사항을 throwing으로 선언하여 받는다
49      @AfterThrowing(pointcut="commonPointcut()", throwing="exception")
50      public void afterThrowingMethod(JoinPoint jp, Exception exception){
51          logger.info("afterThrowingMethod() called.....");
52
53         // 발생한 예외의 메세지를 출력한다.
54          logger.warn(exception.toString());
55           String methodName = jp.getSignature().getName();
56           logger.warn(methodName + "() 메소드 수행 중 예외 발생!");
57
58           if(exception instanceof IllegalArgumentException) {
59               logger.warn("부적합한 값이 입력되었습니다.");
60           }else if(exception instanceof NumberFormatException) {
61               logger.warn("숫자 형식의 값이 아닙니다.");
62           }else if(exception instanceof Exception) {
63               logger.warn("예외가 발생했습니다.");
64           }
```

```
65      }
66       // Around Advice
67      // 포인트컷을 직접 지정하여 사용한다
68      @Around("execution(* com.spring.aop.*Controller.*(..))")
69      public Object aroundMethod(ProceedingJoinPoint pjp) throws Throwable {
70          logger.info("aroundMethod() before called.....");
71
72          Object result = pjp.proceed();
73          Logger.info("aroundMethod() after called.....");
74
75          return result;
        }
    }
```

3 Advice 예제

3.1 Pointcut 클래스 작성

com.kh.testSpring.aop.common.PointcutCommon.java

```
01 package com.kh.testSpring.aop.common;
02
03 import org.aspectj.lang.annotation.Aspect;
04 import org.aspectj.lang.annotation.Pointcut;
05
06 @Aspect
07 public class PointcutCommon {
08     @Pointcut("execution(* com.kh.testSpring..*Impl.*(..))")
09     public void allPointcut() {}
10
11     @Pointcut("execution(* com.kh.testSpring..*Impl.get*(..))")
12     public void getPointcut() {}
13 }
```

3.2 Advice 등록

com.kh.testSpring.aop.common.BeforeAdviceOutAspect.java

```
01 package com.kh.testSpring.aop.common;
02
03 import org.aspectj.lang.JoinPoint;
04 import org.aspectj.lang.annotation.Aspect;
05 import org.aspectj.lang.annotation.Before;
06 import org.springframework.stereotype.Service;
07
08 @Service
09 // 의존성 주입을 위한 Service 어노테이션 삽입
10 @Aspect
11 public class BeforeAdviceOutAspect {
12     @Before("PointcutCommon.allPointcut()")
```

```
13      public void beforeLog(JoinPoint jp) {
14          String methodName = jp.getSignature().getName();
15          Object[] args = jp.getArgs();
16
17          System.out.println("[사전처리] " + methodName + "() 메소드 args 정보 : "
18          + args[0].toString());
19      }
20  }
```

com.kh.testSpring.aop.common.BeforeAdvice.java

```
01  package com.kh.testSpring.aop.common;
02
03  import org.aspectj.lang.JoinPoint;
04  import org.aspectj.lang.annotation.Aspect;
05  import org.aspectj.lang.annotation.Before;
06  import org.aspectj.lang.annotation.Pointcut;
07  import org.springframework.stereotype.Service;
08
09  @Service
10  @Aspect
11  public class BeforeAdvice {
12    // 직접 Pointcut을 선언하여 Advice를 등록할 수도 있다.
13      @Pointcut("execution(* com.kh.testSpring..*Impl.*(..))")
14      public void allPointcut() {}
15
16      @Before("allPointcut()")
17      public void beforeLog(JoinPoint jp) {
18          //메소드가 실행되기 전 공통으로 처리할 작업을 위해 사용한다.
19          String methodName = jp.getSignature().getName();
20          Object[] args = jp.getArgs();
21
22          System.out.println("[메소드 호출전 확인] : " + methodName
23                  + "() 메소드 매개변수 갯수 : " + args.length);
24          for(int i = 0; i < args.length; i++)
```

```
25              System.out.println(i + "번째 매개변수 정보 : " + args[i].toString());
26      }
27  }
```

com.kh.testSpring.aop.common.AroundLog.java

```
01  package com.kh.testSpring.aop.common;
02
03  import org.aspectj.lang.ProceedingJoinPoint;
04  import org.aspectj.lang.annotation.Around;
05  import org.aspectj.lang.annotation.Aspect;
06  import org.aspectj.lang.annotation.Pointcut;
07  import org.springframework.stereotype.Service;
08  import org.springframework.util.StopWatch;
09
10  @Service
11  @Aspect
12  public class AroundLog {
13      @Around("PointcutCommon.allPointcut()")
14      public Object aroundLosg(ProceedingJoinPoint pp) throws Throwable{
15          String methodName = pp.getSignature().getName();
16
17          StopWatch stopWatch = new StopWatch();
18          stopWatch.start();
19
20          Object obj = pp.proceed();
21
22          stopWatch.stop();
23
24          System.out.println(methodName + "() 메소드 수행에 걸린 시간 : "
25              + stopWatch.getTotalTimeMillis() + "(ms)초");
26
27          return obj;
28      }
29  }
```

com.kh.testSpring.aop.common.AfterReturningAdvice.java

```
01  package com.kh.testSpring.aop.common;
02
03  import org.aspectj.lang.JoinPoint;
04  import org.aspectj.lang.annotation.AfterReturning;
05  import org.aspectj.lang.annotation.Aspect;
06  import org.aspectj.lang.annotation.Pointcut;
07  import org.springframework.stereotype.Service;
08  import com.kh.testSpring.member.model.vo.Member;
09
10  @Service
11  @Aspect
12  public class AfterReturningAdvice {
13      @AfterReturning(pointcut="PointcutCommon.getPointcut()",
14  returning="returnObj")
15      public void afterLog(JoinPoint jp, Object returnObj) {
16          //비즈니스 메소드가 리턴한 결과 데이터를 다른 용도로 처리할 때 사용한
17  다.
18          String methodName = jp.getSignature().getName();
19
20          if(returnObj instanceof Member) {
21              Member user = (Member)returnObj;
22              if(user.getMember_id().equals("admin")) {
23                  System.out.println("--- 관리자 로그인 ---");
24              } else {
25                  System.out.println("--- "+user.getMember_id()+" 로그인 ---");
26              }
27          }
28
29          System.out.println("[메소드 리턴] " + methodName + "() 메소드 리턴값 : "
30            + returnObj.toString());
31      }
    }
```

com.kh.testSpring.aop.common.AfterAdvice.java

```
01 package com.kh.testSpring.aop.common;
02
03 import org.aspectj.lang.annotation.After;
04 import org.aspectj.lang.annotation.Aspect;
05 import org.aspectj.lang.annotation.Pointcut;
06 import org.springframework.stereotype.Service;
07
08 @Service
09 @Aspect
10 public class AfterAdvice {
11     @After("PointcutCommon.allPointcut()")
12     public void finallyLog() {
13         //예외 발생 여부에 상관없이 무조건 수행되는 어드바이스
14         System.out.println("[후 실행] 비즈니스 로직 수행 후 무조건 작동");
15     }
16 }
```

com.kh.testSpring.aop.common.AfterThrowingAdvice.java

```
01  package com.kh.testSpring.aop.common;
02
03  import org.aspectj.lang.JoinPoint;
04  import org.aspectj.lang.annotation.AfterThrowing;
05  import org.aspectj.lang.annotation.Aspect;
06  import org.aspectj.lang.annotation.Pointcut;
07  import org.springframework.stereotype.Service;
08
09  @Service
10  @Aspect
11  public class AfterThrowingAdvice {
12      @AfterThrowing(pointcut="PointcutCommon.allPointcut()", throwing="exceptObj")
13      public void exceptionLog(JoinPoint jp, Exception exceptObj) {
14          // 비즈니스 메소드 실행도중에 예외가 발생했을 때, 공통적인 예외처리 로
15  직을
16          // 제공할 목적으로 사용하는 어드바이스이다.
17          String methodName = jp.getSignature().getName();
18          System.out.println(methodName + "() 메소드 수행 중 예외 발생!");
19
20
21          if(exceptObj instanceof NumberFormatException) {
22              System.out.println("숫자 형식의 값이 아닙니다.");
23          }else if(exceptObj instanceof Exception) {
24              System.out.println("예외가 발생했습니다.");
25              System.out.println(exceptObj.getMessage());
26          }
27      }
    }
```

3.3 컨트롤러 작성

com.kh.testSpring.member.controller.MemberController.java

```
01  package com.kh.testSpring.member.controller;
02
03  import org.springframework.beans.factory.annotation.Autowired;
04  import org.springframework.stereotype.Controller;
05  import org.springframework.web.bind.annotation.RequestMapping;
06  import org.springframework.web.bind.annotation.RequestMethod;
07  import org.springframework.web.servlet.ModelAndView;
08
09  import com.kh.testSpring.member.model.service.MemberService;
10  import com.kh.testSpring.member.model.vo.Member;
11
12  @Controller
13  public class MemberController {
14      @Autowired
15      MemberService memberService;
16
17      @RequestMapping(value = "aopTest.do", method = RequestMethod.POST)
18      public ModelAndView memberLoginService(Member m, ModelAndView model) {
19
20          m = memberService.loginMember(m);
21
22          if(m != null) {
23              model.addObject("user", m);
24              model.setViewName("loginConfirm");
25          } else {
26              model.setViewName("errorPage");
27          }
28          return model;
29      }
30  }
```

3.4 Service 작성

com.kh.testSpring.member.model.service.MemberService.java

```
01 package com.kh.testSpring.member.model.service;
02
03 import com.kh.testSpring.member.model.vo.Member;
04
05 public interface MemberService {
06     Member loginMember(Member m);
07 }
```

com.kh.testSpring.member.model.service.MemberServiceImpl.java

```
01 package com.kh.testSpring.member.model.service;
02
03 import org.springframework.beans.factory.annotation.Autowired;
04 import org.springframework.stereotype.Service;
05
06 import com.kh.testSpring.member.model.dao.MemberDao;
07 import com.kh.testSpring.member.model.vo.Member;
08
09 @Service("memberService")
10 public class MemberServiceImpl implements MemberService {
11     @Autowired
12     MemberDao memberDao;
13
14     @Override
15     public Member loginMember(Member m) {
16         return memberDao.memberSelect(m);
17     }
18 }
```

3.5 Dao 작성

com.kh.testSpring.member.model.dao.MemberDao.java

```
01 package com.kh.testSpring.member.model.dao;
```

```
02
03  import java.util.HashMap;
04
05  import org.springframework.stereotype.Repository;
06
07  import com.kh.testSpring.member.model.vo.Member;
08
09  @Repository("memberDao")
10  public class MemberDao {
11      private HashMap<String, Member> mmap;
12      public MemberDao() {
13          mmap = new HashMap<String, Member>();
14          mmap.put("admin",                new                Member("admin","admin","관리자
15  ","admin@test.com","010-1111-2222","서울시 강남구"));
16          mmap.put("user11",               new                Member("user11","user11","김유신
17  ","user11@test.com","010-2222-3333","서울시 강남구"));
18          mmap.put("user22",               new                Member("user22","user22","신사임당
19  ","user22@test.com","010-3333-4444","서울시 강남구"));
20          mmap.put("user33",               new                Member("user33","user33","홍길동
21  ","user33@test.com","010-4444-5555","서울시 강남구"));
22      }
23
24      public Member memberSelect(Member m) {
25          Member result = mmap.get(m.getMember_id());
26          String oriPwd = result.getMember_password();
27          String reqPwd = m.getMember_password();
28          if (!(result !=   null && oriPwd.equals(reqPwd))) {
29              result = null;
30          }
31          return result;
32      }
33  }
```

3.6 VO 작성

com.kh.testSpring.member.model.vo.Member.java

```
01  package com.kh.testSpring.member.model.vo;
02
03  import org.springframework.stereotype.Component;
04
05  @Component
06  public class Member implements java.io.Serializable {
07      private static final long serialVersionUID = 1111L;
08
09      private String member_id;
10      private String member_password;
11      private String member_name;
12      private String member_email;
13      private String member_phone;
14      private String member_address;
15
16      public Member() { }
17
18      public Member(String member_id) {
19          this.member_id = member_id;
20      }
21
22      public Member(String member_id, String member_password, String member_name,
23              String member_email, String member_phone, String member_address) {
24          this.member_id = member_id;
25          this.member_password = member_password;
26          this.member_name = member_name;
27          this.member_email = member_email;
28          this.member_phone = member_phone;
29          this.member_address = member_address;
30      }
31
```

```
32      public Member(String member_id, String member_password) {
33          this.member_id = member_id;
34          this.member_password = member_password;
35      }
36        /**********  Getter & Setter 구현  **********/
37        /**********  toString Override 구현  **********/
38      @Override
39      public String toString() {
40          return "Member [member_id=" + member_id + ", member_password="
41                      + member_password + ", member_name=" + member_name
42                      + ", member_email=" + member_email + "]";
43      }
44  }
```

3.7 View 작성

src/main/webapp/index.html

```
01 <!DOCTYPE html>
02 <html>
03 <head>
04 <meta charset="UTF-8">
05 <title>Aop Test</title>
06 </head>
07 <body>
08     <h1>Spring AOP Test 페이지</h1>
09     <hr>
10     <form action="aopTest.do" method="post">
11         아이디 : <input type="text" name="member_id"><br>
12         비밀번호 : <input type="password" name="member_password"><br>
13         <button type="submit">로그인</button>
14     </form>
15 </body>
16 </html>
```

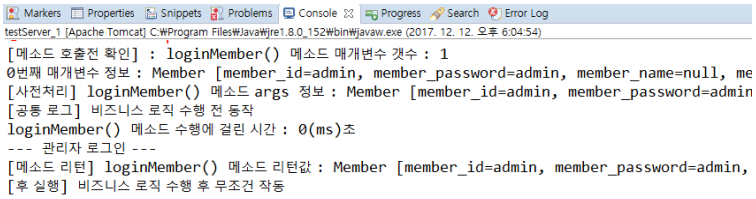
src/main/webapp/WEB-INF/views/loginConfirm.jsp

```
01 <%@ page language="java" contentType="text/html; charset=UTF-8"
02     pageEncoding="UTF-8"%>
03 <%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
04 <!DOCTYPE html>
05 <html>
06 <head>
07 <meta charset="UTF-8">
08 <title>Login Confirm</title>
09 </head>
10 <body>
11     <c:if test="${!empty user}">
12         ${user.member_id}님 접속이 확인 되었습니다.<br>
13         ID : ${user.member_id}<br>
14         Name : ${user.member_name}<br>
```

```
15          Tell : ${user.member_phone}
16      </c:if>
17  </body>
18  </html>
```

해당 프로젝트를 구현 후 실행하였을 때, 이클립스의 콘솔창을 통해 각각의 Advice가 구동 되는 것을 알 수 있다.

- 정상 실행일 경우



- 예외가 발생한 경우

